

Data Engineering on Google Cloud — Day 1 of 2

Iowa Department of Management, Department of IT · October 5, 2026 · Student slide pack

- Slides follow the class agenda
- Hands-on activities: <https://roittraining.github.io/gcp-demos/bigquery.html>
- Labs are in the Google Skills classroom

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 1 OF 135 · COVER & AGENDA

FACILITATOR NOTE

- Welcome the group and set the frame: two days, one story — migrate the SQL Server warehouse to BigQuery and kill the legacy extracts.
- Point at the two logistics lines on the slide: hands-on activities live at the [roittraining.github.io](https://roittraining.github.io/gcp-demos/bigquery.html) demos page, and labs run in the Google Skills classroom — have everyone log in now so account issues surface before the 10:40 lab.
- Slides follow the class agenda, so this deck doubles as the roadmap for both days.

Flow: Opening slide of Day 1; next is the 09:00 Welcome block (audience survey + the two-day migration reframe).

Agenda — Monday, October 5 (1 of 2) · morning

09:00–09:25	Welcome, audience survey & the reframe — your two-day migration story	
09:25–10:10	Module 1 — Data Engineering Tasks & Components + GCP platform orientation	
10:10–10:25	Exploring projects, datasets & tables — star public projects, create your class dataset	DIN
10:25–10:40	Morning break	
10:40–11:10	Loading Data into BigQuery — the first lab, a confidence builder	LAB
11:10–11:50	Module 2 — Data Replication & Migration: killing the nightly extracts	

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 2 OF 135 · COVER & AGENDA

Hook

You know the SSIS/SQL Agent stack cold — this morning maps each piece to GCP, and by 11:50 you've started killing the nightly extracts.

Say · cover

- Hands-up survey first: ETL devs vs M&O, who's opened the GCP console — pitch depth off that, not the slides.
- M1's sources/transforms/sinks maps 1:1 onto their SSIS data-flow mental model — say so.
- 10:10 DIN is load-bearing: star bigquery-public-data + roi-bq-demos, create the `class` dataset — the 13:45 loading DINs depend on it.
- The 10:40 load lab is a deliberate confidence builder — first BigQuery win for console novices.
- M2 frames Datastream as the extract-killer (CDC, not nightly dumps); the lab splits across lunch so replication gets wall-clock time.

Iowa tie

The morning is their migration in miniature: land files in BigQuery (Bronze habit), then replace nightly SQL Server extracts with Datastream CDC.

Design patterns

- Small hands-on win before the conceptual module — confidence compounds.
- Split CDC labs across lunch; don't fill replication wait-time with slideware.

Anti-patterns

- Don't define ETL vs ELT — senior room; spend the time on where things live in the console.
- Don't skip class-dataset creation; it silently breaks the afternoon DINs.

History & what's new

Datastream went GA Nov 2021 and added BigQuery destinations Sept 2022; SQL Server as a source GA'd July 2024 — the headline for this room — and partial backfill (WHERE-filtered subset loads) shipped Sept 30, 2026.

Flow

First agenda card after the cover; next is the 09:00 Welcome (survey + reframe).

Links

<https://cloud.google.com/bigquery/docs/introduction>
<https://cloud.google.com/datastream/docs/overview>
<https://cloud.google.com/bigquery/docs/loading-data>

Agenda — Monday, October 5 (2 of 2) · afternoon

11:50–13:15	Datastream: PostgreSQL → BigQuery — start the stream, lunch while it replicates, validate the CDC moment (lunch 12:00–13:00)	LAB
13:15–13:45	Module 3 — Extract & Load: the pattern for data that still arrives as files	
13:45–14:30	Loading block — console & CLI load, the autodetect pitfall, CSV vs Parquet race, LOAD DATA SQL (break 14:30–14:45)	DIN
14:45–15:25	Module 4 — Extract, Load & Transform (ELT) + the Dataform walkthrough: your Silver→Gold tool	
15:25–16:05	Create & Execute a SQL Workflow in Dataform — the anchor lab	LAB
16:05–16:30	Modules 5+6 compressed: ETL options & automation — then the scheduled-query closing demo	DEMO

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 3 OF 135 · COVER & AGENDA

Hook

After lunch you build the core of your target state: a live CDC stream, four ways to land files, then Dataform — your Silver->Gold tool.

Say · cover

- Resume the Datastream lab at the CDC moment: change a row in PostgreSQL, watch it surface in BigQuery — the extract-killer proof.
- M3 + loading block: for data still arriving as files — console & CLI loads, the autodetect pitfall, CSV vs Parquet, LOAD DATA SQL.
- Jeff's dataform-demo walkthrough is the centerpiece: compiled graph, assertions, release configs -> git — their CI/CD discipline translates.
- 15:25 Dataform SQL-workflow lab is the day's anchor — full slot; SQL people fly here.
- M5+M6 compressed: Scheduler ~ SQL Agent, Composer ~ orchestrator (M&O cares most); close with the one-click scheduled query.

Iowa tie

Afternoon = their Silver->Gold story: Dataform ELT builds the marts Power BI queries, plus file-landing patterns for extracts that can't die yet.

Design patterns

- Prefer Parquet over CSV for typed, self-describing loads.
- Treat Dataform release/workflow configs as the dev->prod promotion path, mirroring existing CI/CD.

Anti-patterns

- Don't let autodetect define production schemas — declare them.
- Don't squeeze the Dataform lab for M5/M6; spillover moves to Day-2 09:00.

History & what's new

Google acquired Dataform Dec 2020; it entered Google Cloud preview in 2022 and is still shipping: the Dataform MCP server (pipeline authoring + git ops) went GA Sept 21, 2026, Knowledge Catalog metadata enrichment Oct 1, 2026.

Flow

Second agenda card; next is 09:00 Welcome. This page resumes 13:00 with Datastream lab part 2.

Links

<https://cloud.google.com/dataform/docs/overview>
<https://cloud.google.com/bigquery/docs/reference/standard-sql/load-statements>
<https://cloud.google.com/bigquery/docs/scheduling-queries>

Welcome — who this course is for

Iowa DOM DoIT data team · Oct 5–6, 2026 · 9:00 AM–4:30 PM CT

- ✓ ETL developers living in SQL Server and SSIS— sources, transforms, sinks are your daily work
- ✓ A dedicated M&O team— you will own what gets designed here, so operations questions are welcome
- ✓ Power BI report authors— the last mile matters: your Gold marts feed the reports you already have
- ✓ Near-zero GCP console experience across the room— that is expected, and this course is built for it
- ✓ Senior in data engineering, new to one console— we compress concepts, slow down on where things live

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 4 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Name the room first: ten people who ship SSIS packages daily, near-zero GCP console time — that is the design point, not a gap.

Say · cover

- Read each audience line aloud and watch for nods: ETL developers, the M&O team, Power BI authors — each gets called on later today.
- State the deal explicitly: concepts move fast because they know ETL; platform moves slow because nobody has opened the console.
- Promise the through-line: every demo, DIN and lab hangs on their real migration — no generic cloud tour, no toy examples.
- Tell M&O directly: operations questions are first-class here, and the 16:05 automation block is built for them.

Iowa tie

The whole project team is here — the people who build the extracts, the people who inherit their replacement, and the people whose reports sit on top.

Design patterns

- Calibrate depth to the audience in the first ten minutes
- Name each role explicitly so nobody wonders whether the course is for them

Anti-patterns

- Defining ETL or ELT to people who write pipelines daily
- Running a generic cloud-101 intro for senior data engineers

History & what's new

BigQuery has been Google's serverless warehouse since 2011; in Jan 2025 Google put a managed Apache Iceberg engine into preview — exactly the Day-2 lakehouse topic.

Flow

First slide of the 09:00 welcome block, right after cover/agenda; next is the vendor title card, then your bio, then the reframe.

Links

<https://cloud.google.com/bigquery/docs/introduction>
<https://www.skills.google/ilt/classrooms/37990>
<https://github.anum.cloud>

Introduction to Data Engineering on Google Cloud

Name
Role

INSTRUCTOR NOTES — PAGE 5 OF 135 · 09:00 · WELCOME & COURSE INTRO

FACILITATOR NOTE

- Vendor title card — keep it under 30 seconds: name, role, and straight on to your bio slide, which does the real credibility work.
- If asked: this is Google's official Introduction to Data Engineering on Google Cloud course, wrapped with Iowa-specific custom slides.

Flow: Sits between the audience slide and the instructor bio; etiquette and the migration reframe follow.

Your instructor — Imran Ahmad, PhD

Google Cloud Authorized Trainer · AWS Authorized Instructor ·
Microsoft Certified Trainer

- Large-scale data engineering is the craft — production pipelines over high-volume, multi-source datasets at national scale, across Google Cloud, AWS, and Azure
- PhD in data processing — created a large-scale data-processing algorithm on cloud/grid computing; pioneered techniques applied to data-intensive science, including neutrino-particle experiments
- Taught data engineering and cloud for Citibank, JP Morgan, the United Nations, Deloitte, and the European Commission
- Second act: AI and agents — author of 30 Agents Every AI Engineer Must Build and 50 Algorithms Every Programmer Should Know (Packt)
- Fluent in your world and your target — SQL Server and SSIS to BigQuery; these two days map that craft onto your migration



Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 6 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Credibility in ninety seconds: one sentence on scale, one on who you have taught, one on why SQL Server to BigQuery is home turf — then move on.

Say · cover

- Do not read the slide; the last bullet is the one to land — fluent in their world (SQL Server, SSIS) and in their target (BigQuery).
- Translate the bio into their vocabulary: production pipelines over high-volume, multi-source data — SSIS problems at a bigger blast radius.
- Offer break-time access for migration-specific questions; the parking lot captures whatever the schedule cannot.
- The books get one line, and only if asked — the room is here for BigQuery, not a bibliography.

Iowa tie

Position yourself as the translation layer between their Microsoft stack and GCP — the two-day mapping they keep using after you leave.

Design patterns

- Establish credibility fast, then shift attention to the students' project within two minutes

Anti-patterns

- Reading a bio slide bullet-by-bullet
- Promising anything on slide one that you cannot demo by 16:30 today

History & what's new

Google certifies instructors per course through its Authorized Trainer program; in Oct 2025 it consolidated all training — including this classroom — into the new Google Skills platform.

Flow

Bio done; a quick etiquette slide is next, then the reframe — the centerpiece of this block.

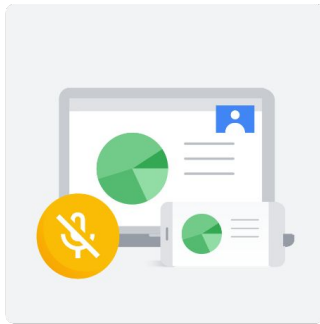
Links

<https://cloud.google.com/learn/training>

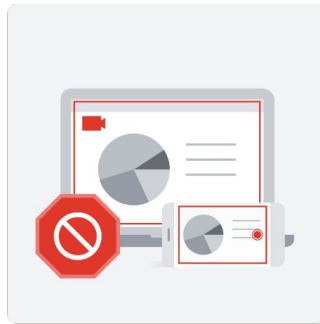
<https://cloud.google.com/blog/topics/training-certifications/google-skills-new-home-ai-learning>

<https://www.skills.google>

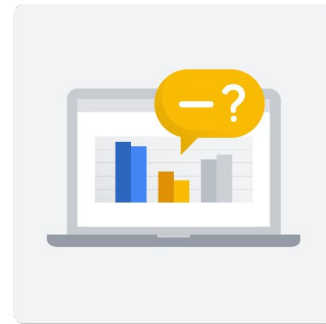
Etiquette



Mute microphone



No recording



Ask questions

Google Cloud

INSTRUCTOR NOTES — PAGE 7 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Three icons, one message: this is a working session, not a webinar — interrupting me is a feature, so say so explicitly.

Say · cover

- Mute when silent, but prefer voice questions over chat — sixteen people is small enough for real conversation.
- No recording: migration specifics and lab credentials stay in the room; saying it out loud makes people ask freely.
- Chat feeds the parking lot: drop questions any time; they get answered visibly at breaks or Day-2 flex time.
- Project-specific questions — their tables, their timelines — are the best kind; park them rather than derail the block.

Iowa tie

With go-live close, their sharpest questions will be about their own extract jobs — the parking lot is the promise those get answered, not lost.

Design patterns

- Name the question channel in the first ten minutes
- Answer parking-lot items visibly at every break

Anti-patterns

- Letting the first hour run as monologue — it sets the tone for both days

History & what's new

Google's instructor-led labs ran on Qwiklabs (acquired 2016), which became Cloud Skills Boost and was relaunched as Google Skills in Oct 2025 — the classroom this course runs on.

Flow

Housekeeping closed; the next slide is the reframe — the most important ninety seconds of the morning.

Links

<https://www.skills.google/it/classrooms/37990>

<https://github.anum.cloud>

<https://roitraining.github.io/gcp-demos/bigquery.html>

The reframe: two days = your migration story

Not a generic Google Cloud tour — every module hangs on the project you are actually running.

- Kill the nightly extracts— Datastream change data capture replicates continuously, no extract window
- Land in BigQuery— serverless, no instance, no patching; your SQL Server DW's replacement
- Bronze / Silver / Gold— the medallion lakehouse from your own architecture doc, taught on Day 2
- Dataform for ELT— SQL workflows and release configs; your SSIS-in-git CI/CD discipline translates
- Power BI last mile— Gold marts feed the reports you already have (Day 2 afternoon)
- Single source of truth— BigQuery becomes the analytics source; SQL Server stays system of record

Every demo, DIN activity and lab in the next two days hangs on one of these steps.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 8 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Everything for two days hangs on one sentence: this course IS your SQL Server to BigQuery migration, taught in your order, with your vocabulary.

Say · cover

- Walk the six steps left to right; each maps to a timed block — Datastream 11:10, loading 13:15, Dataform 14:45, medallion and Power BI tomorrow.
- Kill-the-extracts first: Datastream change data capture replicates continuously — no extract window, no nightly batch.
- BigQuery is serverless: no instance, no patching, no version upgrades — contrast with caring for a SQL Server box.
- Bronze/Silver/Gold is their own architecture doc's language; Google's Day-2 slides teach the same medallion.
- Land the boundary: SQL Server stays the system of record; BigQuery becomes the analytical single source of truth.

Iowa tie

This slide IS their project plan — the architecture doc's Bronze-to-Gold flow with Dataform ELT and the Power BI last mile, compressed into sixteen hours.

Design patterns

- Anchor every new service to the migration step it serves before teaching its features

Anti-patterns

- Product-by-product tour in deck order — it loses a project-focused room by ten o'clock

History & what's new

Datastream's BigQuery destination went GA in Apr 2023; the SQL Server source — their exact path — followed in Jul 2024, with native BigQuery partitioning and clustering for streams arriving in early 2026.

Flow

The reframe frames both days; next, the hands-up survey calibrates depth before the two agenda slides.

Links

<https://cloud.google.com/datastream-for-bigquery>

<https://cloud.google.com/datastream/docs/configure-your-source-sql-server-database>

<https://cloud.google.com/dataform/docs/overview>

Hands up — a quick survey before we start

Your answers set the depth for the next two days. Calibration, not a quiz.

- ETL developers— who builds and maintains the SQL Server / SSIS packages today?
- M&O team— who keeps it running and will inherit whatever we design here?
- Power BI authors— who builds the reports sitting on top of the marts?
- Who has opened the Google Cloud console— even once, even just to click around?
- Who has written a query in BigQuery, anywhere, ever?

Answer honestly, including 'never opened it' — the room is senior in DE, new to this platform.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 9 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Calibration, not a quiz — their hands decide how fast the next two days move, so say that out loud before the first question.

Say · cover

- Ask each question, pause, count hands out loud — the counting buys you cold-call permission for the rest of the course.
- The two that matter most: who has opened the console, and who has ever run a BigQuery query — expect near-zero, and say that is expected.
- Mark the M&O hands: they are your audience at 16:05 automation and in tomorrow's governance block.
- Mark Power BI authors by name if you can; promise them tomorrow at 14:50 explicitly.
- Anyone with real GCP time gets recruited as a table expert for the labs.

Iowa tie

The survey doubles as team inventory — who owns which extract job, which feeds the Day-2 capstone mapping at 15:20.

Design patterns

- Calibrate publicly, then quote the results back when pitching depth

Anti-patterns

- Skipping the survey to save five minutes — you fly blind for two days

History & what's new

The console they will admit to never opening is BigQuery Studio — GA since 2024, when Google folded SQL, loading and notebooks into one web IDE.

Flow

Survey results in hand; walk the Day-1 agenda block by block on the next slide.

Links

<https://cloud.google.com/bigquery/docs/bigquery-web-ui>

<https://survey.anum.cloud>

<https://www.skills.google/ilt/classrooms/37990>

Day 1 — Landing your data in BigQuery

09:00–09:25	Welcome, hands-up survey, the reframe (this block)	
09:25–10:25	M1 DE tasks & components + console orientation — create your class dataset	DIN
10:40–11:10	Lab: Loading Data into BigQuery — your first win in a new console	LAB
11:10–13:15	M2 killing the nightly extracts — Datastream CDC lab runs across lunch	LAB
13:15–14:30	M3 extract & load + four loading DINs — autodetect pitfalls, CSV vs Parquet	DIN
14:45–16:30	M4 ELT + Dataform demo and anchor lab; M5/M6 compressed; day close	LAB

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 10 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Day 1 has one job: by 16:30 their data has landed in BigQuery four different ways — and one of them killed the nightly extract.

Say · cover

- Flag the anchors now: the Datastream lab spanning lunch — their migration in miniature — and the 15:25 Dataform lab, their Silver-to-Gold tool.
- The 10:10 console orientation is not optional: everyone creates the class dataset, and every afternoon DIN depends on it.
- 13:45 loading block: four ways to land files, including the autodetect failure reproduced on purpose.
- M5/M6 compressed at 16:05 is deliberate — M&O gets automation vocabulary, not another console marathon.
- Closing demo is five minutes: one scheduled query — automation made concrete before they leave.

Iowa tie

Today's sequence mirrors their migration sequence: land files in Bronze, replicate the database continuously, transform Silver to Gold with Dataform.

Design patterns

- Preview anchor labs at the start of the day so students pace energy and questions

Anti-patterns

- Reading the agenda as a list — sell the through-line instead

History & what's new

Dataform — the 15:25 anchor — went GA in BigQuery in Jun 2023; the Nov 2025 release notes added GA automation for BigLake Iceberg table creation.

Flow

Day 1 framed; the next slide previews Day 2, then three vendor slides cover the lab platform.

Links

<https://cloud.google.com/datastream-for-bigquery>
<https://cloud.google.com/dataform/docs/overview>
<https://cloud.google.com/bigquery/docs/loading-data>

Day 2 — Lakehouse & medallion, your target model

09:00–09:15	Recap quiz — which GCP tool replaces which piece of your current stack	QUIZ
09:15–10:30	M7 lake vs warehouse vs lakehouse + M8 building a data lakehouse	
10:45–12:00	Federated query lab, external tables (Bronze made tangible), M9 modernizing DWs	LAB
13:00–14:35	Iceberg lab + deep-dive; M10 governance + Google's own Bronze/Silver/Gold slides	LAB
14:50–15:20	M11 compressed — BQML direction + the Power BI to BigQuery last mile	DEMO
15:20–16:30	Capstone workshop: map one of your real extract jobs; summary and Q&A	WORKSHOP

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 11 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Day 2 is their target operating model: the lakehouse and medallion vocabulary from their own architecture doc, taught with Google's slides.

Say · cover

- Morning builds the foundation: lake vs warehouse vs lakehouse, then GCS, Iceberg and federated queries — Bronze made tangible.
- 14:00 M10 is the centerpiece: governance plus Google's own Bronze/Silver/Gold slides — tell them to photograph those.
- 14:50 is the last mile: Gold marts feeding the Power BI reports they already have — the project's definition of done.
- 15:20 capstone: everyone maps one real extract job from the pre-course survey onto the new stack.
- Mention the follow-on three-day BigQuery data-warehousing course now being scheduled.

Iowa tie

Day 2 literally teaches their architecture document — medallion, Dataform ELT, Power BI — so tell them to bring the doc and annotate it.

Design patterns

- End each day naming tomorrow's first block; start each day with a recap quiz

Anti-patterns

- Letting morning theory eat the capstone — protect the 15:20 slot

History & what's new

The Day-2 lakehouse story is young: BigQuery's managed Apache Iceberg engine only entered preview in Jan 2025 and has been going GA piece by piece since.

Flow

Both days framed; next, three vendor slides on the Google Skills lab platform, starting with what a lab gives you.

Links

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>

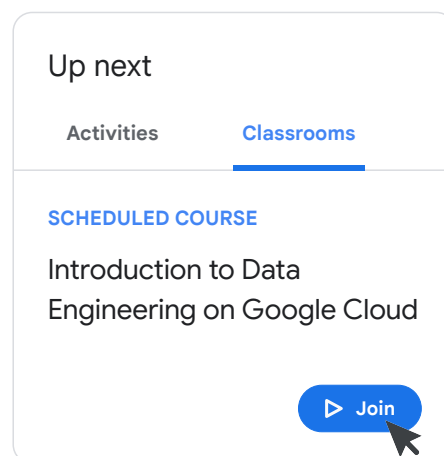
<https://cloud.google.com/bigquery/docs/introduction>

Open Google Skills

- 1 Open an incognito window (or private and/or anonymous window).
- 2 Go to the URL your instructor provides.
- 3 Sign In with existing account or Join with new account (with email you used to register for the course).
- 4 Launch the course from the **Dashboard** page.

Access issues

The process to open Google Skills can differ based on credentials used. Please reach out to your instructor if you have any access issues.



Google Cloud

INSTRUCTOR NOTES — PAGE 13 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Ninety seconds from now every seat is inside the classroom — do this together, before content starts, while help is cheap.

Say · cover

- Everyone, with me: incognito window, the classroom URL, Join, then sign in with the email used to register.
- Incognito is not paranoia — it dodges cached personal and agency Google accounts, the classic login loop.
- Hand up now if stuck, not at 10:40 — access problems are two-minute fixes at 09:15 and lab-burners later.
- Signing in with a different email than the registration email is the number-one Join failure — check that first.
- Success looks like the course card and a Join button that becomes your class dashboard — home base for two days.

Iowa tie

DOM DoIT machines may carry agency or personal Google sessions; the incognito habit transfers directly to their real tenant work later.

Design patterns

- Verify every seat's access in the first half hour, before any content depends on it

Anti-patterns

- Deferring sign-in to the first lab — sixteen simultaneous login problems burn the lab slot

History & what's new

Per-lab temporary credentials are the Qwiklabs model Google has kept since the 2016 acquisition, carried through the Oct 2025 Google Skills relaunch.

Flow

Signed in; the next two slides show where labs and lecture notes live inside this dashboard.

Links

<https://www.skills.google/ilt/classrooms/37990>

<https://www.skills.google>

<https://cloud.google.com/blog/topics/training-certifications/google-skills-new-home-ai-learning>

View your labs

Do **NOT** launch a lab until instructed to do so!

Labs	Lecture Notes
✓	
○	
⚠ Lab Currently Disabled	
⚠ Lab Currently Disabled	

← Lab completed

← To be completed

← Not yet available

Google Cloud

INSTRUCTOR NOTES — PAGE 14 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

This dashboard is the next two days at a glance — and the one rule printed on it saves us an hour of pain.

Say · cover

- Labs tab lists every lab with its state: completed, to be completed, or not yet available.
- The rule: never launch a lab until told — launching early starts the timer and can strand you mid-step.
- 'Lab Currently Disabled' is normal; labs unlock as the class reaches them.
- During the first lab, roam the states: a grey lab at 11:00 means someone launched nothing — help them then, not at the debrief.
- Point out the Lecture Notes tab beside Labs — that is the next slide.

Iowa tie

The 11:50 Datastream lab is deliberately split across lunch — they launch Part 1 only when told, at 11:50 sharp.

Design patterns

- Instructor controls lab-launch timing; students watch the state column, not the clock

Anti-patterns

- Early launches — an expired timer mid-lab is unrecoverable

History & what's new

Per-seat completion tracking comes from the Qwiklabs platform Google acquired in 2016, carried through Cloud Skills Boost into Google Skills in Oct 2025.

Flow

Labs located; the next slide shows where the downloadable lecture notes live.

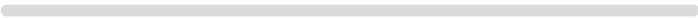
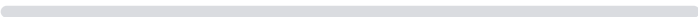
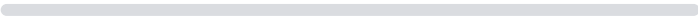
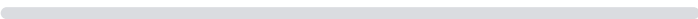
Links

<https://www.skills.google/ilt/classrooms/37990>

<https://www.skills.google>

<https://cloud.google.com/blog/topics/training-certifications/google-skills-new-home-ai-learning>

View lecture notes

Labs	Lecture Notes
01	 
02	 
03	 
04	 

You can download these as PDF files

Google Cloud

INSTRUCTOR NOTES — PAGE 15 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Every module's slides are already in the classroom as PDFs — nobody needs to photograph a screen today.

Say · cover

- Lecture Notes tab, numbered modules, download any as PDF — show it once, live.
- These PDFs are the vendor-original slides; the Iowa custom slides live in the student pack, not the classroom.
- The student pack at github.anum.cloud has the same modules plus Iowa-specific companions — that is the better copy.
- Notes persist in the classroom for two years, so annotate and listen instead of transcribing.

Iowa tie

Their pack adds what the vendor deck lacks: per-module migration mapping to Bronze/Silver/Gold, Dataform and the Power BI last mile.

Design patterns

- Hand out the deck up front so attention goes to the demo, not to copying

Anti-patterns

- Withholding slides 'so they pay attention' — it just produces photographers

History & what's new

Google relaunched its training platform as Google Skills in Oct 2025; per-module PDF notes inside the classroom are the current delivery model for ILT courses like this one.

Flow

Notes located; the next slide covers what happens to all this material once class ends — the two-year window.

Links

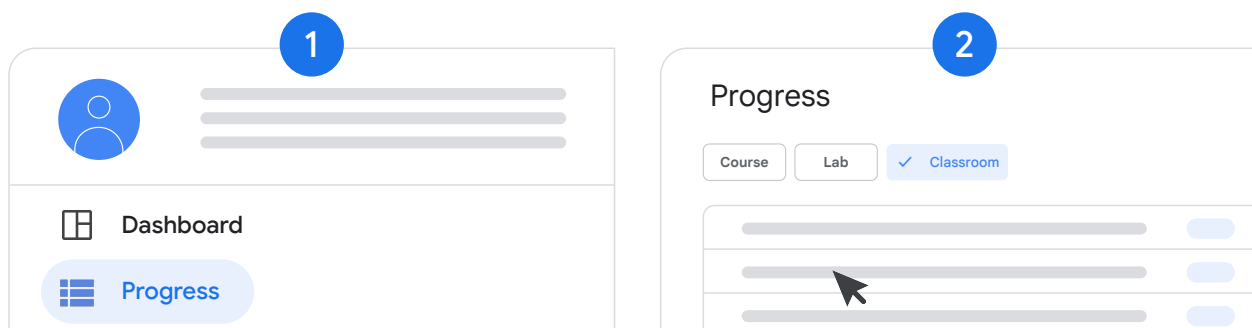
<https://www.skills.google/ilt/classrooms/37990>

<https://github.anum.cloud>

<https://www.skills.google>

End of class - Materials

Materials are available for 2 years



Click on your **Profile** and select **Progress**.

Select the class from the **Classroom** list.

Google Cloud

INSTRUCTOR NOTES — PAGE 16 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Everything survives the class: two years of access, and the way back is Profile, then Progress.

Say · cover

- Walk the path once: profile icon, Progress, pick this classroom — labs and notes reappear exactly as left.
- The two-year window covers their entire migration timeline — nothing to bookmark, nothing to download in a panic.
- Lab projects themselves die at timer expiry; the instructions and notes are what persist.
- If someone loses the classroom link entirely, signing in at skills.google and opening Progress finds the class again.
- Completion checkmarks double as a training record if anyone needs credit.

Iowa tie

Migration runs well past 2026 — the two-year shelf life means the Dataform lab steps are still reachable at go-live.

Design patterns

- Capture durable references — docs links and the pack — not screenshots of ephemeral consoles

Anti-patterns

- Treating the lab project as persistent — it deletes on schedule, so export nothing into it

History & what's new

Google relaunched the platform as Google Skills in Oct 2025, keeping the post-class access model; the two-year material window on this slide is the current policy.

Flow

Platform tour done; the next slide is Iowa-specific logistics and ground rules.

Links

<https://www.skills.google/ilt/classrooms/37990>

<https://www.skills.google>

<https://cloud.google.com/blog/topics/training-certifications/google-skills-new-home-ai-learning>

Logistics and ground rules

- Google Skills classroom—
<https://www.skills.google/ilt/classrooms/37990>
— all lab steps live there
- Temporary lab projects— each lab gives a throwaway GCP project plus a lab timer; nothing bills to you
- Labs are safe to break— disposable projects exist for experiments; break things freely
- Lunch 12:00–1:00 PM CT— on Day 1 the Datastream lab keeps replicating while we eat
- Breaks— 15 minutes mid-morning and mid-afternoon, both days
- Parking lot— off-track questions get written down and answered at breaks or Day 2 flex time
- Pre-course survey— fill your row if you have not yet: role, 0–3 self-ratings, one extract job you own (link in the chat and the student pack)

Keep two browser tabs ready each morning: the Skills classroom and the Google Cloud console.
Pre-course survey: survey.anum.cloud — student pack: github.anum.cloud

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 17 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Six ground rules that make two virtual days run like one well-run project meeting — read them as commitments, not admin.

Say · cover

- Two tabs pinned every morning: the Skills classroom and the Google Cloud console — have them do it now.
- Labs are disposable: nothing bills, everything resets — experiment aggressively and break things freely.
- Lunch 12:00-13:00 CT; the Datastream stream keeps replicating while they eat — that is the lesson, so do not stop it.
- Parking lot: off-track questions get written down and answered at breaks or Day-2 flex — visibly, every time.
- Pre-course survey: role, self-ratings, one extract job they own — the Day-2 capstone consumes that row.

Iowa tie

The survey's 'one extract job you own' answer becomes raw material for tomorrow's 15:20 capstone mapping exercise.

Design patterns

- Parking lot with visible follow-through at every break
- Two-pinned-tabs rule for virtual deliveries

Anti-patterns

- Answering deep project questions inline — it detonates the schedule for fifteen other people

History & what's new

Timed, disposable lab projects have been Google's instructor-led model since the Qwiklabs days (2016 acquisition); Google Skills kept the design in the Oct 2025 relaunch.

Flow

Ground rules set; the next slide puts the survey and the student pack physically in their pockets.

Links

<https://www.skills.google/ilt/classrooms/37990>

<https://survey.anum.cloud>

<https://github.anum.cloud>

Take these with you — scan any code

Everything from these two days, in your pocket before you leave the room.



Pre-course survey

survey.anum.cloud



Student pack — notes, slides, labs

github.anum.cloud

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 18 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Two QR codes: one tells me who is in the room, the other is everything I just promised you — scan both now.

Say · cover

- survey.anum.cloud takes three minutes: role, zero-to-three self-ratings, one extract job you own — first break is the deadline.
- github.anum.cloud is the student pack: these slides, lab companions, DIN guides, both days — the artifact that outlives the class.
- Phone cameras work; both links are also in chat and on the previous slide — three channels, zero excuses.
- Watch who scans; anyone without a survey row by 10:25 gets a gentle nudge at break — the capstone needs every row.

Iowa tie

Survey answers drive tomorrow's capstone and shape the follow-on three-day course; the pack is what they hand to teammates who could not attend.

Design patterns

- Put key resources in the room three ways — QR, chat, slide

Anti-patterns

- Links that live only inside a deck they cannot click

History & what's new

Short-URL and QR joins are now standard for Google's classrooms; the platform behind them relaunched as Google Skills in Oct 2025, consolidating roughly 3,000 courses and labs.

Flow

Resources pocketed; three vendor slides now preview today's labs before M1 starts at 09:25.

Links

<https://survey.anum.cloud>

<https://github.anum.cloud>

<https://www.skills.google/ilt/classrooms/37990>

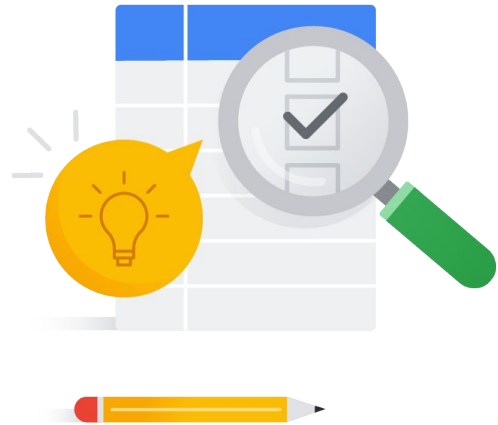
Lab: Loading Data into BigQuery

30:00

🕒 30 min

Learning objectives

- Load data into BigQuery from various sources.
- Load data into BigQuery using the CLI and the Google Cloud console.
- Use DDL to create tables.



Google Cloud

INSTRUCTOR NOTES — PAGE 19 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Their first win in a new console: a file becomes a queryable table — the simplest thing their nightly extracts do.

Say · cover

- Runs 10:40-11:10 after console orientation; deliberately trivial for SQL people — confidence, not a test, so say so.
- Three ways in, one destination: console click-through, bq load in Cloud Shell, and DDL — CREATE TABLE and LOAD DATA.
- Map it to their world: Import Wizard becomes console load; BCP and BULK INSERT become bq load; DDL puts load logic in source control.
- Seed the gotcha now: the autodetect checkbox can build a one-column string table — 13:45 reproduces that failure on purpose.
- Early finishers — expect many — do DIN section 24: load from the public bucket into their class dataset.

Iowa tie

This is the Bronze-landing mechanic for file-based sources — the batch half of killing the extracts.

Design patterns

- First lab deliberately easy in an unfamiliar console
- Debrief maps each loading method to a current-stack tool

Anti-patterns

- Skipping the easy lab — the console muscle memory pays off all afternoon

History & what's new

Batch load jobs have long run free on BigQuery's shared slot pool; BigQuery Studio, GA in 2024, folded console, CLI and DDL loading into one web IDE.

Flow

Previews the 10:40 lab; the next two slides preview the Datastream and Dataform labs before M1.

Links

<https://cloud.google.com/bigquery/docs/loading-data>

<https://cloud.google.com/bigquery/docs/bq-command-line-tool>

<https://cloud.google.com/bigquery/docs/quickstarts/load-data-console>

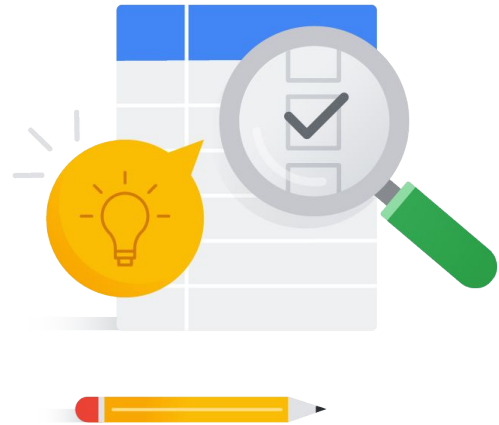
Lab (optional): Datastream: PostgreSQL Replication to BigQuery

45:00

🕒 45 min

Learning objectives

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database.
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data and changes are replicated correctly into BigQuery.



Google Cloud

INSTRUCTOR NOTES — PAGE 20 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

Their migration wearing a lab costume: a database that never stops changing, replicated continuously — and no extract window anywhere.

Say · cover

- Split lab by design: Part 1 at 11:50 builds and starts the stream; Part 2 at 13:00 validates and watches a live change land.
- Mechanics: prepare Cloud SQL for PostgreSQL, create two connection profiles — their SSIS connection managers — then one stream, started.
- The headline is scheduled: their data changes while they eat lunch, and no job ran — that feeling is the takeaway.
- Why PostgreSQL: free-tier friendly, and the mechanics are identical for SQL Server, a supported Datastream source since Jul 2024.
- Instruction to repeat twice: do not pause or stop anything at lunch — replication running through the break IS the demo.

Iowa tie

Kill-the-extracts in miniature: change data capture from an operational database straight into BigQuery — their exact target pattern.

Design patterns

- Connection profiles separate connectivity from pipelines, same discipline as SSIS connection managers

Anti-patterns

- Treating CDC as a faster batch job
- Stopping the stream at lunch and killing the reveal

History & what's new

Datastream's PostgreSQL source and BigQuery destination went GA in Apr 2023, the SQL Server source followed in Jul 2024, and native BigQuery partitioning and clustering for streams arrived in early 2026.

Flow

Runs 11:50 after M2; the next slide previews the Dataform anchor lab, then a divider and the M1 handoff.

Links

<https://cloud.google.com/datastream-for-bigquery>

<https://cloud.google.com/datastream/docs/create-a-stream>

<https://cloud.google.com/datastream/docs/configure-your-source-sql-server-database>

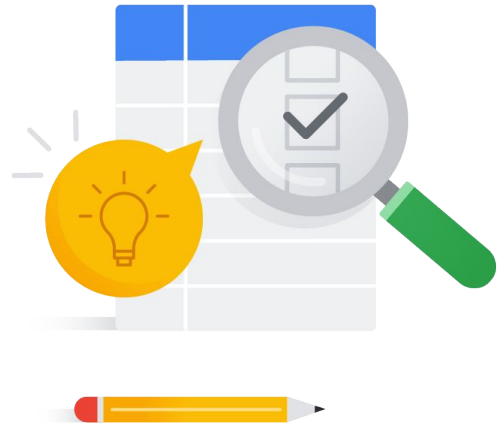
Lab: Create and Execute a SQL Workflow in Dataform

30:00

🕒 30 min

Learning objectives

- Create a Dataform repository.
- Create and initialize a Dataform development workspace.
- Create and execute a SQL workflow.
- View execution logs in Dataform.



Google Cloud

INSTRUCTOR NOTES — PAGE 21 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

The anchor lab: forty minutes building the Silver-to-Gold engine their architecture document already names.

Say · cover

- Runs 15:25-16:05, right after the dataform-demo walkthrough — they see a real raw-to-marts repo, then build the same bones.
- Arc: create repository, development workspace — their feature branch — a sqlx staging view, a mart table via ref(), execute, read logs.
- The magic moment is the compiled graph: dependency order inferred from ref() calls, never scheduled by hand.
- Executing is not committing is not deploying — release configs off git are the CI/CD translation; M&O questions especially welcome.
- Execution logs reward the SQL Agent instinct: when something turns red, the log is the first place to look.

Iowa tie

Their doc names Dataform for Silver-to-Gold ELT; these forty minutes are the most transferable skill of Day 1.

Design patterns

- Workspace per developer backed by git; releases run off the repo, not the workspace

Anti-patterns

- Editing in someone else's workspace
- Treating a manual execution as a deployment

History & what's new

Dataform — a 2018 startup Google acquired in 2020 — went GA in BigQuery in Jun 2023; the Nov 2025 release notes added GA automation for creating BigLake Iceberg tables.

Flow

Last lab preview; a logo divider follows, then the handoff slide into M1 at 09:25.

Links

<https://cloud.google.com/dataform/docs/overview>
<https://cloud.google.com/dataform/docs/release-notes>
<https://github.com/jwdavis/dataform-demo>



INSTRUCTOR NOTES — PAGE 22 OF 135 · 09:00 · WELCOME & COURSE INTRO

FACILITATOR NOTE

- Pure Google Cloud logo divider — do not linger; one breath ('that is the platform, here is what happens next') and advance.
- If energy dipped during the lab-logistics run, this is the reset beat before the M1 handoff slide.

Flow: Divides the 09:00 welcome block from the M1 preview slide that ends it; M1 itself starts at 09:25.

Next at 09:25 — M1: Data Engineering Tasks & Components

- ✓ You have the shape of the two days— kill the extracts, land in BigQuery, medallion, Dataform, Power BI
- ✓ M1 maps your SSIS mental model 1:1— sources, transforms, sinks with Google service names on each box
- ✓ And orients you in the platform— console, projects, IAM, billing, regions
- ✓ At 10:10 you go hands-on— star two projects and create your class dataset (everything after lunch needs it)
- ✓ Tabs ready: Skills classroom + Google Cloud console

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 23 OF 135 · 09:00 · WELCOME & COURSE INTRO

Hook

They have the map; M1 fills in the terrain — their SSIS mental model with Google service names on each box.

Say · cover

- M1's promise: sources, transforms, sinks map one-to-one to SSIS thinking; Datastream, Dataflow, Dataform and BigQuery take their seats.
- Platform orientation rides along: console, projects, IAM, billing, regions — pitched at GCP-novice pace, no concept dumping.
- 10:10 is the first hands-on: star two projects and create the class dataset — everything after lunch depends on it.
- Tabs ready now: Skills classroom plus console; parking-lot questions get first air at the 10:25 break.

Iowa tie

The dataset they create at 10:10 becomes their working Bronze landing zone for every loading exercise this afternoon.

Design patterns

- End each block by naming the next block and the one action students must be ready for

Anti-patterns

- Letting welcome run past 09:25 — M1 is content-critical and the clock is tight

History & what's new

GCP's project, IAM and billing model has been stable for a decade, but the surface changes: the console's unified BigQuery Studio went GA in 2024.

Flow

Ends the 09:00 welcome block; the next page opens M1 — DE tasks and components, 09:25-10:10, leading to the console-orientation DIN.

Links

<https://cloud.google.com/iam/docs/overview>

<https://cloud.google.com/resource-manager/docs/creating-managing-projects>

<https://cloud.google.com/bigquery/docs/datasets-intro>

WATCH FIRST · 2:27 · GOOGLE CLOUD TECH

What is BigQuery?

- ✓ The two-minute answer before we open the console: a serverless warehouse — no instance to start, no version to patch, storage and compute scaled apart.
- ✓ Watch for: why 'serverless' is not a marketing word — it is the reason you never size anything this week.



Scan to open on YouTube
youtu.be/m8WqxLd1jSc

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 24 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS**Hook**

- Play before the M1 slides — 2:27, sound on. It lands the serverless claim so you don't have to make it twice.

Say

- "That's the warehouse your nightly extract is about to become." — then straight into the block.

Cue

- Start at 0:10; if running late, stop at 2:00 — the last 20 seconds are a recap you can cover verbally.

Link

- <https://www.youtube.com/watch?v=m8WqxLd1jSc>

M1 — What this block gets you

- ✓ Your SSIS mental model, unchanged— sources → transforms → sinks maps 1:1 onto Google Cloud
- ✓ Where every piece lives in GCP— console, projects, IAM, billing, regions
- ✓ Why file format matters more than file location— CSV vs Parquet sets up this afternoon
- ✓ BigQuery as your target warehouse— serverless, no instance, no patching
- ✓ Hands-on: orient in the console, create your class dataset, then your first load lab

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 25 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Everyone here has built SSIS packages — this block is 45 minutes of relabeling boxes they already know with Google names, then their first hands-on in the console.

Say · cover

- Sources, transforms, sinks: their SSIS data-flow model maps 1:1 onto GCP services — say that early and often.
- Weave in platform orientation constantly: console, projects, IAM, billing, regions — they are GCP novices, not DE novices.
- File format gets its own slide because it sets up this afternoon's CSV-vs-Parquet race — flag that payoff now.
- BigQuery is the target warehouse: serverless means no instance, no patching, nothing to size.
- Block ends hands-on: console orientation (create the class dataset), then the first load lab after break.

Iowa tie

This block is the map of their whole migration: kill the extracts, land in BigQuery, Bronze to Silver to Gold, out to Power BI. Point at the map before touring the pieces.

Design patterns

- Anchor every new GCP name to the SQL Server tool it replaces
- Preview the hands-on payoff before the theory

Anti-patterns

- Presenting GCP as an alphabet soup of products with no mapping to their stack

History & what's new

BigQuery descends from Google's internal Dremel engine (in use 2006; VLDB paper 2010). At Cloud Next 2026 Google announced BigQuery Fluid Scaling and agentic data-cloud features.

Flow

Welcome and survey just ended; next slide is the morning roadmap card, then the DE-tasks content.

Links

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/bigquery/docs/resource-hierarchy>

<https://cloud.google.com/blog/topics/inside-google-cloud/whats-new-google-cloud>

This morning, block by block

09:25–10:10	M1 — Data engineering tasks & components, plus GCP platform orientation	
10:10–10:25	Console orientation — star the demo projects, create your class dataset	DIN
10:25–10:40	Break	
10:40–11:10	Lab: Loading Data into BigQuery — your first win in a new console	LAB

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 26 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Set the contract for the morning: 45 minutes of concepts, then hands-on for the rest of it.

Say · cover

- Walk the timeline: M1 concepts, 10:10 console orientation, break, 10:40 first lab.
- Call out the dependency: the class dataset created at 10:10 is required by every loading activity after lunch.
- Frame the 10:40 lab as a deliberate confidence-builder — their first win in an unfamiliar console.
- Point out the 10:25 break explicitly — it is when stragglers catch up on starring and dataset creation.

Iowa tie

By 11:10 everyone will have loaded a table into BigQuery — the smallest possible version of the nightly extract jobs they run today.

Design patterns

- Make cross-block dependencies (the class dataset) explicit before they are needed
- Name the time boxes out loud — platform novices relax when they know exactly when hands-on starts

Anti-patterns

- Letting the first hands-on block slip — momentum matters more than covering slides

History & what's new

The console they will live in today is BigQuery Studio, the unified data interface Google announced at Next '23 (August 2023).

Flow

Roadmap card right after the block opener; next is the deck's five data-engineering tasks.

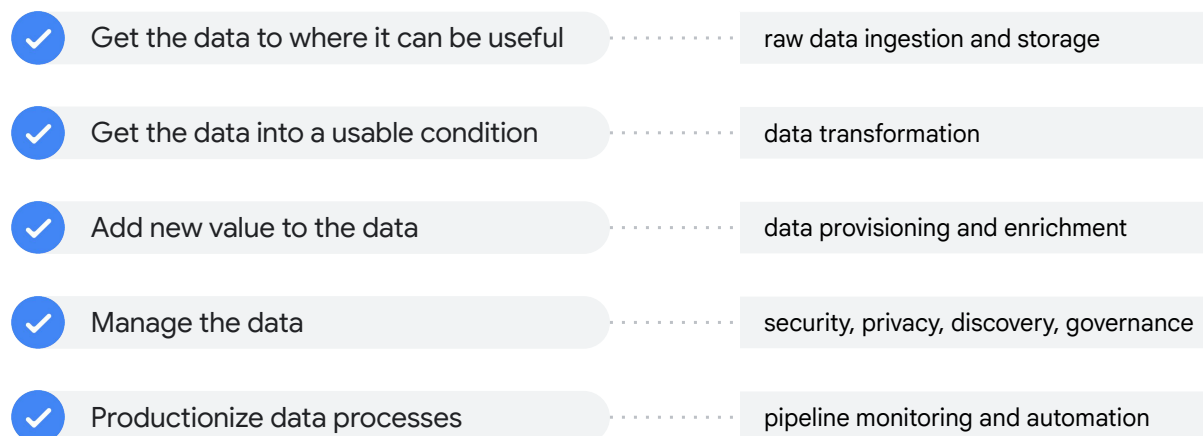
Links

<https://cloud.google.com/bigquery/docs/bigquery-web-ui>

<https://roitraining.github.io/gcp-demos/bigquery.html>

<https://cloud.google.com/bigquery/docs/datasets-intro>

A data engineer builds data pipelines to enable data-driven decisions



Google Cloud

INSTRUCTOR NOTES — PAGE 27 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Poll the room: which of these five eats your week? Most ETL teams answer ingestion and transformation — numbers four and five are why the M&O team exists.

Say · cover

- Get data where it is useful = raw ingestion and storage; into usable condition = transformation.
- Add new value = provisioning and enrichment — their marts and reference-data joins.
- Manage the data = security, privacy, discovery, governance — the single-source-of-truth requirement.
- Productionize = monitoring and automation — SQL Agent territory today, platform-native on GCP.
- All five are the job regardless of vendor; only the tools change.

Iowa tie

Tasks four and five map to their M&O team's world — GCP moves governance and monitoring into the platform instead of bolted-on tooling.

Design patterns

- Treat governance and productionizing as first-class pipeline work, not phase two

Anti-patterns

- Declaring a pipeline done at first successful load — no monitoring, no rerun path

History & what's new

At Cloud Next 2026 Google pitched an agentic data cloud where agents query operational and analytical data without movement pipelines (The Cloud Pod, 2026).

Flow

First deck content slide after the custom openers; next is the EL/ELT/ETL diagram.

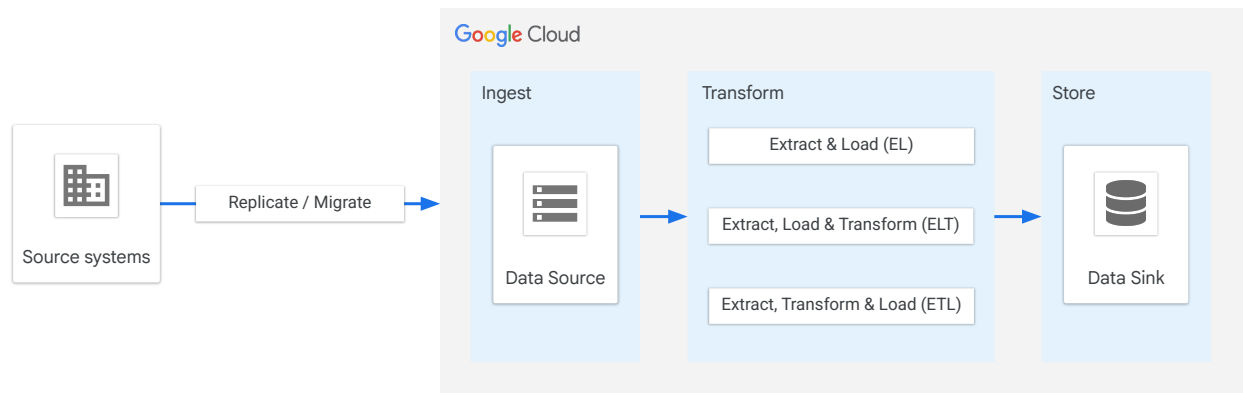
Links

<https://cloud.google.com/learn/what-is-etl>

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/blog/topics/inside-google-cloud/whats-new-google-cloud>

Data engineering tasks evolve around ingesting, transforming, and storing data



Google Cloud

INSTRUCTOR NOTES — PAGE 28 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

They know all three patterns cold — the migration question is which letter moves, and the answer is why Dataform exists.

Say · cover

- ETL: transform before load — classic SSIS, with business logic held in a separate engine.
- EL: land raw and stop — rarely sufficient alone, but it is the Bronze ingestion step.
- ELT: load first, transform in the warehouse with SQL — the cloud default because storage and compute are cheap and decoupled.
- ELT only works with discipline: versioned SQL, tests, scheduled releases — without them the warehouse becomes a swamp.

Iowa tie

Their target architecture is ELT: Datastream and loads land Bronze/Silver, then Dataform transforms Silver to Gold into the marts Power BI reads.

Design patterns

- Land raw first for replay and lineage; transform close to the data in set-based SQL

Anti-patterns

- Row-by-row transforms in an external engine when one BigQuery SQL statement would do

History & what's new

ELT tooling went mainstream with cloud warehouses; Google acquired Dataform in December 2020 and made it BigQuery's built-in SQL workflow layer.

Flow

From the five tasks to the pipeline verbs; next slide translates their entire stack tool-by-tool.

Links

<https://cloud.google.com/learn/what-is-etl>
<https://cloud.google.com/dataform/docs/overview>
<https://cloud.google.com/bigquery/docs/loading-data>

Your SQL Server stack, translated

Everything on the next slides is a tool you already own, wearing a Google name.

- SSIS data flow— sources → transforms → sinks, unchanged as a mental model
- SQL Agent jobs— Cloud Scheduler and BigQuery scheduled queries (end of Day 1)
- SSIS packages in git + CI/CD— Dataform repositories and release configs (this afternoon)
- Nightly extract windows— Datastream change data capture, replicating continuously (M2)
- Production SQL Server— stays the system of record while BigQuery becomes the analytics source of truth
- Your data marts → Power BI— Gold marts in BigQuery feed the reports you already have (Day 2)

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 29 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

This is the photograph-this-slide moment: every tool they own, wearing a Google name.

Say · cover

- Datastream is the extract-killer: continuous change data capture from SQL Server, not nightly windows — and yes, SQL Server is a supported source.
- Dataform repositories plus release configurations = SSIS packages in git with CI/CD — their existing discipline carries over.
- Cloud Scheduler and BigQuery scheduled queries = SQL Agent; they will see one scheduled at 16:20 today.
- SQL Server stays the system of record; BigQuery becomes the analytics source of truth, and Gold marts feed the Power BI they already have.
- T-SQL becomes GoogleSQL: same declarative SQL, different dialect — they will write it within the hour.

Iowa tie

This single table is their migration: every row is a workstream in the project plan — extracts, SSIS, Agent jobs, marts, Power BI.

Design patterns

- Keep system of record and analytics source of truth as explicitly separate roles during migration

Anti-patterns

- Lift-and-shifting SSIS package logic into bespoke scripts instead of managed services

History & what's new

Datastream's SQL Server source went GA in July 2024 (Google Cloud blog); Datastream itself GA'd in 2021 with Oracle and MySQL first.

Flow

The reframe slide; next, the deck tours GCP services box-by-box, starting with replicate and migrate.

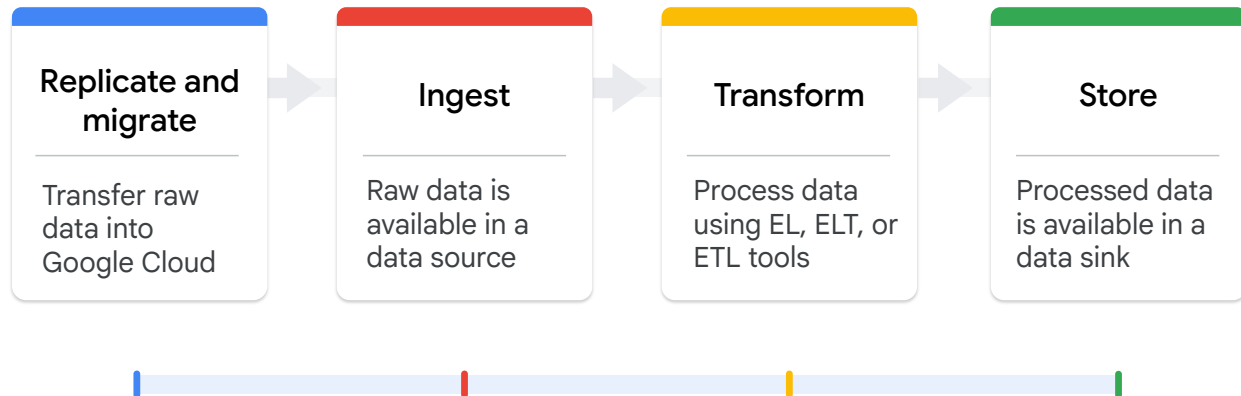
Links

<https://cloud.google.com/blog/products/data-analytics/datastream-makes-it-easy-to-replicate-sql-server-databases>

<https://cloud.google.com/dataform/docs/overview>

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery-aad>

Data engineering tasks evolve around ingesting, transforming, and storing data



Google Cloud

INSTRUCTOR NOTES — PAGE 30 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Same picture as their SSIS data flow, with one addition on top: replication and migration for onboarding whole databases.

Say + cover

- Ingest: transfer raw data into Google Cloud; raw data starts in a source.
- Transform: EL, ELT, or ETL tools add the new value.
- Store: processed data lands in a sink where consumers query it.
- Replicate/migrate wraps the whole loop for continuous database onboarding — the extract-killer pattern, detailed in M2 next hour.
- Have them sketch these four boxes on paper — it becomes the skeleton of the Day 2 capstone worksheet.

Iowa tie

The replicate/migrate loop is the box their nightly extracts live in today — Module 2 replaces it with Datastream.

Design patterns

- Name a service and an owner for every box in the architecture document

Anti-patterns

- Treating replication as a one-time copy instead of a continuous stream
- Drawing sinks with no named consumer — every sink should answer who queries it

History & what's new

Datastream, Google's serverless CDC service, went GA in December 2021; its SQL Server source followed in July 2024.

Flow

Concept recap; the next four slides put one GCP service on each box, starting with replication services.

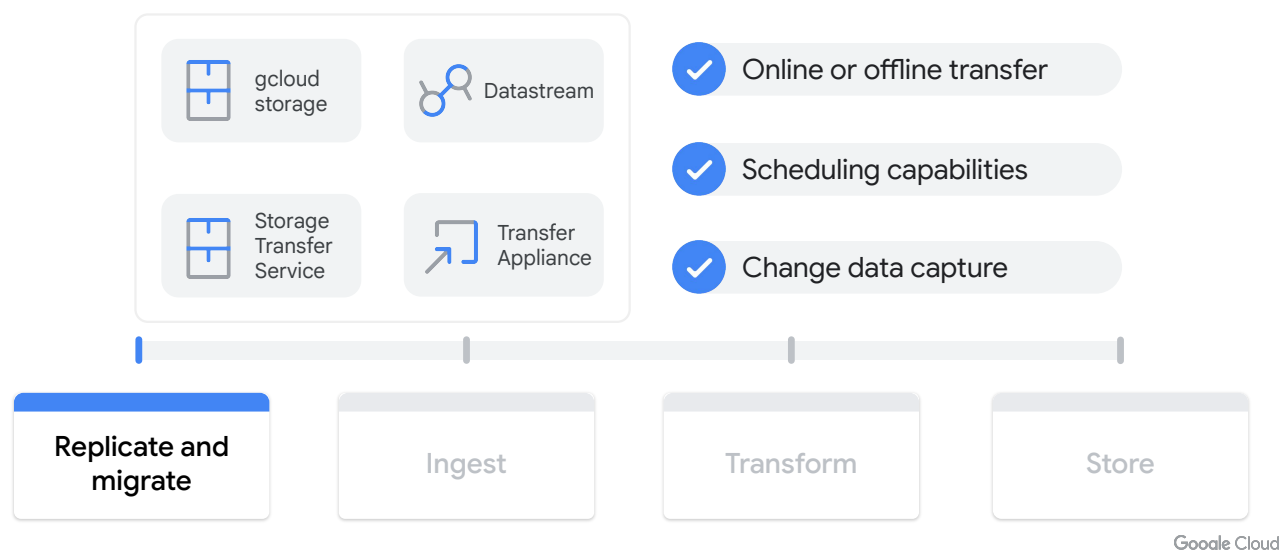
Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/storage-transfer/docs/overview>

<https://cloud.google.com/dataflow/docs/overview>

Replication and migration services onboard your data into Google Cloud



INSTRUCTOR NOTES — PAGE 31 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Four ways to move data in — choose by source type and volume, not by habit.

Say · cover

- Datastream: log-based change data capture from MySQL, Oracle, PostgreSQL and SQL Server into BigQuery — no triggers on production.
- Storage Transfer Service: scheduled, managed batch moves between object stores and file systems.
- gcloud storage: the fast CLI for ad hoc and scripted file transfers.
- Transfer Appliance: a shipped physical device for hundreds of terabytes — know it exists; likely not their path.

Iowa tie

Datastream from production SQL Server is their Bronze ingestion. M&O will ask about load: it is log-based CDC, no polling of production tables.

Design patterns

- CDC for continuously changing OLTP; batch transfer for files; measure the wire before choosing
- Pilot Datastream on one low-risk table before streaming the crown jewels

Anti-patterns

- Nightly full-table dumps when log-based CDC exists
- Ad hoc copies with no scheduling or audit trail

History & what's new

Google rebuilt the storage CLI as gcloud storage in 2022 — up to 94% faster transfers than gsutil (Google Cloud blog, 2022).

Flow

Service tour 1 of 4; next: where raw data originates — data sources.

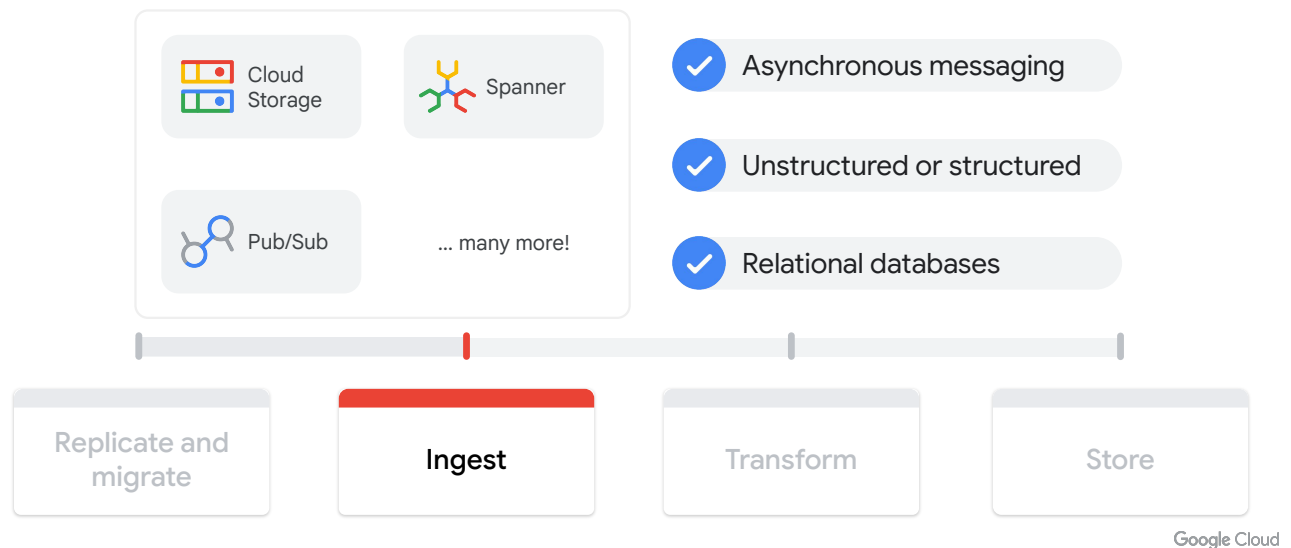
Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/storage-transfer/docs/overview>

<https://cloud.google.com/blog/products/storage-data-transfer/new-gcloud-storage-cli-for-your-data-transfers>

Data sources are the origin point of your raw data on Google Cloud



INSTRUCTOR NOTES — PAGE 32 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Their sources today: relational databases and files. The new one in this picture is the message stream.

Say · cover

- Relational databases are the systems of record — on GCP that is Cloud SQL, Spanner, AlloyDB, or their on-prem SQL Server feeding in via Datastream.
- Pub/Sub is asynchronous messaging: event streams that can land in BigQuery continuously — their future real-time feeds.
- Cloud Storage is the universal drop zone: any file, structured or unstructured, addressed by name over HTTP.
- On GCP a source is usually a managed service, not a server someone patches.

Iowa tie

Their dominant source is production SQL Server; during transition, file-based extracts land in Cloud Storage buckets as raw Bronze.

Design patterns

- Every source lands raw (Bronze) before any transformation touches it
- Register every source in the catalog on day one — undocumented sources become undocumented pipelines

Anti-patterns

- Sources writing straight into curated marts — no replay, no lineage

History & what's new

Datastream keeps adding source types: Spanner, MongoDB, Salesforce, ServiceNow and Workday sources are documented as of 2026 (Datastream docs).

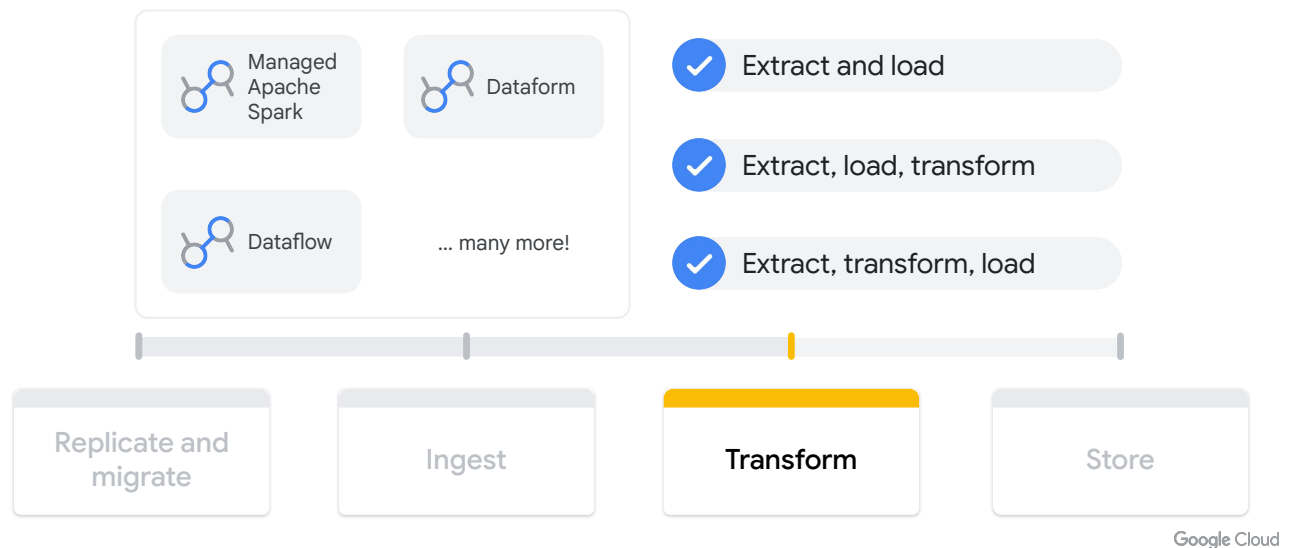
Flow

Tour 2 of 4; next: transformation services — Dataform's first appearance.

Links

<https://cloud.google.com/pubsub/docs/overview>
<https://cloud.google.com/storage/docs/introduction>
<https://cloud.google.com/datastream/docs/sources>

Transformation services add new value to your data



INSTRUCTOR NOTES — PAGE 33 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Three engines, three personas: Spark for existing big-data code, Dataflow for streaming pipelines, Dataform for SQL people — this room.

Say · cover

- Dataform: SQL-based ELT inside BigQuery with git versioning, tests and release configurations — the Silver-to-Gold workhorse this afternoon.
- Dataflow: managed Apache Beam for heavy-duty or streaming transforms; batch and stream in one programming model.
- Managed Apache Spark (Dataproc): run existing Spark and Hadoop code without owning clusters.
- Rule of thumb: SQL-first in Dataform; reach for Dataflow when the logic is not SQL or the data never stops.

Iowa tie

Dataform is the named service in their target architecture — today's 14:45 module plus Jeff's walkthrough goes deep on it.

Design patterns

- Transforms as versioned, tested SQL with declared dependencies; let the tool compute lineage

Anti-patterns

- Stored-procedure sprawl: transformation logic nobody can diff, test, or roll back
- Choosing Spark or Dataflow for transforms that are plain SQL — resume-driven engineering

History & what's new

Google donated the Dataflow SDK to the Apache Software Foundation in 2016, creating Apache Beam (Wikipedia).

Flow

Tour 3 of 4 — flag the afternoon Dataform module; next: sinks, where curated data serves consumers.

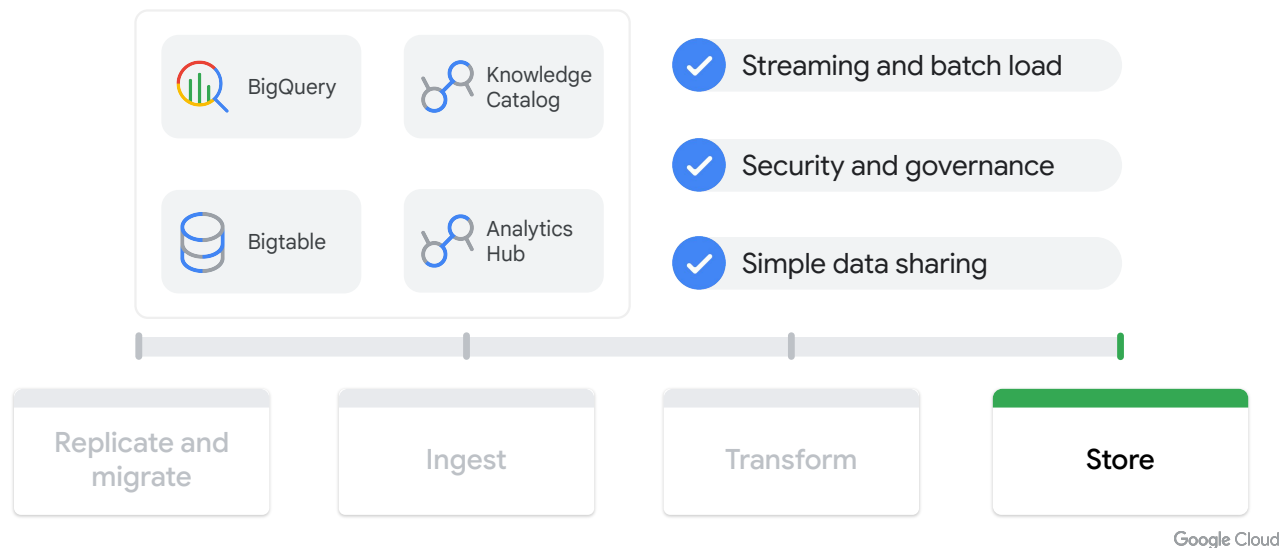
Links

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/dataflow/docs/overview>

<https://cloud.google.com/dataproc/docs/concepts/overview>

Data sinks store your processed data on Google Cloud



INSTRUCTOR NOTES — PAGE 34 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

The sink is where curated data meets its consumers — for this room that answer is almost always BigQuery.

Say · cover

- BigQuery: the analytical sink — Silver and Gold tables and marts, fed by streaming and batch load.
- Bigtable: low-latency NoSQL serving at massive scale — operational use cases, not their migration path.
- Knowledge Catalog: discovery and governance over the sinks — the metadata layer behind a single source of truth.
- Analytics Hub: publish datasets to other organizations without copying — one line only, per the agenda.

lowa tie

The sink side of their design: Gold marts in BigQuery feeding Power BI; the catalog underpins their governance requirement.

Design patterns

- One governed sink with sharing beats a copied extract per consumer
- Treat the catalog as part of the sink, not an optional add-on

Anti-patterns

- Every report team keeping private copies of the same data

History & what's new

Google's catalog was renamed Knowledge Catalog in April 2026, after stints as Dataplex Universal Catalog and BigQuery universal catalog (release notes).

Flow

Tour 4 of 4; next: data formats — the slide that sets up this afternoon's CSV-vs-Parquet race.

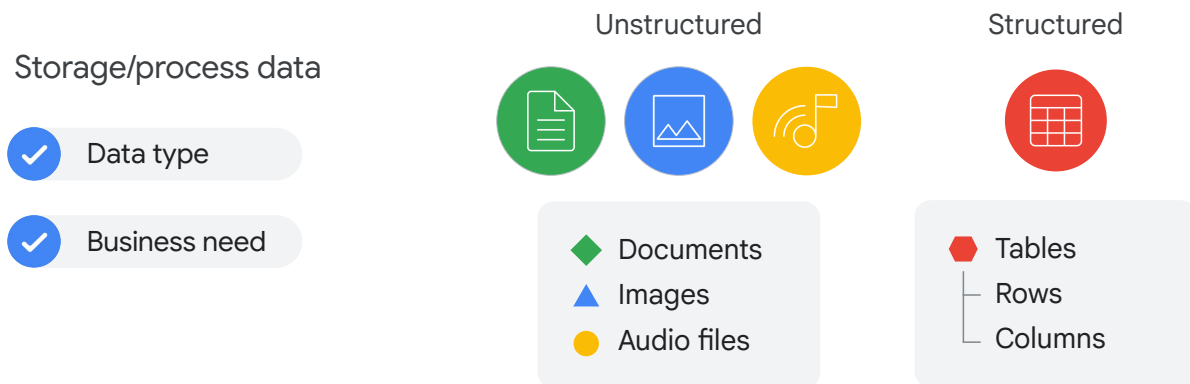
Links

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/bigtable/docs/overview>

<https://docs.cloud.google.com/knowledge-catalog/docs/release-notes>

Data can have different formats



Google Cloud

INSTRUCTOR NOTES — PAGE 35 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Format is an architecture decision: it decides who can read the data, how fast, and at what cost.

Say · cover

- Structured = tables, rows, columns — their entire SQL Server world.
- Unstructured = documents, images, audio — a growing share of state data (scanned forms, GIS imagery).
- Semi-structured sits between: JSON, Avro and Parquet carry schema with the data.
- Data type plus business need drives the storage and processing choice.
- Tee up the afternoon: within structured files, row layout (CSV) versus columnar (Parquet) changes load time by an order of magnitude.

Iowa tie

Today's extracts are CSV; their Bronze design wants open columnar formats — tomorrow's Iceberg and BigLake story builds on this slide.

Design patterns

- Keep raw data in open, self-describing formats (Parquet, Avro), not CSV

Anti-patterns

- CSV as the enterprise interchange standard — no types, no schema, re-parsed on every read

History & what's new

Apache Parquet debuted in 2013 as Hadoop's columnar format; Apache Iceberg (created at Netflix in 2017) later put table semantics on top.

Flow

From services to the bytes themselves; next: where those bytes live — Cloud Storage.

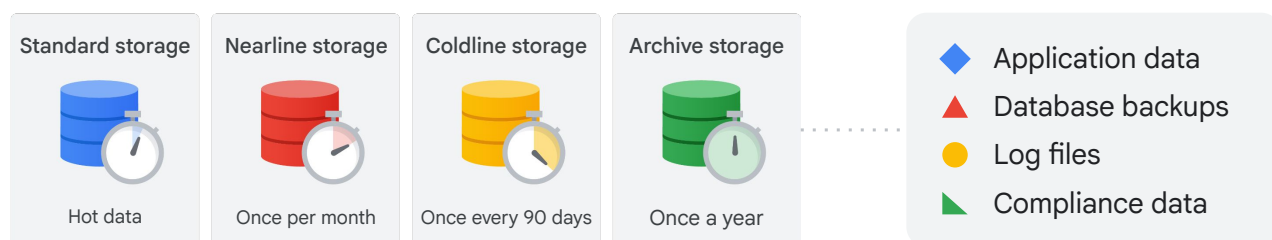
Links

<https://cloud.google.com/storage/docs/introduction>

<https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-parquet>

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

Cloud Storage holds your unstructured data



✓ Reliability and scalability

✓ Accessed by HTTP request

✓ Retrieved by object name

✓ Max object size of 5 TB

Google Cloud

INSTRUCTOR NOTES — PAGE 36 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Think of Cloud Storage as an infinite, versioned file share — and classes as the price you pay for each access pattern.

Say · cover

- Objects are retrieved by name over HTTP; max object size 5 TB; eleven-nines durability.
- Standard is hot data; Nearline fits monthly access (30-day minimum); Coldline quarterly (90-day); Archive yearly (365-day) yet still millisecond retrieval.
- The slide's examples map exactly: database backups, log files, compliance data.
- Bronze landing buckets stay Standard; lifecycle rules or Autoclass cool data down automatically.
- Storage class is per-object metadata: buckets carry a default, objects inherit it, lifecycle rules change it over time.

Iowa tie

Their Bronze raw extracts land here; M&O's retention and compliance data is the Archive-class use case.

Design patterns

- Set lifecycle policies (or Autoclass) so data cools automatically instead of by ticket
- Co-locate buckets with the BigQuery dataset location to avoid egress and consistency surprises

Anti-patterns

- Everything in Standard forever — or archiving data that is queried weekly and paying retrieval fees

History & what's new

Cloud Storage launched in 2010; the newest addition is Rapid, a zonal class for I/O-heavy AI/ML workloads (Google docs, 2026).

Flow

Unstructured storage done; next: the structured-store menu.

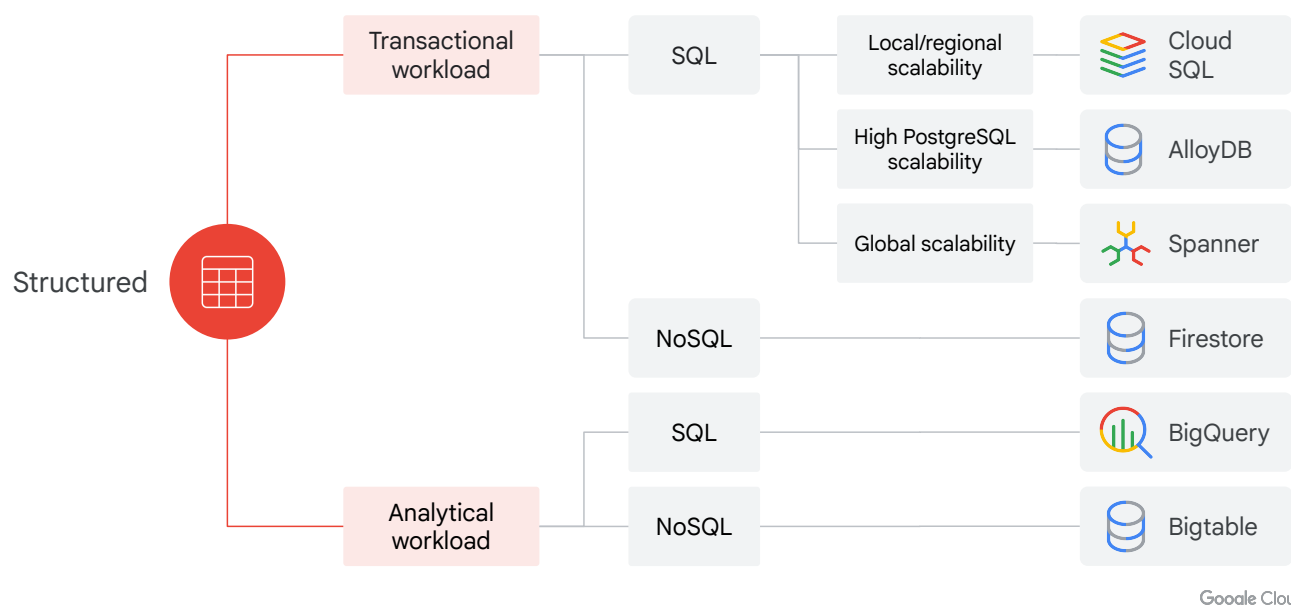
Links

<https://cloud.google.com/storage/docs/storage-classes>

<https://cloud.google.com/storage/docs/introduction>

<https://cloud.google.com/learn/what-is-a-data-lake>

Options for storing structured data



INSTRUCTOR NOTES — PAGE 37 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

The whole GCP database menu on three axes: SQL versus NoSQL, transactional versus analytical, regional versus global.

Say · cover

- Transactional SQL: Cloud SQL (managed MySQL, PostgreSQL, SQL Server), AlloyDB for high-performance PostgreSQL, Spanner for global scale.
- NoSQL: Firestore for documents, Bigtable for wide-column massive throughput.
- Analytical: BigQuery — the only box on this slide that matters for their migration.
- Pick by workload, not familiarity: OLTP stays on operational engines; analytics goes to BigQuery.

Iowa tie

Their decision is already made — BigQuery for analytics, SQL Server remains the OLTP system of record; this slide is vocabulary for the architecture document.

Design patterns

- Match the engine to the workload (OLTP vs OLAP) before matching the brand
- Keep the operational estate boring: one OLTP engine, one analytical engine

Anti-patterns

- Running analytical scans on the OLTP box — their current pain — or adopting Spanner or Bigtable without the scale case

History & what's new

AlloyDB, Google's PostgreSQL-compatible engine, GA'd in June 2022; at Next 2026 it scaled to 10 billion vectors with the ScaNN index (Google Cloud blog).

Flow

Storage menu complete; next: the lake-versus-warehouse framing that names their target architecture.

Links

<https://cloud.google.com/products/databases>
<https://cloud.google.com/spanner/docs/overview>
<https://cloud.google.com/alloydb/docs/overview>

Data lake versus data warehouse

	Data lake	Data warehouse
Data format	Native (unstructured, semi-structured, structured)	Schema (structured or semi-structured)
Data type	Raw	Pre-processed and aggregated from multiple data sources
Purpose	Data science, applications, business decisions	Long-term business analysis
Dependencies	Tools/processes to enable data discovery, governance, security, metadata management	Standalone
Service	 Cloud Storage	 BigQuery

Google Cloud

INSTRUCTOR NOTES — PAGE 38 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

This comparison table is the two halves of their own architecture document — and the argument for having both.

Say · cover

- Lake (Cloud Storage): raw, native formats, any data type — but needs discovery, governance and metadata tooling wrapped around it.
- Warehouse (BigQuery): schema'd, pre-processed, aggregated — standalone, optimized for long-term business analysis.
- Bronze is the lake, Silver and Gold are the warehouse; tomorrow's M7/M8 shows them converging into one lakehouse over open formats.
- Semi-structured data such as JSON logs can live in either — the deciding factor is who queries it and how often.

Iowa tie

This is literally their Bronze-to-Silver-to-Gold design — tomorrow morning teaches it properly; today, plant the vocabulary.

Design patterns

- Raw-first lake landing plus governed warehouse marts, one catalog over both
- Let format follow function: raw stays open and native, curated gets schema

Anti-patterns

- Lake without governance becomes a swamp
- Warehouse as the only copy — no replay, no full fidelity

History & what's new

The term lakehouse entered the vocabulary around 2020; Gartner published its first market guide for lakehouse platforms in late 2025 (CIO).

Flow

Vocabulary planted; next: BigQuery itself — the target warehouse.

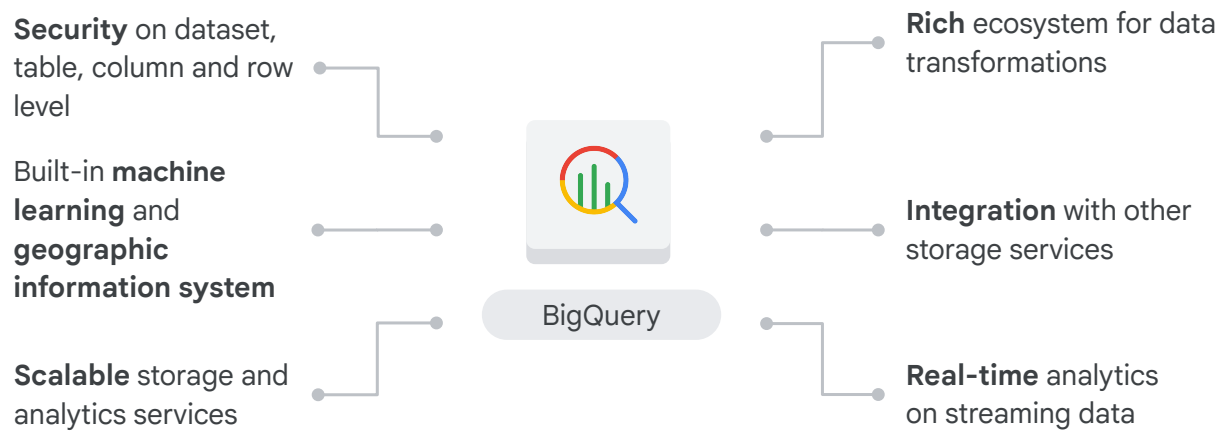
Links

<https://cloud.google.com/learn/what-is-a-data-lake>

<https://cloud.google.com/learn/what-is-a-data-warehouse>

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

BigQuery is a serverless, fully managed data warehouse



Google Cloud

INSTRUCTOR NOTES — PAGE 39 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Here is the payoff: no instance, no patching, no capacity planning — you create datasets and run queries.

Say · cover

- Serverless means storage and compute are decoupled; Google allocates the compute (slots) per query.
- Real-time analytics on streaming data, plus integration with Cloud Storage and other services.
- Built-in machine learning and geospatial — mention, don't drill; Day 2 previews both.
- Security at dataset, table, column and row level — the next two slides detail it.
- For M&O: no failover clusters and no patch windows — a genuinely different operations story.

Iowa tie

This is the target warehouse — the single source of truth that replaces the SQL Server data warehouse.

Design patterns

- Let Google manage capacity; spend M&O effort on data quality and access design

Anti-patterns

- Sizing BigQuery like a server (there is nothing to size)
- Reading serverless as no management needed — you still own schema, cost and access design

History & what's new

BigQuery opened publicly in 2012 atop Google's Dremel engine; Next 2026 added agentic features and Fluid Scaling autoscaling (Rittman Analytics, 2026).

Flow

Met the target; next: how BigQuery organizes resources — the hierarchy they will touch in 20 minutes.

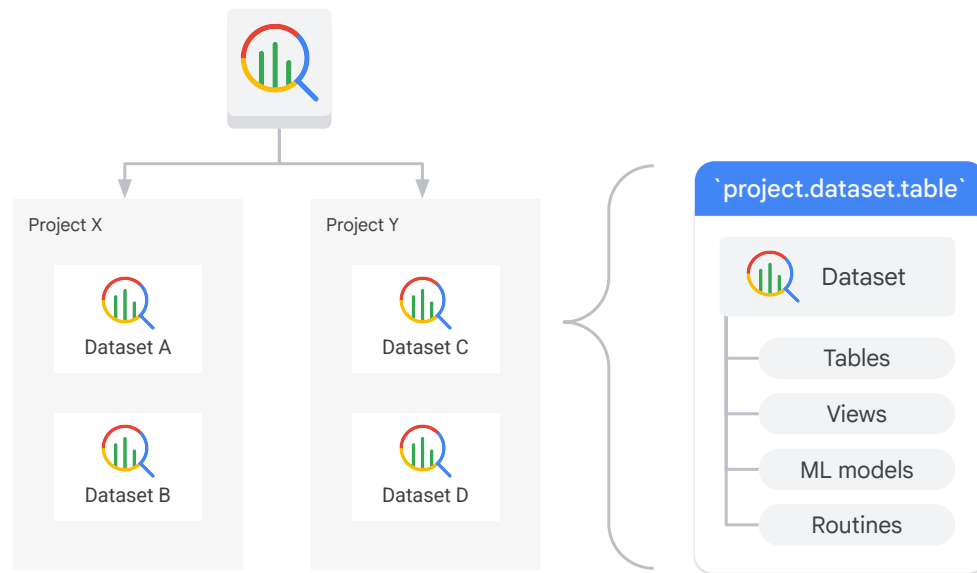
Links

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/bigquery/docs/sandbox>

<https://cloud.google.com/blog/topics/inside-google-cloud/whats-new-google-cloud>

How BigQuery's resources are organized



Google Cloud

INSTRUCTOR NOTES — PAGE 40 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

project.dataset.table is instance.database.schema.table with a Google accent — the skeleton everything hangs off.

Say · cover

- Organization to folders to projects; BigQuery adds datasets, which hold tables, views, ML models and routines.
- A dataset is secured as a unit and has a location — chosen once at creation, permanent afterward.
- Billing, quotas and IAM attach at the project; jobs run in one project against data living in many.
- Google's documented patterns: a central raw-storage project plus department mart projects — matches a central-warehouse ambition.

Iowa tie

Project layout and dataset location (data residency) are deliberate architecture decisions for their migration, not administrative afterthoughts.

Design patterns

- One raw/storage project plus separate mart projects; permissions inherit down the tree
- Mirror the org chart in folders and projects so IAM inheritance does the paperwork

Anti-patterns

- One mega-project where billing, quotas and access cannot be separated
- Choosing a dataset location casually — it cannot be changed later

History & what's new

The console embodying this hierarchy is BigQuery Studio, announced at Next '23 as a single interface for data engineering, analytics and ML.

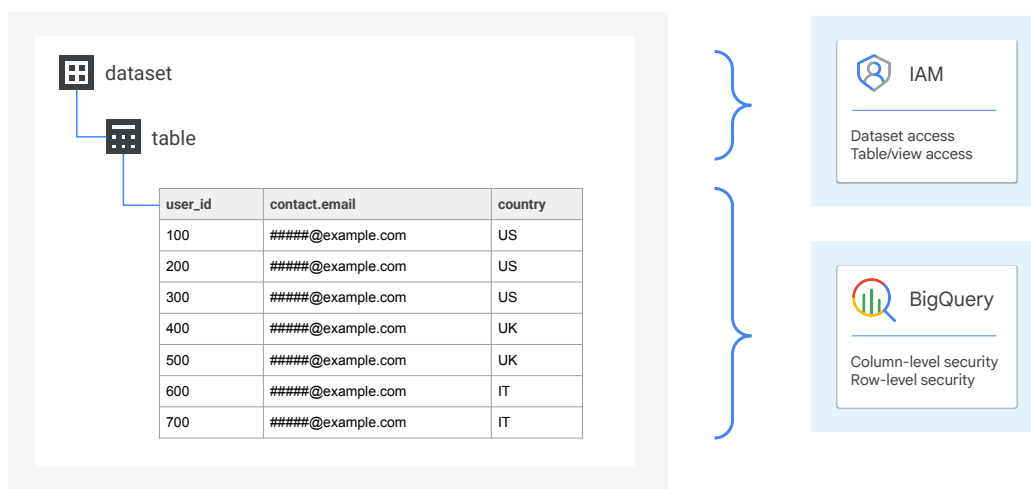
Flow

Hierarchy done; next: the security layers that ride on it — IAM through row-level.

Links

<https://cloud.google.com/bigquery/docs/resource-hierarchy>
<https://cloud.google.com/bigquery/docs/datasets-intro>
<https://cloud.google.com/bigquery/docs/access-control>

You can secure your BigQuery resources on multiple levels



Google Cloud

INSTRUCTOR NOTES — PAGE 41 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Same table, five audiences, five different answers — enforced by the engine, not by discipline in their WHERE clauses.

Say · cover

- IAM at the project; dataset access; table and view access — authorized views share results without exposing raw rows.
- Column-level security via policy tags masks sensitive columns — the hashed-out emails on the slide.
- Row-level security uses row access policies per user or group; FILTER USING works like a WHERE clause.
- SQL Server parallel: object GRANTS, row-level security predicates, dynamic data masking — the concepts port 1:1.

Iowa tie

State data means agency-level row filters and PII masking — the governance half of their single source of truth; tomorrow's M10 goes deep.

Design patterns

- Default-deny: grant at dataset level, then refine with authorized views, policy tags and row policies

Anti-patterns

- Copying filtered extracts per consumer instead of one table with row access policies

History & what's new

Row access policies now support subquery lookup tables, with policy-tag data masking layered on top — the fine-grained stack keeps growing (BigQuery docs, 2026).

Flow

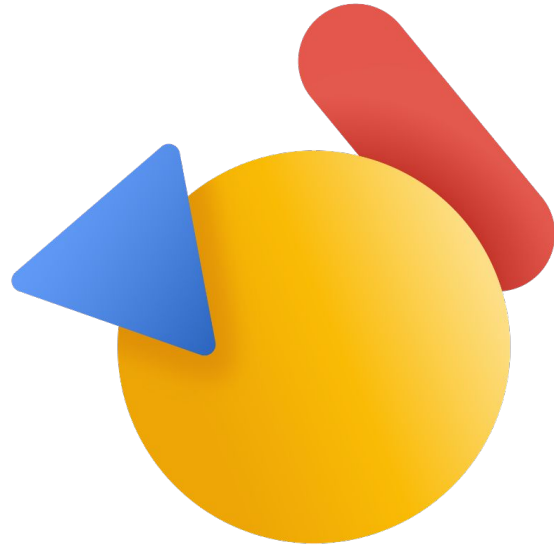
Security layers done; next: the deck demo — terabytes in seconds — then everyone goes hands-on.

Links

<https://cloud.google.com/bigquery/docs/row-level-security-intro>
<https://cloud.google.com/bigquery/docs/column-level-security-intro>
<https://cloud.google.com/bigquery/docs/access-control>

Demo

Query terabytes of data in seconds



Google Cloud

INSTRUCTOR NOTES — PAGE 42 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Don't talk about scale — show it. Ten minutes, one live query, one number they remember.

Say · cover

- Run against a bigquery-public-data table; show the bytes-processed estimate before clicking Run — that is the cost meter.
- Pick a column-pruned query (named columns, never SELECT star) to teach columnar reads while impressing.
- Narrate elapsed time and bytes scanned; contrast with provisioning an equivalent SQL Server box.
- Fallback if anything wobbles: the roitraining gcp-demos BigQuery page has canned demo queries.
- If a student asks about cost: on-demand pricing charges per byte scanned — this demo query costs cents, the lesson is free.

Iowa tie

This is the why-migrate moment; this afternoon's DIN section 26 race (about 60M rows, CSV versus Parquet) makes it personal.

Design patterns

- Show bytes-processed before every big query — builds cost literacy from day one

Anti-patterns

- Demoing SELECT star on a huge table — scans everything and teaches the wrong habit

History & what's new

The Dremel paper (VLDB 2010) already showed interactive scans over trillion-row data; at 10:10 students browse a 100-billion-row public table.

Flow

Last ten minutes of M1; next slide previews the hands-on console orientation.

Links

<https://cloud.google.com/bigquery/public-data>

<https://cloud.google.com/bigquery/docs/quickstarts/query-public-dataset-console>

<https://roitraining.github.io/gcp-demos/bigquery.html>

DIN: Exploring projects, datasets and tables

15 minutes · Google Cloud console — your lab project

1. Open the BigQuery console and star the bigquery-public-data and roi-bq-demos projects
2. Create a dataset named class in your lab project — every afternoon loading activity writes into it
3. Browse a public table: preview rows, inspect the schema, find the table details
4. Notice where you are: project → dataset → table, and the region you picked at creation

Why it matters — near-zero console experience is the only real gap in this room — this is the 15 minutes that closes it, and the class dataset is required by everything after lunch.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 43 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Hands on keyboards now — near-zero console experience is the only real gap in this room, and these 15 minutes close it.

Say · cover

- Run as follow-along: pin BigQuery, then star bigquery-public-data and roi-bq-demos (the guide also stars bigquery-samples).
- Everyone creates a dataset named class in their own lab project — required by the 13:45 loading block; verify all 16 before break.
- Browse bigquery-samples.wikipedia_benchmark.100B: Schema, Details, Preview — 100 billion rows, zero cost, no query.
- Top gotcha: the project picker in the top bar — 'I can't see what you see' almost always means wrong project.
- Starred projects are read-only; a dataset's location is permanent once chosen.

Iowa tie

The class dataset is their Bronze layer in miniature; starring equals querying data where it lives — the single-source-of-truth mechanic.

Design patterns

- Instructor drives, students mirror; nobody advances until the class dataset exists

Anti-patterns

- Letting students skip dataset creation — they will be creating it at 13:45 while everyone else loads data

History & what's new

The Studio console they are learning was announced at Next '23 (August 2023) as one interface for data engineering, analytics and ML.

Flow

This slide previews the 10:10 DIN block that starts now; then break, then the 10:40 load lab.

Links

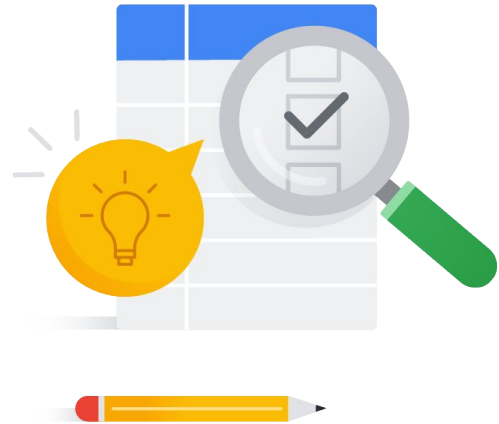
<https://cloud.google.com/bigquery/docs/bigquery-web-ui>
<https://cloud.google.com/bigquery/docs/datasets-intro>
<https://roittraining.github.io/gcp-demos/bigquery.html>

Lab: Loading Data into BigQuery

🕒 30 min

Learning objectives

- Load data into BigQuery from various sources.
- Load data into BigQuery using the CLI and the Google Cloud console.
- Use DDL to create tables.



Google Cloud

INSTRUCTOR NOTES — PAGE 44 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Deliberately easy — a first win in an unfamiliar console, not a skills test. For GCP novices that is the feature.

Say · cover

- Three ways in, same destination: console click-through, bq load (scriptable), and DDL — load logic as code, ready for their CI/CD discipline.
- Thirty minutes; SQL people often finish in 10 to 12 — route early finishers to DIN section 24 and have everyone note their load time.
- Autodetect-schema can silently build a one-column string table — note it and move on; section 25 reproduces the failure on purpose at 13:45.
- Expect Enable-API prompts and Cloud Shell first-run consent clicks; roam the room — 16 seats, pre-warmed.

Iowa tie

This is the smallest version of their nightly extract jobs: a file becomes a queryable table — the Bronze-landing mechanic for file-based sources.

Design patterns

- Same load via GUI, CLI and DDL — automation is the same job expressed as code

Anti-patterns

- Trusting schema autodetect for production loads — declare schemas explicitly

History & what's new

Google rebuilt the transfer CLI as gcloud storage in 2022 (up to 94% faster than gsutil); the bq tool has shipped with BigQuery since 2012.

Flow

The lab runs 10:40 to 11:10 after the break; then Module 2 — Datastream, the extract-killer.

Links

<https://cloud.google.com/bigquery/docs/loading-data>

<https://cloud.google.com/bigquery/docs/bq-command-line-tool>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language>

M1 — What you now have

- ✓ The pipeline shape you know— ingest, transform, store — with Google service names on each box
- ✓ A mental map of the console: projects, datasets, tables, IAM and billing at the project level
- ✓ Your class dataset created— the landing zone for this afternoon's four loading activities
- ✓ Next: killing the nightly extracts— Datastream and change data capture (M2)

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 45 OF 135 · 09:25 · MODULE 1 — DE TASKS & COMPONENTS

Hook

Ninety minutes ago GCP was alphabet soup; now every box on their architecture sketch has a Google name on it.

Say · cover

- Recap fast: pipeline shape unchanged; console map learned — projects, datasets, tables, with IAM and billing at the project.
- The class dataset exists — this afternoon's four loading activities land in it.
- Bridge line: next hour kills the nightly extract — Datastream reads the SQL Server transaction log, no triggers, no polling, no production load.
- Confirm before moving on: everyone created the dataset and finished the lab.

Iowa tie

Module 2 is the hour their migration hinges on — kill-the-extracts; this bridge is where you pre-empt the M&O objection.

Design patterns

- Close every block by re-anchoring to the audience's migration map

Anti-patterns

- Rolling into M2 with students who never created the dataset or finished the lab

History & what's new

The SQL Server to BigQuery CDC path only went GA in July 2024 — this migration would have been far harder two years earlier.

Flow

M1 closes; next block is M2 Replication and Migration at 11:10, with the Datastream lab at 11:50.

Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/blog/products/data-analytics/datastream-makes-it-easy-to-replicate-sql-server-databases>

<https://cloud.google.com/bigquery/docs/resource-hierarchy>

Exploring projects, datasets & tables (console orientation)

15 minutes — 10:10–10:25 · BigQuery console — instructor leads, you follow

1. Open BigQuery from the nav menu and pin it — check the project picker shows your lab project
2. Star three projects: bigquery-samples, bigquery-public-data, roi-bq-demos
3. Expand bigquery-samples → wikipedia_benchmark and read a dataset's metadata
4. Create your own dataset named exactly class (your project's three-dot menu → Create dataset)
5. Open the 100B table — Schema, Details, Preview: 100 billion rows, no query, no cost

Why it matters — project → dataset → table is the same skeleton as instance → database → schema.table — and class is the landing zone for every load this afternoon.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 46 OF 135 · 10:10 · DIN — EXPLORING PROJECTS, DATASETS & TABLES (15 MIN)

Hook

They live in SSMS Object Explorer all day; the BigQuery Explorer pane is the same tree with new labels — project, dataset, table instead of instance, database, schema.table.

Say · cover

- Demo the project picker first — the console remembers the last-used project; wrong project is the day's number-one confusion.
- Star bigquery-samples, bigquery-public-data, roi-bq-demos: read-only yet queryable — data queried where it lives, no copies.
- Create the dataset class, exactly lowercase; location is permanent — an architecture decision in production.
- Open wikipedia_benchmark.100B: Schema is the sp_help equivalent, Details shows row count and size, Preview shows rows with no query job, no charge.
- The three-dot menus hide Create dataset and Create table — used at 10:40.

Iowa tie

The class dataset is their Bronze layer in miniature — every afternoon load lands there. Production Bronze/Silver/Gold maps onto datasets or projects, so this hierarchy is a design decision, not scaffolding.

Design patterns

- Pin BigQuery and star shared projects once per engagement; the Explorer becomes the working map.
- Choose dataset location deliberately — it cannot be changed later.

Anti-patterns

- Assuming the console shows the right project — check the picker before debugging permissions.
- Creating objects inside starred, read-only projects.
- Treating dataset location as a throwaway default.

History & what's new

The console became BigQuery Studio at Cloud Next in August 2023 — one interface for SQL, notebooks, and pipelines; the 2026 docs show Git-backed Files and Repository tabs plus data agents in preview.

Flow

Right after M1's concepts; next the 15-minute break, then the 10:40 loading lab writing into class.

Links

<https://cloud.google.com/bigquery/docs/bigquery-web-ui>

<https://cloud.google.com/bigquery/docs/resource-hierarchy>

<https://cloud.google.com/bigquery/docs/datasets-intro>

Morning Break

15 minutes · 10:25–10:40

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 47 OF 135 · 10:25 · MORNING BREAK (15 MIN)

FACILITATOR NOTE

- Fifteen minutes, back at 10:40 sharp — the loading lab is exactly 30 minutes before M2 starts, so protect the restart time.
- While they step away, spot-check a few screens: the class dataset is the one hard prerequisite for this afternoon's 13:45 loading block — anyone missing it should create it during the lab's dead time, not at 13:45.
- Good moment to ask how the console felt; expect wrong-project confusion to be the top complaint and re-show the project picker after the break.

Flow: Sits between the console orientation they just finished and the 10:40 Loading Data into BigQuery lab.

Lab — Loading Data into BigQuery

30 minutes — 10:40–11:10 · Google Skills classroom — Course 1, Module 1 lab

1. Launch the lab from the Skills classroom — lab timer running, lab credentials, right project
2. Load data into BigQuery from various sources through the Cloud console
3. Repeat a load from Cloud Shell with the bq command-line tool
4. Create and load a table with DDL — load logic as code
5. Early? DIN §24: load a CSV from jwd-gcp-demos into your class dataset — note the load time

Why it matters — the simplest version of what your nightly extract jobs do — a file becomes a queryable table. Deliberately easy: a first win in an unfamiliar console.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 48 OF 135 · 10:40 · LAB — LOADING DATA INTO BIGQUERY (30 MIN)

Hook

Every nightly extract job reduces to this: a file becomes a queryable table. Deliberately easy — a first win in an unfamiliar console, not a skills test.

Say · cover

- Gate the start: lab launched, timer running, lab credentials, project picker on the lab project — open instructions are not a running lab.
- Three ways in, same destination: console Create table (the Import/Export Wizard done right), bq load (the BCP shape), DDL (load logic as code).
- Autodetect yields one giant string column? Note it and move on — 13:45 reproduces that failure on purpose.
- Enable-API prompts and first-run Cloud Shell consents are normal; click through.
- Early finishers: DIN §24 — load the jwd-gcp-demos CSV into class, record load time for the §26 CSV-vs-Parquet race.

Iowa tie

Console load replaces one-off flat-file SSIS packages, bq load replaces BCP/BULK INSERT, DDL rides their CI/CD discipline. Datastream at 11:10 is the fifth way in — the extract-killer.

Design patterns

- Prefer DDL or scripted bq for anything recurring — load logic belongs in source control.
- Declare schemas explicitly for production loads.

Anti-patterns

- Autodetect on recurring production loads — column types drift silently.
- GUI-only click-through for nightly jobs: nothing to version or redeploy.

History & what's new

The LOAD DATA SQL statement went into preview in 2022 and is now GA; October 2024 release notes added loading Parquet files with geospatial data straight into native GEOGRAPHY columns.

Flow

First hands-on win, built on the class dataset from 10:10; next is M2 at 11:10 — Datastream, killing the nightly extracts.

Links

<https://cloud.google.com/bigquery/docs/loading-data>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/load-statements>

<https://cloud.google.com/bigquery/docs/bq-command-line-tool>

WATCH FIRST · 2:11 · GOOGLE CLOUD TECH

Replicate MySQL → BigQuery with Datastream

- ✓ A live change-data-capture stream, configured in two minutes: backfill once, then every INSERT, UPDATE, DELETE lands in BigQuery.
- ✓ MySQL in the demo, SQL Server in your world — same log-based capture, same console. Your 11:50 lab is the PostgreSQL twin of exactly this.



Scan to open on YouTube
youtu.be/vkCgqUILeeU

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 49 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

- Play right after the M2 opener — it makes CDC concrete before you ever say 'log-based'.

Say

- “Swap MySQL for SQL Server in your head — Datastream's source list includes it; that is your migration path.”

Cue

- The full 2:11 plays fine; the stream-creation clicks at ~1:00 are what the lab mirrors step for step.

Link

- <https://www.youtube.com/watch?v=vkCgqUILeeU>

M2 — What this block gets you

- ✓ Replication vs migration— backfill once, then a live change stream; Datastream does both as one operation
- ✓ Change data capture (CDC) as a managed service— every INSERT, UPDATE, DELETE lands in BigQuery
- ✓ SQL Server is a supported Datastream source— your SQL Server → BigQuery path is first-class
- ✓ Log-based capture— no triggers, no polling, no production load (the answer for your M&O team)
- ✓ Hands-on: start a Datastream stream before lunch, validate the replication after it

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 50 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

This is the block your migration is named after. Every nightly extract you run exists only because data can't move any other way — this module removes that constraint.

Say · cover

- Migration is one backfill; replication is continuous sync; Datastream does both as a single stream.
- CDC as a managed service: every INSERT, UPDATE, DELETE lands in BigQuery with no job you schedule.
- SQL Server is a first-class Datastream source — your path, not a workaround.
- Log-based capture means no triggers and no polling — the answer your M&O team will ask for.
- At 11:50 we start a real stream; after lunch we validate what it replicated.

Iowa tie

This module is the kill-the-nightly-extracts plan: the SQL Server to BigQuery feed that becomes the top of your Bronze layer.

Design patterns

- Backfill once, then stream changes forever — never re-extract a whole table.

Anti-patterns

- Keeping nightly truncate-and-reload jobs when CDC could carry only the deltas.

History & what's new

Datastream entered preview in 2021 and gained direct BigQuery replication in September 2022; SQL Server as a source went GA in July 2024, which is why this course can teach your exact path.

Flow

Lab 1 (file loading into BQ) just ended; this block runs 11:10–11:50, then the Datastream lab starts before lunch.

Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/datastream/docs/sources>

<https://cloud.google.com/datastream/docs/quickstart-replication-to-bigquery>

This block, across the lunch break

11:10–11:50	M2 — Data replication & migration: killing the nightly extracts	
11:50–12:00	Lab: Datastream PostgreSQL → BigQuery, part 1 — set up and start the stream	LAB
12:00–13:00	Lunch — the stream replicates while we eat	
13:00–13:15	Lab part 2 + CDC demo — validate backfill, watch a live change land	LAB
13:15–13:45	M3 — Extract & Load: the pattern for data that still arrives as files	

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 51 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Notice the design of the next two hours: the stream does its real work while you are at lunch — that unattended replication is the whole pitch.

Say · cover

- 11:10–11:50: concepts — replication vs migration, CDC, the four transfer tools.
- 11:50–12:00: lab part 1 — two connection profiles, one stream, start it.
- 12:00–13:00: lunch while the stream backfills and replicates on its own.
- 13:00–13:15: lab part 2 — validate row counts, then watch a live UPDATE land.
- The timing is deliberate: freshness with no batch window is a feeling, not a fact — we want you to feel it.

Iowa tie

The split lab mirrors your project plan: initial copy of the warehouse, then continuous changes into BigQuery — no maintenance window.

Design patterns

- Prove CDC on a small pilot table before committing the biggest ones.

Anti-patterns

- Pausing or cleaning up the stream at lunch — the unattended replication IS the lesson.

History & what's new

The lab uses the direct Datastream-to-BigQuery path Google made GA in September 2022; SQL Server joined the GA source list in July 2024.

Flow

Orientation card for the block; next is the module divider, then the transfer-tool landscape.

Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/datastream/docs/create-a-stream>

<https://cloud.google.com/datastream/docs/quickstart-replication-to-bigquery>



Data replication and migration

Google Cloud

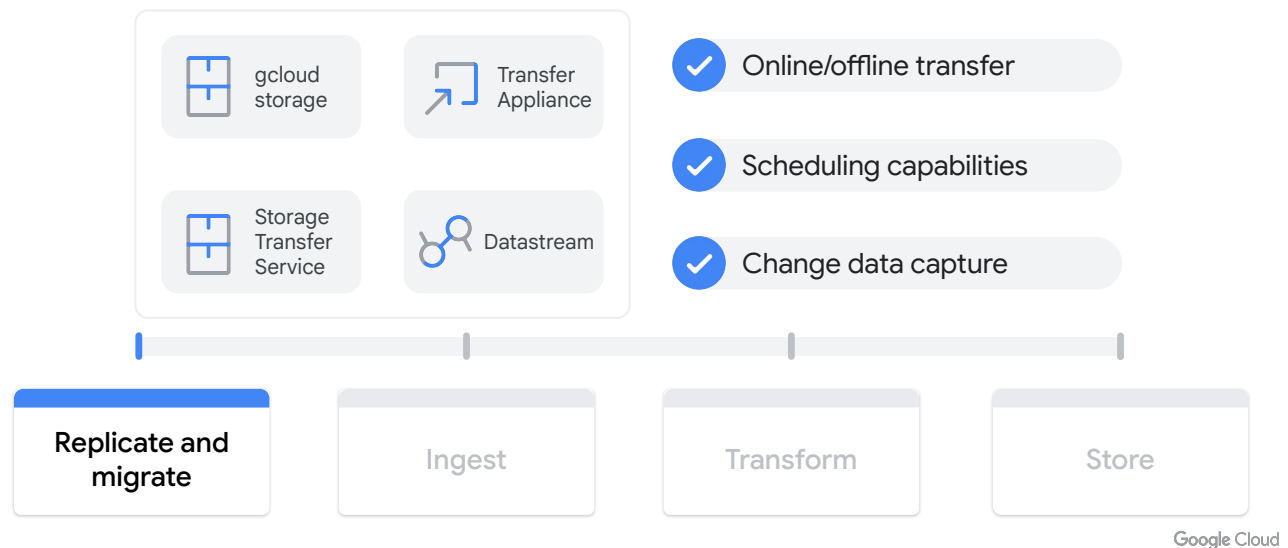
INSTRUCTOR NOTES — PAGE 52 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

FACILITATOR NOTE

- Section divider for Module 2 — Data replication and migration. Take a beat and frame the promise: by 13:15 everyone will have a live replication stream landing changes in BigQuery.
- Quick show of hands while people settle: who runs nightly extracts today, and how long is the window? You will cash those answers in on the WAL and M&O slides.

Flow: Divider; next slide is the transfer-service landscape at the Ingest stage (Ingest, Transform, Store).

Replication and migration services onboard your data into Google Cloud



INSTRUCTOR NOTES — PAGE 53 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Four tools, one decision axis: how much data, and how often it changes. You already know this tradeoff from sizing your SSIS extract jobs.

Say · cover

- This slide sits at the Ingest stage of Ingest, Transform, Store — everything here lands data into Google Cloud.
- gcloud storage: CLI for small or ad-hoc copies — the robocopy equivalent.
- Storage Transfer Service: managed and scheduled; S3, Azure Blob, on-prem file systems, HDFS.
- Transfer Appliance: a physical, offline box for data too big for the wire.
- Datastream: the only one doing continuous CDC from databases — the one your project cares about.
- Scheduling capability is what separates managed services from one-off CLI copies.

Iowa tie

Files that still arrive as extracts ride gcloud storage or STS into Bronze buckets; the warehouse itself migrates via Datastream.

Design patterns

- Match the tool to data size plus change frequency, not to habit.

Anti-patterns

- Scripting your own scheduled copy jobs when STS already provides scheduling, retries, and logging.

History & what's new

Google replaced gsutil with the far faster gcloud storage CLI (GA 2022, up to 94% faster transfers), and Datastream added a BigLake Iceberg-table destination in June 2025.

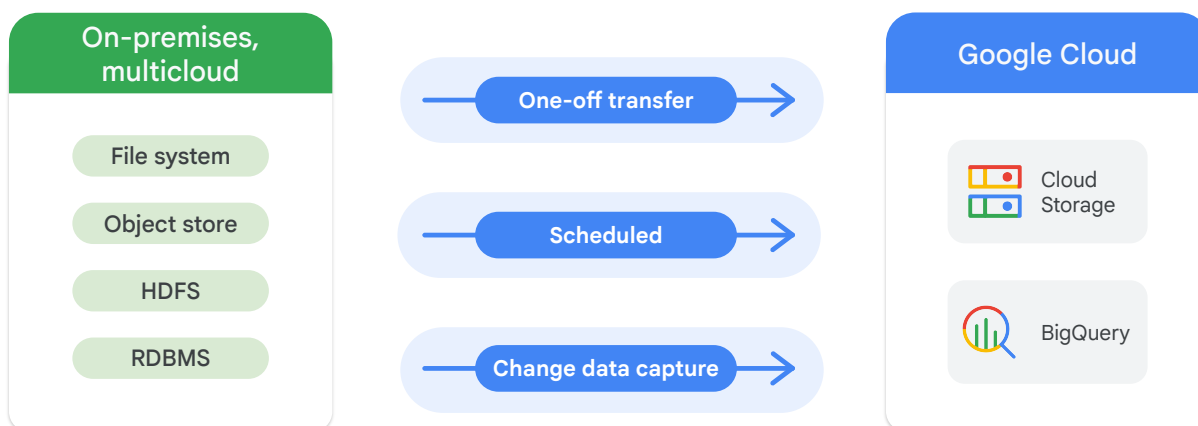
Flow

First content slide of M2 after the divider; next: which ingestion scenarios map to which tool.

Links

<https://cloud.google.com/storage-transfer/docs/overview>
<https://cloud.google.com/transfer-appliance/docs/4.0/overview>
<https://cloud.google.com/datastream/docs/overview>

Common replication and migration scenarios for ingesting your data into Google Cloud



Google Cloud

INSTRUCTOR NOTES — PAGE 54 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Every arrow on this slide is something you do today with a SQL Agent job and a shared drive — here is the managed equivalent of each.

Say · cover

- Left side is your estate: file systems, object stores, HDFS, relational databases — on-prem or multicloud.
- Three trigger patterns: one-off transfer, scheduled recurring transfer, and change data capture.
- Files and objects land in Cloud Storage; database changes can land in BigQuery directly.
- RDBMS sources are the special case: they change continuously, so they get CDC instead of copies.
- Map your world: nightly extract files equal scheduled; the warehouse migration equals backfill plus CDC.

Iowa tie

Your SQL Server warehouse is the RDBMS to CDC to BigQuery arrow; file drops still take the scheduled route into Bronze buckets.

Design patterns

- Land files in Cloud Storage (Bronze) and stream databases straight into BigQuery.

Anti-patterns

- Exporting a database to CSV nightly to treat it as a file source — you lose deletes and deltas.

History & what's new

Storage Transfer Service began as an S3 importer, added agent-based on-premises transfers in late 2019, and picked up Azure ADLS Gen2 sources in 2021.

Flow

From the tool landscape to the scenarios those tools cover; next: the size-vs-bandwidth selection rule.

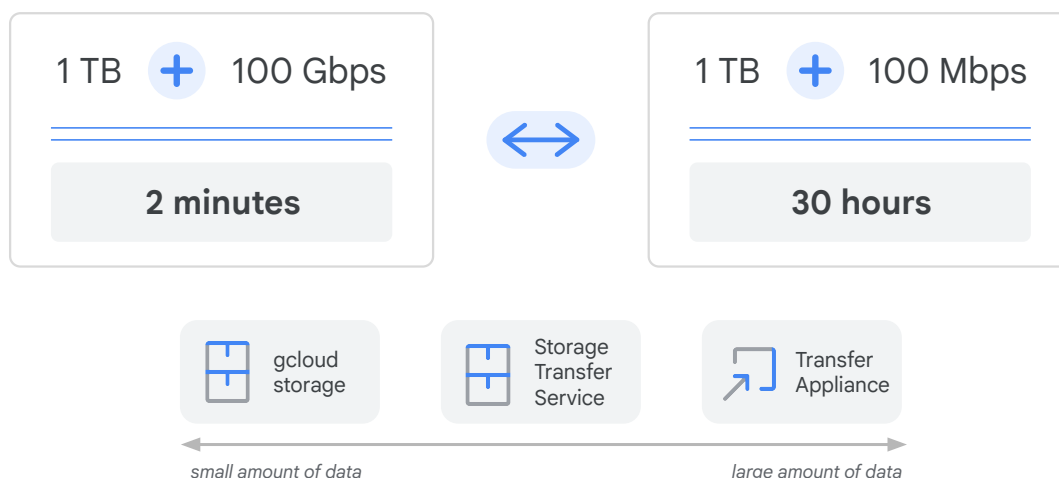
Links

<https://cloud.google.com/storage-transfer/docs/overview>

<https://cloud.google.com/datastream/docs/sources>

<https://cloud.google.com/architecture/migration-to-google-cloud-transferring-your-large-datasets>

Choosing between migration options depends on your amount of data and network bandwidth



Google Cloud

INSTRUCTOR NOTES — PAGE 55 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Do the math out loud: your largest table on your actual WAN link — how long? If the answer embarrasses the network, Google mails you a box.

Say · cover

- The rule: online transfer time equals data size divided by usable bandwidth — and usable is far below the link's sticker speed.
- 1 TB at 100 Gbps is about 2 minutes; the same 1 TB at 100 Mbps is about 30 hours — three orders of magnitude.
- gcloud storage for small ad-hoc moves; Storage Transfer Service for large scheduled ones; Transfer Appliance when the wire cannot win.
- Ask the room for their largest table size right now — the run sheet flags this as a live question.
- Online tools resume and retry; the appliance trades shipping days for impossible bandwidth.

Iowa tie

For the one-time warehouse backfill this math decides: Datastream online if the link holds; staged appliance or STS for the file archives.

Design patterns

- Compute transfer time from measured throughput, not link spec, before choosing online vs offline.

Anti-patterns

- Starting a multi-TB online transfer on an unmeasured link the week before cutover.

History & what's new

Transfer Appliance shipped in 2017 with 100 TB and 480 TB units; the current TA40/TA300 generation (40/300 TB) arrived in 2021.

Flow

Scenario mapping done, selection rule delivered; next slide drills into Storage Transfer Service.

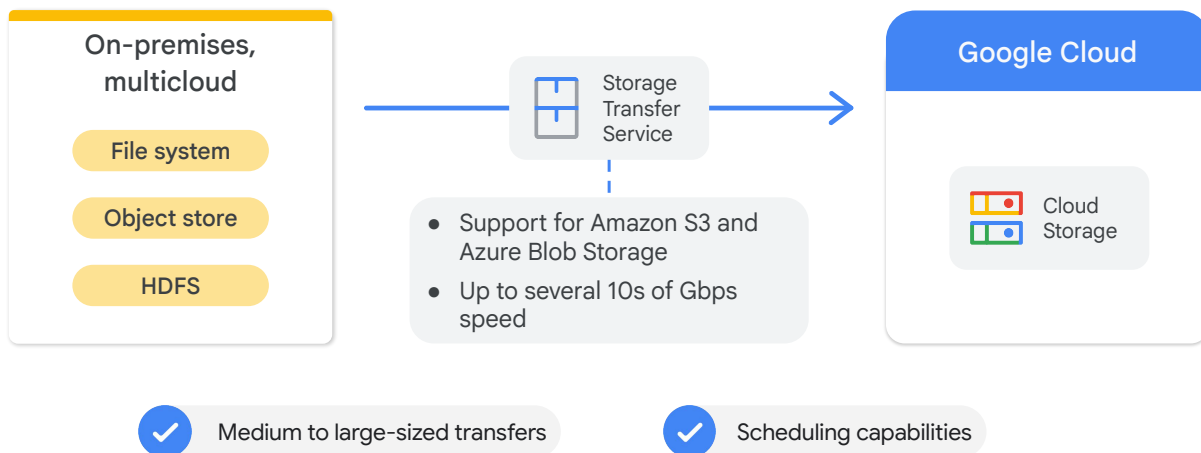
Links

<https://cloud.google.com/architecture/migration-to-google-cloud-transferring-your-large-datasets>

<https://cloud.google.com/storage-transfer/docs/4.0/overview>

<https://cloud.google.com/sdk/gcloud/reference/storage>

Move large amounts of data with Storage Transfer Service



Google Cloud

INSTRUCTOR NOTES — PAGE 56 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

This is the managed successor to every robocopy script and cron'd copy job anyone has ever written — and it speaks S3 and Azure Blob natively.

Say · cover

- Sources: Amazon S3, Azure Blob, on-prem file systems, HDFS, URL lists — destination is Cloud Storage.
- Scheduling down to hourly, with incremental runs that copy only what changed.
- Self-hosted agents reach tens of Gbps and give you bandwidth control on your network.
- For Iowa: this is how legacy extract files and archive shares land in your Bronze bucket on a schedule.
- Not for databases — no CDC, no tables; that gap is exactly what Datastream fills next.

Iowa tie

Bronze-landing for files: scheduled STS jobs replace the hand-rolled copy scripts that feed your lakehouse today.

Design patterns

- Use scheduled incremental STS jobs for recurring file feeds; let the service own retries and logs.

Anti-patterns

- Using STS for database replication — it moves objects, not row-level changes.

History & what's new

STS added agent-based on-prem transfers in 2019, gcloud-managed transfer jobs GA'd in March 2022, and event-driven transfers (Pub/Sub-triggered) entered preview in January 2023.

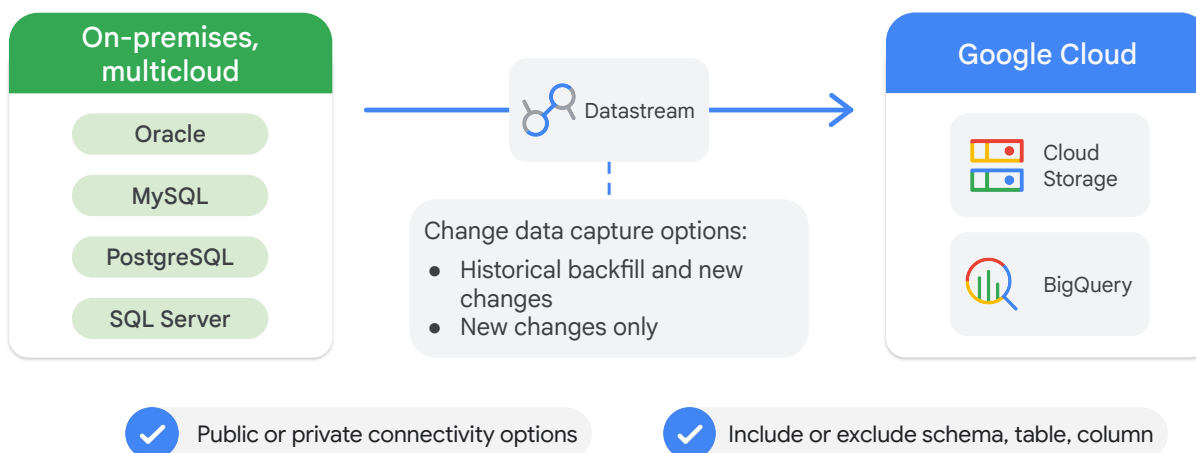
Flow

The file-moving tool covered; next slide is the star of the module: Datastream for databases.

Links

<https://cloud.google.com/storage-transfer/docs/overview>
<https://cloud.google.com/storage-transfer/docs/on-prem-overview>
<https://cloud.google.com/storage-transfer/docs/event-driven-transfers>

Datastream continuously replicates your RDBMS into Google Cloud for analytics



Google Cloud

INSTRUCTOR NOTES — PAGE 57 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Here is the sentence your whole project hinges on: a managed service reads your database's own change log and lands every change in BigQuery.

Say · cover

- Sources: Oracle, MySQL, PostgreSQL, and SQL Server — self-managed or cloud-hosted.
- Two capture modes in one stream: historical backfill plus new changes, or new changes only.
- Destinations: BigQuery directly, or Cloud Storage as Avro/JSON events for custom processing.
- Public or private connectivity — VPN or Interconnect via VPC peering for on-prem sources.
- Include or exclude at schema, table, or column level: replicate the marts you need, not the whole instance.

Iowa tie

This is the SQL Server to BigQuery replication that retires the nightly extracts and feeds your Bronze layer continuously.

Design patterns

- Scope streams to the tables the warehouse actually needs; use private connectivity for production sources.

Anti-patterns

- Replicating entire instances by default — cost and noise for tables nobody reports on.

History & what's new

Datastream entered preview in May 2021 and direct BigQuery replication GA'd in September 2022; MongoDB and Salesforce sources went GA in September 2025.

Flow

File tools done, the database path introduced; next slide confirms SQL Server support explicitly.

Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/datastream/docs/sources>

<https://cloud.google.com/datastream/docs/quickstart-replication-to-bigquery>

SQL Server IS a supported Datastream source

The deck's source graphic is the short list — SQL Server is fully supported.

- Self-managed or cloud-hosted SQL Server— supported alongside Oracle, MySQL and PostgreSQL
- Captured from the SQL Server transaction logs— the same log your databases already write
- Your SQL Server → BigQuery replication is a first-class Datastream scenario, not a workaround
- Today's lab uses PostgreSQL only because it is free-tier friendly— the mechanics are identical
- This lab is your project's pilot: source database → continuous changes → BigQuery

If the room takes one sentence from this module: Datastream supports SQL Server, and that is your path.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 58 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Say it verbatim: Datastream supports SQL Server. The deck's graphic is behind the docs — your migration path is first-class.

Say · cover

- Supported: self-managed on-prem or VM, Amazon RDS for SQL Server, Cloud SQL for SQL Server, Azure SQL Database (tier S3 and later).
- Capture comes from the SQL Server transaction logs — the same log your databases already write.
- Today's lab uses PostgreSQL only because it is free-tier friendly; every click maps one-to-one onto SQL Server.
- Connection profiles, streams, backfill, CDC — same mechanics, same console pages.
- Treat this lab as your project's pilot: source database, continuous changes, BigQuery.

Iowa tie

Your warehouse is SQL Server — this slide is the reassurance that the path is documented, GA, and supported, not a workaround.

Design patterns

- Pilot on a non-production SQL Server database with CDC enabled before touching the warehouse.

Anti-patterns

- Assuming the deck's short source graphic is the full list — always check the supported-sources doc.

History & what's new

SQL Server as a Datastream source went GA on July 10, 2024 — recent enough that older decks and blog posts still omit it.

Flow

Datastream introduced, Iowa-specific confirmation delivered; next: what teams build with the change stream.

Links

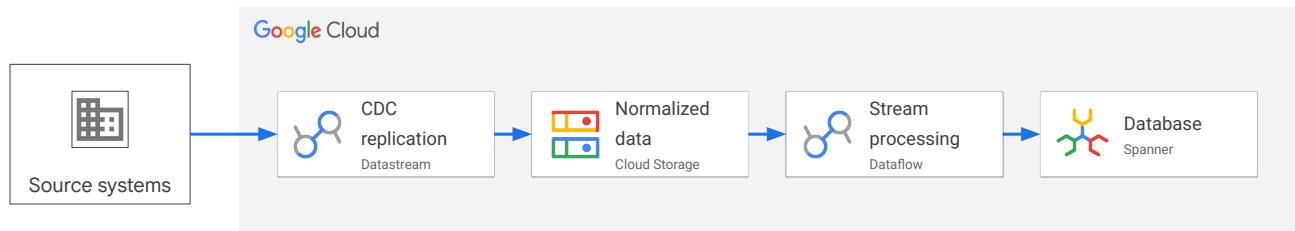
<https://cloud.google.com/datastream/docs/configure-your-source-sql-server-database>

<https://cloud.google.com/blog/products/data-analytics/datastream-makes-it-easy-to-replicate-sql-server-databases>

<https://cloud.google.com/datastream/docs/sources>

Datastream use cases

- Analytics with database replication into BigQuery
- Analytics with custom data processing in Dataflow into BigQuery
- Data processing using an event-driven architecture
- **Example:** database replication and migration using Dataflow templates



Google Cloud

INSTRUCTOR NOTES — PAGE 59 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Four ways teams consume the change stream — you will use the first on day one, and the second when you outgrow straight replication.

Say · cover

- Direct replication into BigQuery: normalized, query-ready tables — your Bronze and Silver feed.
- Stream to Cloud Storage, then Dataflow templates for custom processing into BigQuery, Spanner, or Cloud SQL.
- Event-driven architectures: treat database changes as events for downstream systems.
- The Cloud Storage landing pattern gives you a replayable raw event log — auditability M&O will appreciate.
- Start direct-to-BigQuery; add Dataflow only when you genuinely need enrichment in flight.

Iowa tie

Path one (Datastream straight to BigQuery) feeds Bronze; Dataform then handles Silver to Gold — no Dataflow needed in your design.

Design patterns

- Start with the direct BigQuery destination; keep transformations in Dataform, not in flight.

Anti-patterns

- Building a Dataflow processing stage before the direct path proves insufficient — premature complexity.

History & what's new

Google keeps extending the direct path: in November 2025 Datastream added BigQuery partition and cluster configuration for replicated tables.

Flow

What Datastream is and what you build with it; next: how capture actually works — the write-ahead log.

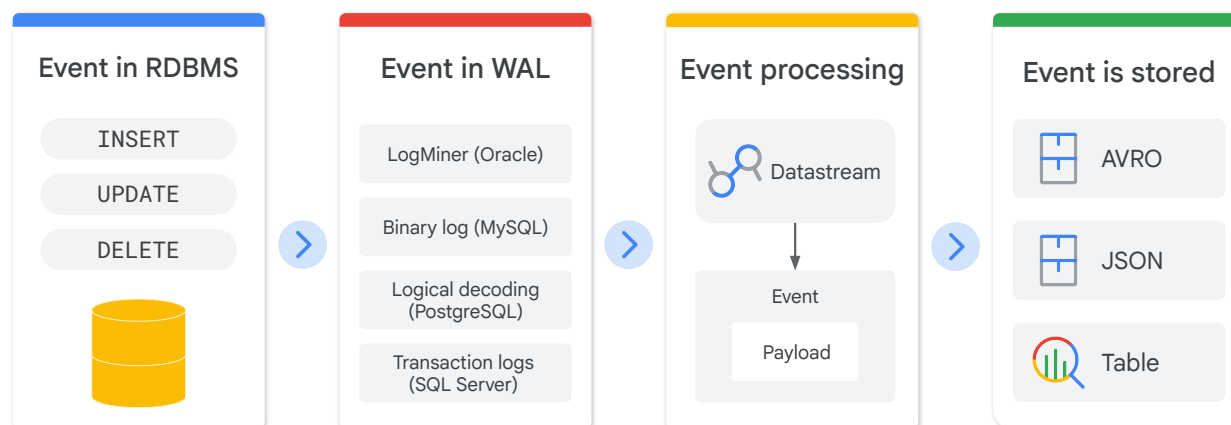
Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/datastream/docs/destination-bigquery>

<https://cloud.google.com/datastream/docs/destination-gcs>

Datastream uses the database's write-ahead log (WAL) to process change events



Google Cloud

INSTRUCTOR NOTES — PAGE 60 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Every database you manage already writes a diary of every change — CDC just reads the diary instead of interrogating the tables.

Say · cover

- Every INSERT, UPDATE, DELETE commits to the write-ahead or transaction log first — that is the durability guarantee you already rely on.
- Datastream tails that log: transaction logs for SQL Server, logical decoding for PostgreSQL, LogMiner for Oracle, binary log for MySQL.
- Events are normalized into Avro or JSON payloads with a unified schema on the way out.
- Log retention matters: entries must survive until Datastream reads them — a configuration conversation to have with your DBAs.
- The read is sequential I/O on a log file, not queries against your tables.

Iowa tie

For your SQL Server estate, this slide implies exactly two DBA work items: enable CDC capture and size log retention for the stream.

Design patterns

- Size log retention for the longest expected stream outage, not the average one.

Anti-patterns

- Trigger-based or timestamp-polling CDC when a log-based service exists — write-path load and missed deletes.

History & what's new

Datastream reads SQL Server transaction logs the same way SQL Server's own CDC feature does; Google GA'd the SQL Server source in July 2024 with documented DBA setup steps.

Flow

Capture mechanics covered; next slide turns this into the objection-handling script for your M&O team.

Links

<https://cloud.google.com/datastream/docs/configure-your-source-sql-server-database>
<https://cloud.google.com/datastream/docs/sources>
<https://cloud.google.com/datastream/docs/overview>

Log-based CDC — the answer for your M&O team

Datastream reads the write-ahead log / transaction log the database is already writing.

- No triggers— nothing is added to your tables or to the write path
- No polling— Datastream never runs change-detection queries against production
- No batch window— there is no 1 AM–4 AM slot where the source gets hammered
- The source keeps serving its workload— capture reads a log, not the tables
- Deletes arrive too— change_type and is_deleted metadata on every replicated row

Pre-empt the objection before M&O raises it: replication adds no triggers, no polling, no production load.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 61 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Your M&O team will ask exactly one question: what does this do to production? This slide is your answer, memorized.

Say · cover

- No triggers — nothing is added to your tables or to the write path.
- No polling — Datastream never runs change-detection queries against production.
- No batch window — the 1 AM to 4 AM extract slot simply ceases to exist.
- The source keeps serving its workload; capture reads a log the database already writes.
- Deletes arrive too — change_type and is_deleted metadata ride on every replicated row, so nothing silently vanishes.

Iowa tie

This pre-empts the M&O objection to streaming your SQL Server warehouse — bring this slide verbatim to the ops review.

Design patterns

- Hand M&O the log-based story plus the stream monitoring page before they raise the objection.

Anti-patterns

- Hand-rolled delta detection (triggers, checksum columns): write-path load plus logic you must maintain forever.

History & what's new

Google lists minimal source impact as a core Datastream design goal; the SQL Server source reached GA in 2024 with documented, DBA-friendly CDC configuration.

Flow

The M&O answer delivered; next: how data types survive the journey — unified type mapping.

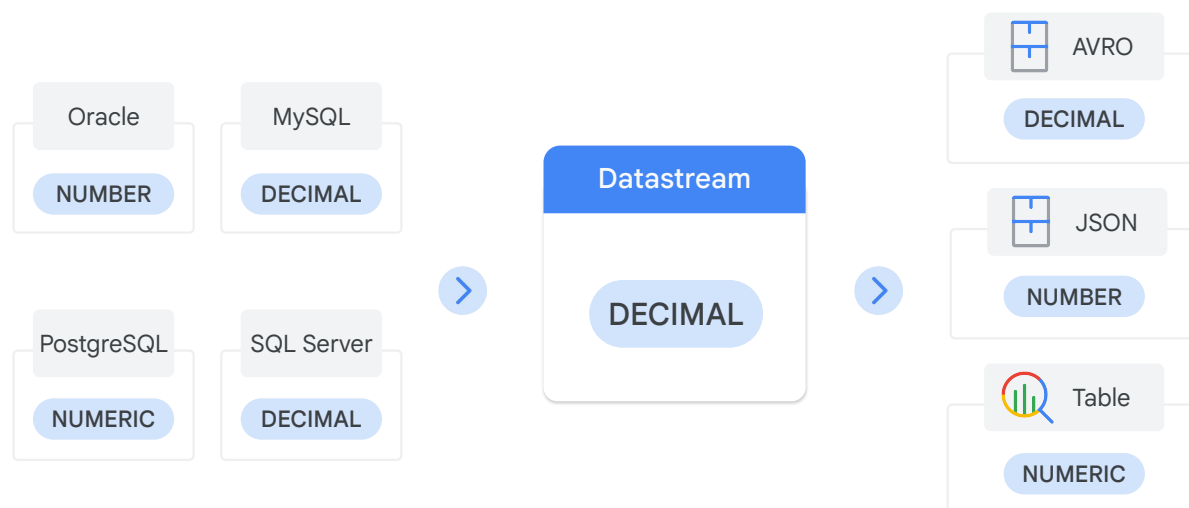
Links

<https://cloud.google.com/datastream/docs/monitor-a-stream>

<https://cloud.google.com/datastream/docs/configure-your-source-sql-server-database>

<https://cloud.google.com/datastream/docs/overview>

Datastream uses unified types to map source to destination data types



Google Cloud

INSTRUCTOR NOTES — PAGE 62 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

DECIMAL here, NUMBER there, NUMERIC somewhere else — Datastream picks one canonical type system so your schema survives translation.

Say · cover

- Each source type maps to a lossless unified type — a superset across all sources — then onward to the destination type.
- SQL Server examples: DATETIMEOFFSET becomes TIMESTAMP WITH TIME ZONE; MONEY and SMALLMONEY become DECIMAL; UNIQUEIDENTIFIER becomes STRING.
- TIMESTAMP/ROWVERSION becomes BYTES — it is a row version, not a date; do not build logic on it after migration.
- Into BigQuery, unified types land as native BigQuery types; check the mapping table for your tricky columns before the pilot.
- Events written to Cloud Storage carry the same types as Avro or JSON — one schema story either way.

Iowa tie

Concrete homework: run your warehouse column inventory against the SQL Server mapping table before the pilot stream.

Design patterns

- Audit exotic types (DATETIMEOFFSET, MONEY, ROWVERSION) against the mapping doc at design time.

Anti-patterns

- Discovering type surprises at report-validation time instead of schema-mapping time.

History & what's new

The unified-types table keeps growing — Salesforce and MongoDB mappings joined when those sources went GA in September 2025.

Flow

Types covered; next: the four-way comparison table that ties the module together before the lab.

Links

<https://cloud.google.com/datastream/docs/unified-types>

<https://cloud.google.com/datastream/docs/destination-bigquery>

<https://cloud.google.com/datastream/docs/configure-your-source-sql-server-database>

Compare migration and replication options

	gcloud storage	Storage Transfer Service	Transfer Appliance	Datastream
Transfer type	online	online	offline	online
Source locations	On-premises, Google Cloud	On-premises, Google Cloud, multicloud	On-premises	On-premises, Google Cloud, multicloud
Data range	less than 1 TB recommended	more than 1 TB recommended	7 TB, 40 TB, 300 TB	10,000 tables per stream
Velocity	Batch	Batch (hourly at minimum)	Batch	Batch and Stream
Data format	any	any	any	structured

Google Cloud

INSTRUCTOR NOTES — PAGE 63 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Screenshot this table — it is the one-slide answer to which-tool-for-which-job in every migration meeting from now on.

Say · cover

- gcloud storage: online CLI, under about 1 TB recommended, batch, any format — ad-hoc work.
- Storage Transfer Service: online and managed, over 1 TB recommended, scheduled batch (hourly at minimum), any format.
- Transfer Appliance: offline, 7, 40, or 300 TB units — when the bandwidth math fails.
- Datastream: online, structured data only, up to 10,000 tables per stream — and the only option that streams.
- Velocity is the separator: three batch tools and one streaming tool; live databases deserve the stream.

Iowa tie

Your selection: Datastream for the warehouse (streaming), STS for scheduled file feeds into Bronze, appliance only if archive backfill breaks the wire.

Design patterns

- Choose on velocity first (batch vs stream), then size, then format.

Anti-patterns

- One-tool-fits-all thinking — forcing Datastream to move files or STS to replicate tables.

History & what's new

The lineup keeps moving: gcloud storage GA'd in 2022, and Datastream added BigLake Iceberg tables as a destination in June 2025.

Flow

Concepts complete; next slide opens the lab we split around lunch.

Links

<https://cloud.google.com/storage-transfer/docs/overview>

<https://cloud.google.com/transfer-appliance/docs/4.0/overview>

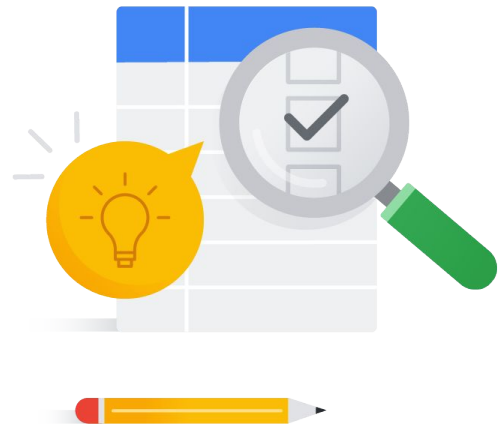
<https://cloud.google.com/datastream/docs/overview>

Lab (optional): Datastream: PostgreSQL Replication to BigQuery

🕒 45 min

Learning objectives

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database.
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data and changes are replicated correctly into BigQuery.



Google Cloud

INSTRUCTOR NOTES — PAGE 64 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Everything we just covered, you are about to do with your own hands in ten minutes — then lunch does the teaching.

Say · cover

- Objectives: prepare Cloud SQL for PostgreSQL, seed data, create two connection profiles, create a stream, validate replication.
- PostgreSQL only because it is free-tier friendly — the mechanics are identical for SQL Server.
- A connection profile is a saved, tested connection definition — the SSIS connection manager, basically.
- We split the lab: ten minutes of setup before lunch, fifteen minutes of validation after.
- The step-by-step lives in the Google Skills classroom; your companion sheet lists the common gotchas.

Iowa tie

This lab is your migration pilot in miniature: source database, backfill plus CDC, BigQuery tables at the far end.

Design patterns

- Follow click-for-click in part 1; part 2 is where understanding gets checked.

Anti-patterns

- Rushing the region setting — region mismatch between profiles, source, and the BQ dataset is the classic first failure.

History & what's new

The quickstart this lab mirrors — replicate to BigQuery in real time — has existed since the direct BigQuery destination GA'd in September 2022.

Flow

Concepts done; the next two cards are the lab runbook, then lunch while the stream replicates.

Links

<https://cloud.google.com/datastream/docs/quickstart-replication-to-bigquery>
<https://cloud.google.com/datastream/docs/create-connection-profiles>
<https://cloud.google.com/datastream/docs/create-a-stream>

Lab part 1 — Datastream: PostgreSQL to BigQuery (setup & start)

10 minutes — 11:50–12:00, then lunch while it runs · Google Skills classroom — your pre-warmed lab project

1. Open the module 2 lab — the Cloud SQL for PostgreSQL instance is pre-warmed
2. Follow the instructor click-for-click: create the connection profile for the PostgreSQL source
3. Create the connection profile for the BigQuery destination — regions must match
4. Create the stream and start replication — hands up when stream status shows Running
5. Leave everything running and break for lunch — do not pause or clean up

Why it matters — the stream backfills and replicates while we eat — at 13:00 you validate what landed. PostgreSQL is here because it is free-tier friendly; the mechanics are identical for SQL Server.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 65 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Ten minutes, four steps, then we walk away — the point is what happens while we are gone.

Say · cover

- Work in your pre-warmed lab project — check the project picker before creating anything.
- Source connection profile points at Cloud SQL for PostgreSQL — the connection-manager idea from SSIS.
- Destination profile points at BigQuery; its region must match the dataset location or the connection test fails.
- Create the stream, choose what to replicate, start it; hands up when status shows Running.
- Leave everything running over lunch — do not pause, stop, or clean up.

Iowa tie

Same skeleton your real stream will have — SQL Server profile, BigQuery profile, stream — this is the rehearsal.

Design patterns

- Test each connection profile before creating the stream; watch Starting turn to Running on the Streams page.

Anti-patterns

- Debugging a broken stream into the lunch break — watch the shared validation at 13:00 and re-run the lab in the evening.

History & what's new

The tooling keeps improving around this flow — Datastream added native BigQuery partition and cluster configuration for streamed tables in November 2025.

Flow

Lab kickoff at 11:50, stream starts before lunch; next card is the after-lunch validation plan.

Links

<https://cloud.google.com/datastream/docs/create-connection-profiles>

<https://cloud.google.com/datastream/docs/create-a-stream>

<https://cloud.google.com/datastream/docs/run-a-stream>

Lab part 2 — validate backfill and changes (after lunch)

15 minutes — 13:00–13:15 · BigQuery console + your lab's Cloud SQL instance

1. Open the BigQuery tables the stream created — destination tables appear as the stream runs
2. Validate the backfill: row counts vs the source, eyeball the data
3. Inspect the Datastream metadata columns — `change_type` and `is_deleted` on every row
4. Apply the lab's source change (or your own UPDATE/INSERT) against Cloud SQL
5. Poll the BigQuery table until the change lands — expect roughly a minute

Why it matters — one-time backfill, then incremental changes forever — this is the end of nightly truncate-and-reload for your largest tables.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 66 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Welcome back — your data has been replicating for an hour with nobody watching. That is not a scheduling accident; that is the product.

Say · cover

- Find the BigQuery tables the stream created — they appear as the stream runs; refresh the Explorer panel.
- Validate the backfill: row counts against the source, then eyeball the data.
- Inspect the Datastream metadata columns — `change_type` and `is_deleted` ride along on every row.
- Run the lab's UPDATE or INSERT against Cloud SQL, then poll the BQ table — expect roughly a minute.
- No job ran and nothing was scheduled — the change simply arrived.

Iowa tie

This is the end of truncate-and-reload for your biggest tables: backfill once, then incremental changes forever.

Design patterns

- Validate cheap to expensive: row counts first, spot-check rows second, live change last.

Anti-patterns

- Declaring the stream broken before checking the Streams status page and refreshing the table preview.

History & what's new

The `change_type` and `is_deleted` metadata columns are documented in the BigQuery-destination guide; native partition and cluster support for streamed tables arrived in 2025.

Flow

Validation runs 13:00–13:15 after lunch; the next card is the same moment on the instructor's screen.

Links

<https://cloud.google.com/datastream/docs/destination-bigquery>
<https://cloud.google.com/datastream/docs/manage-backfill-for-the-objects-of-a-stream>
<https://cloud.google.com/datastream/docs/monitor-a-stream>

Demo — CDC validation: a change lands while we watch

Inside lab part 2 — 13:00–13:15 · Instructor screen — Cloud SQL source to BigQuery destination

1. Run an UPDATE and an INSERT against the source Cloud SQL instance
2. Watch the Datastream stream status and throughput tick in the console
3. Poll the BigQuery destination table until the changed rows appear
4. Read the metadata on the new rows — `change_type` tells insert from update
5. Note the latency: source commit to BigQuery in about a minute, no job ran

Why it matters — your data changed while we ate lunch — no nightly job, no extract window, no SSIS package ran. That is the whole reason you are migrating.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 67 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

One UPDATE and one INSERT, committed while we were eating — and no SSIS package anywhere in sight.

Say · cover

- Run the UPDATE and INSERT against the source Cloud SQL instance, live on screen.
- Watch stream status and throughput tick on the Datastream Streams page.
- Poll the BigQuery table until the changed rows appear — about a minute from source commit to queryable.
- Read `change_type` on the new rows: it distinguishes insert from update, and `is_deleted` marks deletes.
- Land the beat: no nightly job, no extract window, no SSIS package ran — that is why you are migrating.

Iowa tie

The felt proof for your business case: freshness without a batch window — the sentence to repeat to stakeholders back home.

Design patterns

- Narrate the latency clock out loud: source commit time versus BigQuery-visible time.

Anti-patterns

- Promising second-level freshness to stakeholders; set the about-a-minute expectation now.

History & what's new

This demo needs no Dataflow pipeline at all — the direct BigQuery destination launched September 2022, and SQL Server joined the GA sources in July 2024.

Flow

The CDC moment closes the lab; next is the module wrap-up, then M3 on file-based loading at 13:15.

Links

<https://cloud.google.com/datastream/docs/monitor-a-stream>

<https://cloud.google.com/datastream/docs/destination-bigquery>

<https://cloud.google.com/datastream/docs/quickstart-replication-to-bigquery>

M2 — What you now have

- ✓ The distinction that frames your project— one-time backfill, then a continuous change stream, in one operation
- ✓ SQL Server confirmed as a supported Datastream source— your migration path is first-class
- ✓ The M&O answer in your pocket— log-based CDC: no triggers, no polling, no production load
- ✓ A stream you started before lunch and validated after— backfill plus a live change, seen end to end
- ✓ Next: data that still arrives as files— the Extract & Load pattern (M3, 13:15)

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 68 OF 135 · 11:10 · MODULE 2 — DATA REPLICATION & MIGRATION

Hook

Four takeaways — one of which your M&O team needs verbatim — plus a stream that is still running in your lab project.

Say · cover

- The framing distinction: one-time backfill, then a continuous change stream — one Datastream operation, not two projects.
- SQL Server confirmed as a supported, GA source — your migration path is first-class.
- Log-based CDC in your pocket: no triggers, no polling, no production load — the M&O answer.
- You started a stream before lunch and validated backfill plus a live change after it — end to end, not slideware.
- Next at 13:15: data that still arrives as files — the Extract & Load pattern.

Iowa tie

This module was your project's core — the Bronze feed that kills the nightly extracts; M3 next covers the file stragglers.

Design patterns

- Carry the four-option comparison table straight into your migration planning doc.

Anti-patterns

- Letting the stream mindset stop at the lab — scheduling the SQL Server pilot is the next work item back home.

History & what's new

The path you just walked has only fully existed since July 2024, when SQL Server became a GA Datastream source — you are early adopters of a first-class route.

Flow

Wrap of M2 and the split lab; next block is M3 Extract & Load at 13:15.

Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/datastream/docs/configure-your-source-sql-server-database>

<https://cloud.google.com/datastream/docs/release-notes>

Lab — Datastream: PostgreSQL → BigQuery (split across lunch)

10 min setup 11:50–12:00 + 15 min validation 13:00–13:15 · Google Skills classroom — Course 1, Module 2 lab

1. Part 1 (11:50): prepare the Cloud SQL for PostgreSQL source and import the seed data
2. Create two connection profiles — PostgreSQL source, BigQuery destination (same region!)
3. Create the stream and start it — then go to lunch; it backfills and replicates while you eat
4. Part 2 (13:00): validate the backfill — row counts, and the change_type / is_deleted columns
5. UPDATE/INSERT a row in the source and watch it land in BigQuery in about a minute — no job ran

Why it matters — your data changed while you ate lunch — no nightly job, no extract window. PostgreSQL is free-tier friendly here; the mechanics are identical for SQL Server, a supported Datastream source.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 69 OF 135 · 11:50 · LAB — DATASTREAM: POSTGRESQL -> BIGQUERY (SPLIT: 10 + 15 MIN)

Hook

Tonight's 2 a.m. SQL Agent job firing the SSIS package — you're about to start the stream that makes it unnecessary, then eat while it works.

Say · cover

- Part 1, click-for-click: seed the Cloud SQL source, create two connection profiles (source + BigQuery, same region), then create and start the stream.
- Connection profile = saved, tested connection — an SSIS connection manager; the stream binds both, running backfill plus log-based CDC.
- Console: Datastream under Analytics, Cloud SQL under Databases; confirm the lab project in the picker.
- Pause, stop, delete nothing over lunch — unattended replication while they eat IS the demo.
- 13:00: validate backfill (row counts, change_type / is_deleted), then UPDATE/INSERT a source row and watch it land within a minute — no job ran.

Iowa tie

Kill-the-extracts, hands-on: an always-on stream as the Bronze landing for their SQL Server sources; M&O will care it's log-based CDC — no triggers, no production polling.

Design patterns

- Keep source, profiles, stream, and dataset in one region — mismatch is the classic failure.
- Validate via metadata columns (change_type, is_deleted), not eyeballed rows.

Anti-patterns

- Hand-rolled delta detection (timestamps, triggers, checksums) when the change log already records everything.
- Debugging into the lunch window — check regions first, else use shared-screen validation.

History & what's new

Datastream went GA in 2021 as Google's serverless CDC service into BigQuery; SQL Server support followed. September 2026 release notes add partial backfill — WHERE-clause subsets of existing rows, SQL Server included.

Flow

Follows the M2 replication session; part 2 validation runs 13:00–13:15, then M3 Extract & Load.

Links

<https://docs.cloud.google.com/datastream/docs/quickstart-replication-to-bigquery>
<https://docs.cloud.google.com/datastream/docs/configure-your-source-sql-server-database>
<https://docs.cloud.google.com/datastream/docs/release-notes>

Lunch

60 minutes · 12:00–13:00

the Datastream stream keeps replicating while we eat

INSTRUCTOR NOTES — PAGE 70 OF 135 · 12:00 · LUNCH (60 MIN)

FACILITATOR NOTE

- Sixty minutes. Shut nothing down — the Datastream stream you started at 11:50 is backfilling and replicating while you eat, and that unattended replication is the whole lesson.
- Be back at 13:00 sharp: we validate the backfill in BigQuery, then make one change at the source and watch it land with no job running.

Flow: Part 2 of the Datastream lab resumes at 13:00, followed by M3 Extract & Load at 13:15.

WATCH FIRST · 3:24 · GOOGLE CLOUD TECH

What is Storage Transfer Service?

- ✓ The managed mover for bulk files — bucket-to-bucket, from S3, or from your on-prem file shares — no scripts, no transfer VM to babysit.
- ✓ This afternoon you load files by hand; this is what you hand the historical archive to when 'by hand' means 40 TB.



Scan to open on YouTube
youtu.be/1olw2mT_4-M

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 71 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD**Hook**

- Play as the M3 opener — it frames the four manual loads that follow as the small-data case.

Say

- “Console loads are for gigabytes; this service is for the archive sitting next to your SQL Server backups.”

Cue

- All 3:24; the use-case slide at ~2:00 is the 'when not to build this yourself' moment.

Link

- https://www.youtube.com/watch?v=1olw2mT_4-M

M3 — What this block gets you

- ✓ Extract & Load = your Bronze-landing pattern for files— land the file in BigQuery as-is, transform later
- ✓ One load job, three front doors— console form, bq load, and LOAD DATA SQL; batch loads are all free
- ✓ Autodetect is a first look, not a pipeline— pinned schemas in source control for anything recurring
- ✓ Data Transfer Service = scheduled ingest— the managed answer to your SQL Agent import jobs
- ✓ BigLake / external tables = query files where they live— the bridge to Day 2's lakehouse design

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 72 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Datastream covered databases that never stop changing; the next 30 minutes cover the other half — files arriving on a schedule, your SSIS flat-file territory.

Say · cover

- EL is the Bronze-landing pattern: land the file as-is, transform later in Dataform — land-first is what makes it Bronze.
- One load job, three front doors: console form, bq load, LOAD DATA SQL — same job type; pick per situation.
- Batch loads are free: they run on a shared slot pool, not your query slots — you will verify 'O B' billed shortly.
- Autodetect is a first look, not a pipeline: recurring feeds get pinned schemas in source control.
- DTS is the managed replacement for SQL Agent import jobs; external/Lakehouse tables query files without loading — the bridge to Day 2.

Iowa tie

The file-based half of kill-the-extracts: every nightly file drop becomes a Bronze table, zero transformation.

Design patterns

- Land as-is, transform later (Bronze discipline)
- Pin schemas for anything recurring
- Express loads as SQL in source control

Anti-patterns

- Autodetect in a production feed
- Row-by-row inserts for file drops
- One giant gzipped CSV as the landing format

History & what's new

BigQuery turned 10 in May 2020; its model has stayed 'land files free, pay to query'. Newest: April 2026 — Google renamed BigLake to Google Cloud Lakehouse, making this module's external-table half a first-class product family.

Flow

Just finished: lunch plus the Datastream CDC validation demo; next: the 30-minute module, then four hands-on loading DINs at 13:45.

Links

<https://cloud.google.com/bigquery/docs/loading-data>

<https://cloud.google.com/bigquery/docs/batch-loading-data>

<https://cloud.google.com/bigquery/docs/biglake-intro>

After lunch: land the files, 13:15–14:45

13:15–13:45	M3 Extract & Load — EL as Bronze landing; bq CLI; Data Transfer Service; BigLake / external tables	
13:45–14:30	Loading block, four free DINs — console & CLI load → autodetect pitfall → CSV vs Parquet race → LOAD DATA SQL	DIN
14:30–14:45	Break	

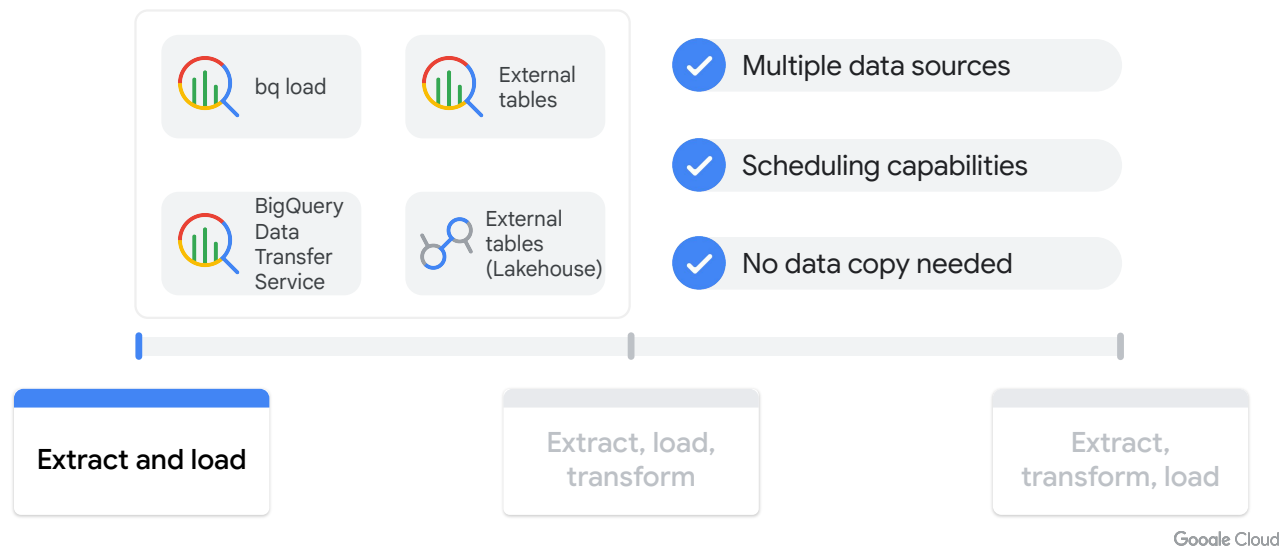
INSTRUCTOR NOTES — PAGE 73 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

FACILITATOR NOTE

- Orient them: 30 minutes of module, then 45 minutes of four free loading DINs, then break. The DINs replace the BigLake Qwik Start lab — same content, hands-on instead.
- Pace discipline: end the module by 13:45 sharp, and inside the DIN block start the ~2-minute CSV load early so its wait becomes the lesson.

Flow: Just finished: lunch and the Datastream CDC validation. Next: p071, the EL pattern overview.

Extract and load ingests data into BigQuery without the need for transformation



INSTRUCTOR NOTES — PAGE 74 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Ask: who owns an SSIS package that only stages files — no transform? That's EL, and BigQuery turns it into a first-class, zero-cost pattern.

Say · cover

- Contrast the three shapes: ETL transforms in flight (your SSIS data flow), ELT transforms after landing (your Dataform target), EL just ingests — transform is a later, separate decision.
- EL ingests without transformation: no extra copy, and many sources feed one destination.
- Three doors into the same load job: bq load for scripts, BigQuery Data Transfer Service for scheduled ingest, external tables when data should stay put.
- External tables are the 'no copy' corner — query files in place; that is the lakehouse option and Day 2's subject.

Iowa tie

EL is your Bronze pattern for files: vendor drops and legacy extracts land in BigQuery as-is; Dataform does Silver-to-Gold later.

Design patterns

- Choose EL when the transform belongs downstream in Dataform
- One raw-landing dataset per source

Anti-patterns

- Transforming during the load just because SSIS did
- Treating EL as 'no design needed' — schema and format decisions still apply

History & what's new

EL-style bulk loading has been in BigQuery since its 2010 beta, and batch loads have never billed. The newest corner of this slide is the lakehouse one: BigLake (2022) became Google Cloud Lakehouse in April 2026.

Flow

Block objectives set on p069-70; next: which file formats you can load and export.

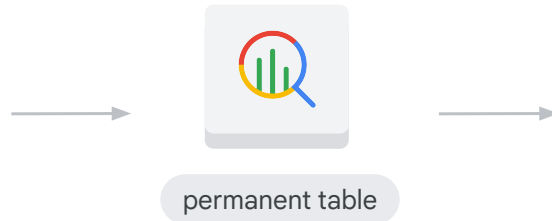
Links

<https://cloud.google.com/bigquery/docs/loading-data>
<https://cloud.google.com/bigquery/docs/dts-introduction>
<https://cloud.google.com/bigquery/docs/external-data-sources>

BigQuery supports importing from and exporting to multiple formats

Loading data

- Avro
- Parquet
- ORC
- CSV
- JSON
- Firestore export



Query results

- CSV
- JSON
- Google Sheets

Table export

- CSV
- JSON
- Avro
- Parquet

Google Cloud

INSTRUCTOR NOTES — PAGE 75 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

SSIS flat-file connection managers trained you on CSV; in BigQuery the format list is short, but the choice is a performance decision, not a convenience.

Say · cover

- Load formats: Avro, Parquet, ORC, CSV, newline-delimited JSON, plus Firestore exports — the columnar ones carry real types inside the file.
- CSV and JSON are text: no types, so they need a declared schema or autodetect; Parquet, ORC and Avro are binary and self-describing.
- Query results export to CSV, JSON or Google Sheets; full table export adds Avro and Parquet — the migration-friendly interchange format.
- Rule of thumb: Parquet whenever you control the format; CSV only when a vendor forces it.

Iowa tie

This is your Bronze-layer format standard: what the legacy extracts land as, and what you ask vendors to deliver at next contract.

Design patterns

- Standardize the landing format per source in the design doc
- Prefer columnar, self-describing formats for bulk

Anti-patterns

- Letting every vendor pick a different format
- Handing around Excel or fixed-width extracts and expecting clean loads

History & what's new

Parquet/ORC support for Cloud Storage data went GA in early 2020 after an October 2019 beta. Still moving: October 2024 added GA support for loading and exporting Parquet files with GeoParquet metadata.

Flow

EL pattern framed; next: the two most common doors — the console form and the LOAD DATA statement.

Links

<https://cloud.google.com/bigquery/docs/loading-data>
<https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-parquet>
<https://cloud.google.com/bigquery/docs/exporting-data>

Load data into BigQuery by using the UI or the **LOAD DATA** statement

Create table from

Upload ▼

Select file **BROWSE**

File format

CSV ▼

Schema

☒ Auto detect

```
# create a new table by loading data from
# CSV files using schema auto-detect
LOAD DATA INTO dataset_name.table_name
FROM FILES(
  format='CSV',
  uris = ['gs://mybucket/*.csv']
)
```

- ✓ Use **LOAD DATA INTO** to append to an existing table.
- ✓ Use **LOAD DATA OVERWRITE** to overwrite the existing table.

Google Cloud

INSTRUCTOR NOTES — PAGE 76 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Two ways past Import/Export-wizard muscle memory: click the console form to explore, or write **LOAD DATA** — **BULK INSERT** as a SQL statement.

Say · cover

- Console path: Create table, pick the source (upload or Cloud Storage), format, Auto detect or typed schema, Create — it submits a load job, not a wizard copy.
- **LOAD DATA** runs the same job expressed as SQL: a format plus a list of `gs://` URIs, wildcards allowed.
- **LOAD DATA INTO** appends; **LOAD DATA OVERWRITE** truncates first — decide deliberately which one each pipeline means, because re-running appends.
- Because it is SQL, it lives in scripts, procedures and scheduled queries — where your SQL Agent instincts transfer directly.

lowa tie

One scheduled **LOAD DATA** with a date-built URI replaces a nightly extract job — DIN S27 proves it in 30 minutes with Bytes billed = 0.

Design patterns

- Console for one-off exploration; SQL for anything repeatable
- Name append vs overwrite explicitly per pipeline

Anti-patterns

- Click-ops loads for recurring feeds
- Assuming a re-run replaces — it appends

History & what's new

LOAD DATA is young: preview August 2022, GA for cross-cloud S3/Azure loads November 2022, full GA for Avro/CSV/JSON/ORC/Parquet in July 2023 — a modern SQL front on a much older free load engine.

Flow

Formats settled; next: the same loads from the command line with `bq`.

Links

<https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-csv>
<https://cloud.google.com/bigquery/docs/reference/standard-sql/load-statements>
<https://cloud.google.com/bigquery/docs/batch-loading-data>

Interact with BigQuery by using the `bq` command



Google Cloud

INSTRUCTOR NOTES — PAGE 77 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Everything you clicked last slide is one command — `bq load` is the scriptable front door your CI/CD pipelines will call.

Say · cover

- `bq mk` creates datasets — note `--location=US`; pick the location once and standardize it across the project.
- Read the example: `--source_format=CSV`, `--skip_leading_rows=2`, comma-separated wildcard URIs, and `./table_schema.json` — the pinned schema as a file.
- That schema file is the point: it lives in source control next to the pipeline, versioned like your SSIS packages.
- `bq` ships inside the `gcloud` CLI; Cloud Shell has it preinstalled and authenticated, and your build agents can run the same binary.

Iowa tie

This is the extract-killer's scripting surface: `bq` commands inside your existing CI/CD deploy pipelines, schema JSON reviewed in pull requests.

Design patterns

- Schema JSON as a versioned artifact beside the pipeline
- Parameterize URIs by run date in scripts

Anti-patterns

- Inline schemas that drift from source control
- Loads run from a personal workstation instead of the pipeline identity

History & what's new

`bq` has shipped with the Cloud SDK since BigQuery's early days. Recent CLI change next door: `gsutil` is deprecated — after March 2027 it leaves the `gcloud` CLI bundle; `gcloud storage` (GA 2022, far faster transfers) is the replacement. Pair `bq` for loads with `gcloud storage` for bucket work.

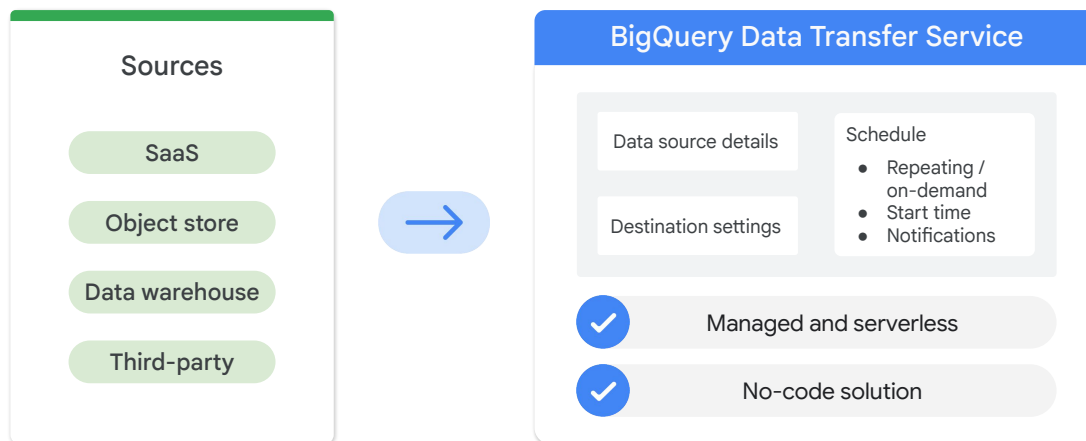
Flow

UI and SQL doors covered; next: the CLI door's scheduling cousin, the Data Transfer Service.

Links

<https://cloud.google.com/bigquery/docs/bq-command-line-tool>
<https://cloud.google.com/bigquery/docs/batch-loading-data>
<https://docs.cloud.google.com/storage/docs/gsutil>

Use the BigQuery Data Transfer Service to load data from other structured data sources



Google Cloud

INSTRUCTOR NOTES — PAGE 78 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Your SQL Agent jobs that import on a schedule — DTS is that, managed and serverless, with no VM to patch.

Say · cover

- What it is: scheduled, managed ingest into BigQuery — configure source, schedule, destination; no code, no infrastructure.
- Source families: Google SaaS apps, object stores (Cloud Storage, Amazon S3, Azure Blob), data warehouses (Teradata, Redshift), and third-party connectors.
- Schedules are repeating or on-demand, with start times and Pub/Sub run notifications for your M&O monitoring stack.
- For live SQL Server sources the answer is Datastream (this morning), not DTS — DTS is for files and SaaS; do not force it onto databases.

Iowa tie

M&O's future inbox: DTS run notifications replace SQL Agent failure emails for every file-based feed you migrate.

Design patterns

- Wire run notifications to Pub/Sub for M&O alerting
- Use backfill runs to catch up missed windows

Anti-patterns

- Using DTS where Datastream belongs (live databases)
- No notification channel — silently failed transfers

History & what's new

DTS was introduced at Cloud Next in March 2017 and went GA that November, initially for Google SaaS apps. It keeps growing: August 2024 added GA incremental transfers for Teradata warehouse migrations — Google is investing in exactly your migration shape.

Flow

Three load doors plus scheduling done; pivot next: what if the data should not move at all — external tables.

Links

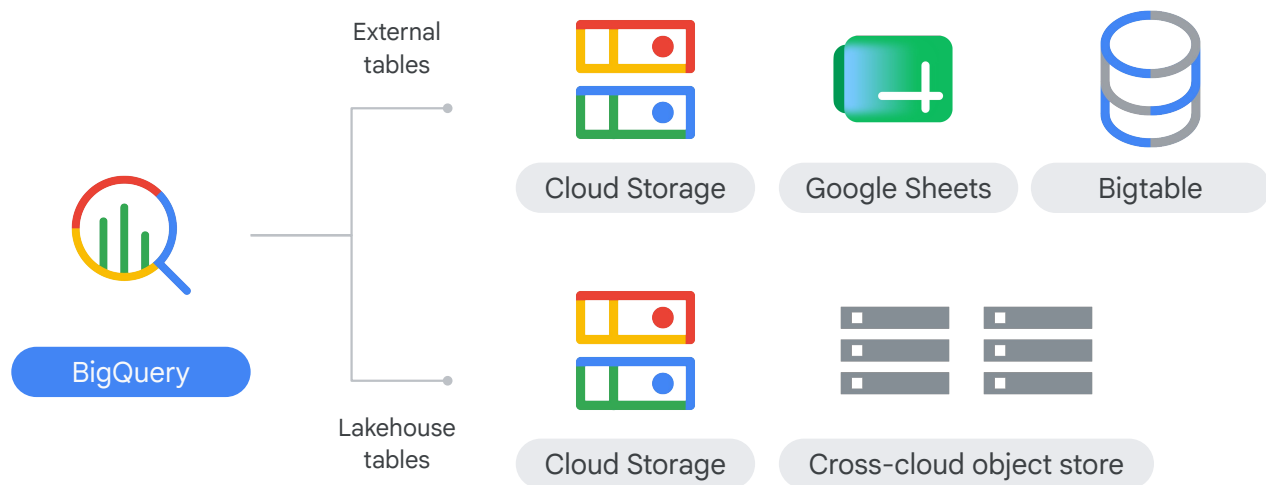
<https://cloud.google.com/bigquery/docs/dts-introduction>

<https://cloud.google.com/bigquery/docs/cloud-storage-transfer>

<https://cloud.google.com/blog/products/gcp/announcing-bigquery-data-transfer-service-general-availability>

Your data isn't stored in BigQuery?

You can still query it!



Google Cloud

INSTRUCTOR NOTES — PAGE 79 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

SQL Server's linked servers let you query without importing; BigQuery's version is external tables — and it scales to buckets, not one remote server.

Say · cover

- The message: data in Cloud Storage, Google Sheets, Bigtable — even cross-cloud object stores — is queryable from BigQuery with no load step.
- Two flavors behind this slide: classic external tables (direct link; the user needs access to the source) and Lakehouse tables (via a connection; a service account reads the files).
- Nothing moves and nothing is copied — the schema lives in BigQuery, the bytes stay where they are.
- Position it now, use it on Day 2: this is the foundation of the lakehouse design and your Bronze-on-buckets option.

Iowa tie

This is how Bronze can sit on Cloud Storage in open formats while BigQuery stays the single query front door — the Day-2 lakehouse conversation starts here.

Design patterns

- External tables for exploration before committing to a load
- Keep rarely-scanned archives external

Anti-patterns

- External tables under hot, daily-queried data
- Assuming external means free — queries still bill bytes scanned

History & what's new

Federated querying is old BigQuery — schema auto-detection for federated sources landed March 2016. The modern chapter: BigLake (2022) added governance and caching over external files and became Google Cloud Lakehouse in April 2026.

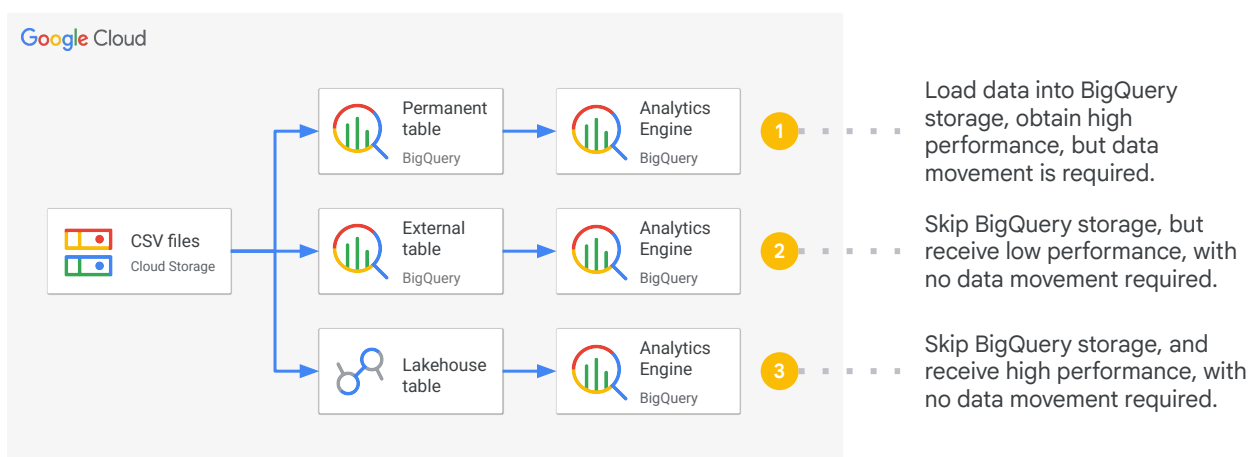
Flow

Scheduled ingest (DTS) closed; next: the choice broken into three concrete architectures.

Links

<https://cloud.google.com/bigquery/docs/external-data-sources>
<https://cloud.google.com/bigquery/docs/external-data-drive>
<https://cloud.google.com/bigquery/docs/biglake-intro>

Three ways of analyzing structured data in BigQuery



Google Cloud

INSTRUCTOR NOTES — PAGE 80 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

One engine, three table types — the slide to photograph, because every migration design decision you make is a row in this comparison.

Say · cover

- Walk left to right: permanent table — data loaded into BigQuery storage, fastest, but movement required; external table — no movement, read at query time, slowest; Lakehouse table — no movement, fast, via metadata caching.
- The engine is BigQuery in all three: same SQL, same console, same security surface.
- The choice is per table, not per platform: one dataset can mix all three and join across them in a single query.
- The trade triangle is movement vs performance vs freshness — external data is always current precisely because it never moved.

Iowa tie

Your medallion mapping: Gold data marts as permanent tables for Power BI speed, Bronze on buckets as Lakehouse, ad-hoc vendor peeks as external.

Design patterns

- Decide table type per dataset with a written rationale
- Join across table types freely — the engine handles it

Anti-patterns

- Loading everything because that is what you did on-prem
- External tables under a latency-sensitive Power BI report

History & what's new

The third option is the new one: Google announced BigLake at the April 2022 Data Cloud Summit and made it GA that July; in April 2026 it was renamed Google Cloud Lakehouse — hence this deck's vocabulary.

Flow

External-tables concept placed on p076; next: the everyday example — Google Sheets as a live source.

Links

<https://cloud.google.com/bigquery/docs/external-data-sources>
<https://cloud.google.com/bigquery/docs/biglake-intro>
<https://cloud.google.com/blog/products/data-analytics/introducing-biglake>

External tables let you query Google Sheets data directly in BigQuery

SCHEMA

DETAILS

LINEAGE

...

Table information

Table ID	your-project-id.sheets.sheets_table
...	

External data configuration

Source URI(s)	https://docs.google.com/spreadsheets/...
Source format	GOOGLE_SHEETS



```
# Query the Google Sheets table
SELECT ...
FROM
  `your-project-id.sheets.sheets_table`;
```

- No query cost estimation, table preview, or query caching available.
- Performance is slower than querying a permanent table in BigQuery.

Google Cloud

INSTRUCTOR NOTES — PAGE 81 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Every agency has that one spreadsheet that is secretly a database — BigQuery can query it directly, no export macro.

Say · cover

- Show the console entry: an external table definition whose Source URI is the docs.google.com URL and format GOOGLE_SHEETS — then plain SELECT against it.
- The sheet stays the system of record; every query reads it live — great for lookups maintained by business users.
- Read the slide's fine print: no cost estimation, no table preview, no query caching, and slower than a permanent table.
- Know the reverse direction too: Connected Sheets lets business users analyze BigQuery data from the Sheets UI.

Iowa tie

Candidate use: agency-maintained reference sheets feeding Silver transforms — but promote anything load-bearing to a real table in Dataform.

Design patterns

- External sheet tables for small, human-maintained lookups
- Snapshot to a permanent table before heavy reporting

Anti-patterns

- Nightly pipelines built on a sheet humans edit
- Joining a million-row fact to a live sheet every query

History & what's new

Sheets-as-source dates to BigQuery's 2016 federated era; the reverse direction — Connected Sheets, analyzing BigQuery data inside Sheets — went GA in June 2020 for enterprise tiers.

Flow

Three architectures compared; next: the lakehouse idea proper — the BigQuery engine over your data lake.

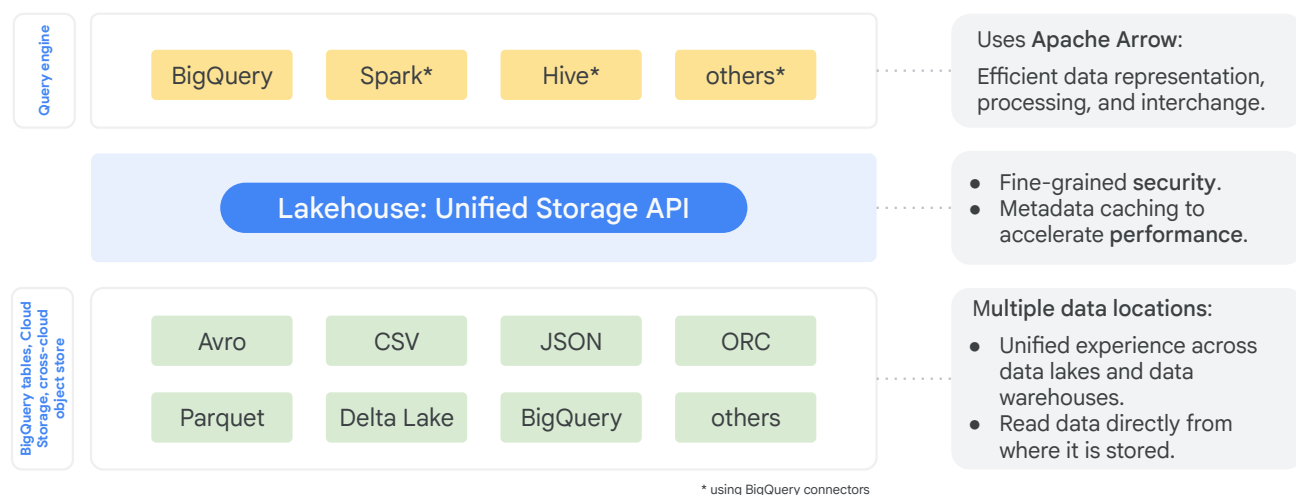
Links

<https://cloud.google.com/bigquery/docs/external-data-drive>

<https://cloud.google.com/bigquery/docs/connected-sheets>

<https://workspace.google.com/blog/product-announcements/connected-sheets-is-generally-available>

Lakehouse: BigQuery on your data lake... and much more!



Google Cloud

INSTRUCTOR NOTES — PAGE 82 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Lakehouse means warehouse-grade governance on lake-format files — the architecture your Bronze-Silver-Gold design is already drawn as.

Say · cover

- Read the diagram bottom-up: files in open formats (Avro, CSV, JSON, ORC, Parquet, Delta Lake) in Cloud Storage or cross-cloud stores; BigQuery queries them in place.
- The unified Storage API is the plumbing — high-throughput reads using Apache Arrow, an efficient in-memory columnar representation.
- Multiple engines, one copy: Spark and Hive reach the same data through BigQuery connectors — no extract between systems.
- Fine-grained security plus metadata caching are what make this 'warehouse manners on lake files'; the next slides detail both.

Iowa tie

Day-2 preview of your target state: Bronze files on GCS in open formats, one engine, governance intact.

Design patterns

- Open formats in the lake so any engine can read them
- One copy of the data, many engines

Anti-patterns

- Proprietary formats in the Bronze layer
- Copying lake data into every engine that needs it

History & what's new

The Arrow-based BigQuery Storage API went GA in March 2020; BigLake productized the pattern in 2022, and April 2026 brought the rename to Google Cloud Lakehouse plus a cross-cloud Lakehouse preview for AWS and Azure in the same week.

Flow

Sheets case done; next: what a Lakehouse table actually looks like in the console.

Links

<https://cloud.google.com/bigquery/docs/biglake-intro>
<https://cloud.google.com/bigquery/docs/reference/storage>
<https://docs.cloud.google.com/lakehouse/docs/release-notes>

A Lakehouse table behaves the same way as a permanent table

SCHEMA

DETAILS

LINEAGE

...

Table information

Table ID	your-project-id.biglake.csv_table
...	

External data configuration

Source URI(s)	gs://your-bucket/your_file.csv
...	



```
# Query the Lakehouse table
SELECT ...
FROM
  `your-project-id.biglake.csv_table`;
```

- ✓ Use Lakehouse if loading data into BigQuery isn't an option for your use case.
- ✓ Queries are performant due to metadata caching.
- ✗ No query cost estimation or table preview.

Google Cloud

INSTRUCTOR NOTES — PAGE 83 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

The console panel on this slide is the whole trick: it looks like a table and quacks like a table — but the bytes never left your bucket.

Say · cover

- Show the Details panel: the Table ID sits in your dataset, while External data configuration points at gs:// URIs — metadata in BigQuery, data in Cloud Storage.
- Querying is identical SQL: SELECT from project.dataset.table — consumers cannot tell, and should not have to.
- Two visible differences: no query cost estimation and no table preview, because BigQuery has not scanned the files.
- Performance stays close to native thanks to metadata caching — the next slide shows exactly what gets cached.
- Guidance on the slide: use Lakehouse when loading into BigQuery storage is not an option — format, residency, or engine-sharing reasons.

Iowa tie

This is Bronze-on-GCS with warehouse manners: Power BI and Dataform see a table; the files stay open-format for other engines.

Design patterns

- Lakehouse for shared-format zones; permanent tables for curated marts
- Keep URIs hive-partitioned for pruning

Anti-patterns

- Choosing Lakehouse by default where managed storage is simpler and faster
- Forgetting no-preview/no-estimate when demoing cost to the business

History & what's new

BigLake tables went GA in July 2022 ('Unify data lakes and warehouses with BigLake'); in April 2026 the family was renamed Google Cloud Lakehouse — same product, new banner.

Flow

Lakehouse concept placed; next: why it is fast — the metadata cache.

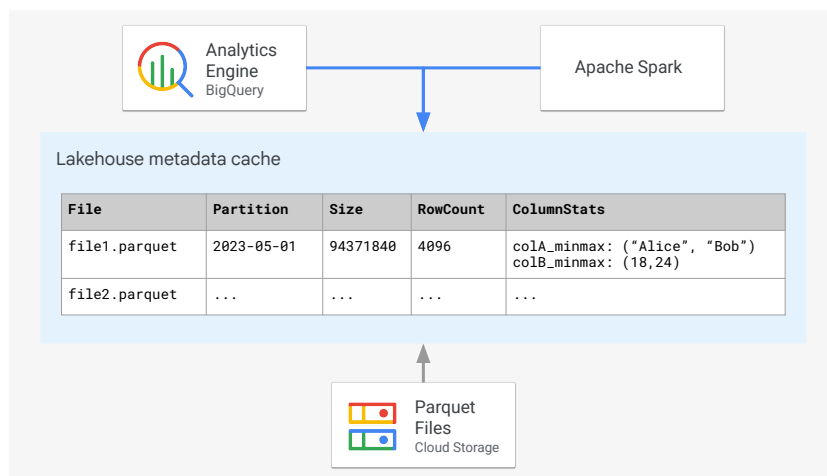
Links

<https://cloud.google.com/bigquery/docs/create-cloud-storage-table-biglake>

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/blog/products/data-analytics/unify-data-lakes-and-warehouses-with-biglake-now-generally-available>

Lakehouse uses metadata caching to accelerate performance



Advantages of BigQuery

- Skip object list
- Faster file and partition pruning
- Dynamic predicate pruning

Advantages of Spark

- Can read metadata statistics using the spark-bigquery connector.

Cache properties

- Staleness between 30 min and 7 days
- Automatic or manual refresh

Google Cloud

INSTRUCTOR NOTES — PAGE 84 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Why can a Lakehouse table feel native? Because BigQuery remembers what it learned about your files — this slide is that memory.

Say · cover

- The cache stores per-file metadata: path, partition values, row counts, sizes, and column min/max statistics — the same statistics Parquet footers carry.
- Three wins: skip the object-list call (no bucket listing per query), faster file and partition pruning, and dynamic predicate pruning via min/max stats.
- Spark benefits too — the spark-bigquery connector can read the same cached statistics.
- Two properties to set deliberately: staleness between 30 minutes and 7 days, and automatic vs manual refresh — pick by how fast files churn.

Iowa tie

For Bronze zones with frequent drops, keep staleness near 30 minutes; curated, slow-changing zones can stretch toward days.

Design patterns

- Match cache staleness to drop frequency
- Manual refresh right after bulk backfills

Anti-patterns

- Seven-day staleness on a zone receiving hourly files
- Assuming caching rescues a bad partition layout

History & what's new

Metadata caching arrived in stages: June 2023 brought it (preview) to BigLake tables over Amazon S3, and August 2024 made Delta Lake BigLake tables GA — widening the formats the cache covers.

Flow

Table anatomy done; next: who can see what — the security difference between external and Lakehouse.

Links

<https://cloud.google.com/bigquery/docs/metadata-caching>

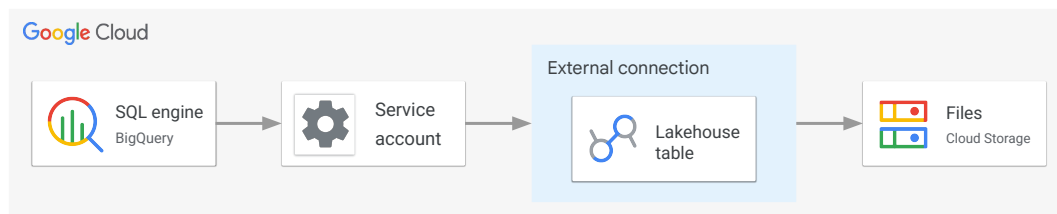
<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/bigquery/docs/create-cloud-storage-table-biglake>

Compare security approaches: Non-native stored data



External tables: the user needs separate permissions to access the table and the data source.



Lakehouse: access is delegated using a service account. It decouples access to the table from the underlying data source.

Google Cloud

INSTRUCTOR NOTES — PAGE 85 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

The governance question your security review will ask: when data lives in a bucket, who needs permission to what?

Say · cover

- External table: the querying user needs permissions on both the table and the underlying files — access follows the user's identity to the source.
- Lakehouse table: a connection holding a service account sits in the middle — users need table access only; the service account reads the files. Access is delegated.
- That decoupling is the headline: grant analysts BigQuery table access without ever granting bucket access.
- It also unlocks warehouse-style controls over files: column-level security, data masking, and row-level security on the Lakehouse table itself.

Iowa tie

This is the answer for the DOM DoIT security review: least-privilege on buckets, managed table access for consumers — a pattern your M&O team can operate.

Design patterns

- Service-account delegation; users never touch buckets
- Row/column policies on the Lakehouse table itself

Anti-patterns

- Handing bucket read access to every analyst
- Leaving security to folder naming conventions in GCS

History & what's new

Delegated access via connections was BigLake's core pitch at its July 2022 GA; the family became Google Cloud Lakehouse in April 2026, with cross-cloud Lakehouse (S3 and Azure Blob behind the same model) in preview.

Flow

Performance explained; next: a one-minute art-of-the-possible demo — object tables and a remote ML model.

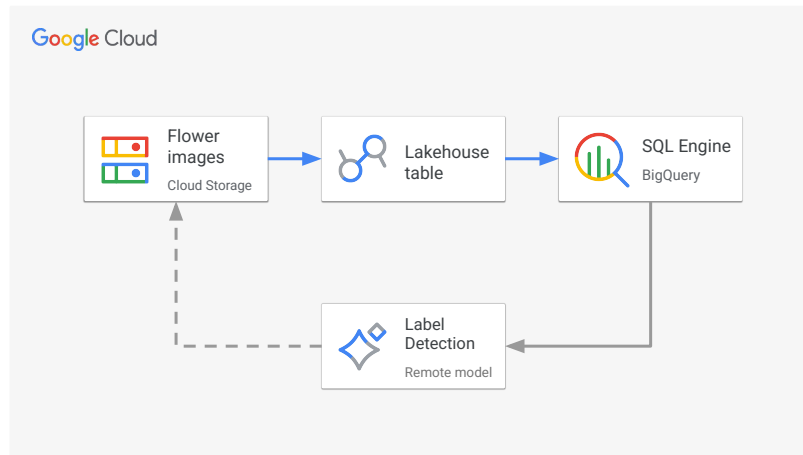
Links

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/bigquery/docs/create-cloud-storage-table-biglake>

<https://cloud.google.com/blog/products/data-analytics/unify-data-lakes-and-warehouses-with-biglake-now-generally-available>

[Demo] Analyze an object table by using a remote model



```
gs://biglake-flowers/flower_photos/
daisy/pic.jpg
```

```
"label_annotations": [{
  "description": "Flower",
  "score": 0.98062
}]
```

Google Cloud

INSTRUCTOR NOTES — PAGE 86 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

One minute of art-of-the-possible before the hands-on: querying JPEGs with SQL.

Say · cover

- Keep this brief — the agenda says lightly: an object table is a Lakehouse-side table over unstructured files, here flower photos in `gs://biglake-flowers`.
- The table holds metadata about each object — URI, content type — not the pixels; a remote model, a connection to a Vertex AI vision model, runs label detection from SQL.
- Result rows look like the JSON on the slide: label 'Flower', score 0.98 — inference without leaving BigQuery.
- Why show it at all: the same one-engine governance story now covers unstructured data — relevant when agency documents and images enter scope.

Iowa tie

Not in your migration scope today — but when scanned forms start landing in buckets, this is the pattern: object table plus remote model.

Design patterns

- Object tables to govern unstructured files
- Remote models keep inference inside SQL

Anti-patterns

- Loading images into tables as BLOBs
- Building custom ML pipelines before checking built-in remote models

History & what's new

Object tables were announced in 2022 and went GA in May 2023 — a 'structured record interface for unstructured data' extending BigQuery governance, and remote-model inference, to images, audio and documents.

Flow

Security contrast done; next: the head-to-head — external vs Lakehouse side by side.

Links

<https://cloud.google.com/bigquery/docs/object-tables>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-remote-model>

<https://cloud.google.com/blog/products/data-analytics/bigquery-object-tables-now-ga>

Compare external and Lakehouse tables

	External tables	Lakehouse tables
Query Engine	BigQuery	BigQuery, others using BigQuery connector
Data types	- Parquet, Avro, JSON, ORC, CSV - Datastore export, Firestore export - Google Sheets	- Parquet, Avro, JSON, ORC, CSV - Apache Iceberg, Delta, Hudi - Hive partitioned - Any unstructured (via object tables)
Locations	- Cloud Storage - Bigtable - Google Drive	- Cloud Storage - Amazon S3 - Azure Blob Storage
Access pattern	Direct link to the source. The user needs to have access to the source	External connection with access delegation. The service account has access to the source.
Security	None	- Column-level security, data masking - Row-level security

Google Cloud

INSTRUCTOR NOTES — PAGE 87 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Decision time — if they remember one table from this module, it is this one; it is your Bronze design checklist in disguise.

Say · cover

- Engine: both query via BigQuery; Lakehouse adds other engines (Spark and friends) through BigQuery connectors on the same files.
- Formats: external covers Parquet/Avro/JSON/ORC/CSV plus Datastore/Firestore exports and Google Sheets; Lakehouse adds Iceberg, Delta, Hudi, hive-partitioned layouts, and any unstructured type via object tables.
- Locations: external reaches Cloud Storage, Bigtable and Drive; Lakehouse adds Amazon S3 and Azure Blob — the multi-cloud story.
- Access and security decide it: external means the user needs source access and gets no row/column security; Lakehouse means service-account delegation plus column-level security, masking and row-level security.

lowa tie

Write the default into your design standards: Lakehouse for governed Bronze zones, external tables for throwaway exploration.

Design patterns

- Lakehouse wherever governance must follow the data
- Re-check this comparison per zone, not per habit

Anti-patterns

- Plain external tables in production for convenience
- Assuming Iceberg or Delta sources work on classic external tables

History & what's new

The cross-cloud row is real: BigQuery Omni (GA 2021) and cross-cloud transfers (GA December 2022) brought S3 and Azure within reach, and April 2026 opened a cross-cloud Lakehouse preview under the new Lakehouse name.

Flow

Module content complete — the next four cards preview the DINs you will run at 13:45; full versions repeat then.

Links

<https://cloud.google.com/bigquery/docs/external-data-sources>
<https://cloud.google.com/bigquery/docs/biglake-intro>
<https://cloud.google.com/bigquery/docs/omni-introduction>

DIN: Loading data — console and CLI

~10 minutes · BigQuery console + Cloud Shell — into your class dataset

1. Console: Create table on your class dataset from `jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv`
2. Details tab: 1,000,000 rows; autodetect typed minutes as INTEGER, event_date as DATE
3. Append Tuesday's file: `bq load --autodetect --source_format=CSV class.usage gs://.../usage_2026-07-21.csv`
4. Refresh: 2,000,000 rows — loads append by default; `--replace` truncates first
5. View all jobs: console and bq load are two front ends for the same free load job

Why it matters — your Bronze-landing pattern in miniature — vendor file → bucket → BigQuery table, the same shape as the extract jobs you are replacing.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 88 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Preview only — at 13:45 everyone does this themselves; for now, watch the shape: vendor file, bucket, table.

Say · cover

- The scenario mirrors your world: a vendor drops a daily usage CSV in a bucket; the console lands Monday's file, bq load appends Tuesday's.
- Two teaching beats: the Details tab goes 1,000,000 to 2,000,000 rows — loads append by default, `--replace` truncates — and View all jobs shows both doors as the same Load job.
- Autodetect typed this clean file well (minutes INTEGER, event_date DATE) — hold that thought; the next DIN breaks it.
- Logistics: students load into their own class dataset from the `jwd-gcp-demos` bucket; everything is free — batch loads bill nothing.

Iowa tie

The extract-job replacement in miniature: the nightly vendor feed becomes two commands and zero infrastructure.

Design patterns

- Console for the first look, CLI for the repeat
- Verify row counts after every load

Anti-patterns

- Forgetting loads append — double-loading a day
- Pasting `gs://` into the console's bucket field; it adds `gs://` itself

History & what's new

The console flow shown here now lives in BigQuery Studio, announced August 2023 as a unified workspace — the modern descendant of the 2016-era web UI that first added schema auto-detection.

Flow

Module slides end; at 13:45 this DIN (\$24) opens the four-DIN loading block — full card at p90. Next: the autodetect pitfall preview.

Links

<https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-csv>

<https://cloud.google.com/bigquery/docs/bq-command-line-tool>

<https://cloud.google.com/blog/products/data-analytics/announcing-bigquery-studio>

DIN: The autodetect pitfall — N/A in a numeric column

~10 minutes · Cloud Shell + BigQuery console

1. Paste `autodetect_demo.sh` into Cloud Shell — two vendor drops with N/A in the numeric minutes column
2. Load Monday (100k rows, N/A at row 90,000) with `--autodetect`: fails loudly — Unable to parse
3. Load Tuesday (1,000 rows, N/A at row 700) the same way: succeeds, but minutes lands as STRING
4. `SUM(minutes)` fails — loud failure vs silent wrong schema, decided by the sample window
5. Fix: pinned schema JSON + `--null_marker='N/A'` — 101,000 rows, 2 NULLs, SUM works

Why it matters — autodetect is the SSIS "Suggest Types" button — fine for a first look, never for a recurring feed. The silent failure is the one that costs you days.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 89 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

The SSIS 'Suggest Types' button, weaponized as a lesson: the same bad value causes two different failures — and only one of them tells you.

Say · cover

- Setup: two vendor files with N/A in the numeric minutes column; Monday's N/A sits at row 90,000 of 100,000, Tuesday's at row 700 of 1,000.
- Monday fails loudly — autodetect sampled only numbers, chose INTEGER, and the parse died at row 90,000; Tuesday succeeds silently — the N/A fell inside the sample, minutes lands as STRING, and `SUM(minutes)` fails later.
- The failure mode depends on the sample window, not the data — that non-determinism is why autodetect never belongs in a pipeline.
- The fix is your CI/CD shape: pinned schema JSON plus `--null_marker='N/A'` gives 101,000 rows, exactly 2 NULLs, and a working SUM.

Iowa tie

Vendor-file reality for DOM DoIT: sentinels like N/A live in every legacy extract; the pinned schema becomes a versioned artifact in your repos.

Design patterns

- Autodetect once on a good file, `bq show --schema`, review, then pin
- Declare the vendor's null marker explicitly

Anti-patterns

- Autodetect on recurring feeds
- Dropping `--skip_leading_rows` once the schema is pinned — the header then breaks the load

History & what's new

Schema auto-detection shipped in March 2016 for CSV/JSON loads and federated sources — a decade-old convenience whose sample-window behavior this DIN makes visceral; the pinned-schema alternative has existed just as long.

Flow

Second DIN preview; §25 runs at 13:45 (full card p91). Next: the format race preview.

Links

<https://cloud.google.com/bigquery/docs/schema-detect>
<https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-csv>
<https://docs.cloud.google.com/bigquery/docs/release-notes-archive>

DIN: CSV vs Parquet race — 60 million rows, two ways

~10 minutes · Cloud Shell — `gs://jwd-gcp-demos/ingest_demo`

1. Same 60M rows staged two ways: one 317 MiB gzipped CSV vs 100 Parquet files (~829 MiB total)
2. Load the gzip: `bq load --source_format=CSV` with an inline schema — watch the counter, about two minutes
3. While it runs: a gzip cannot be split — one worker decompresses and parses the whole file alone
4. Load the Parquet copy: `bq load --source_format=PARQUET` — done in seconds; Parquet carries its own schema
5. `bq show` both tables: 60,000,000 rows each — more than twice the bytes, roughly 20x faster

Why it matters — your Bronze-layer format decision: land Parquet wherever you control the format; for vendor CSVs you don't, prefer many small files over one large gzip.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 90 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Same 60 million rows staged two ways — take bets now: the 317 MiB gzip or the 829 MiB of Parquet?

Say · cover

- The trap is intuitive: the gzip is smaller so it 'should' load faster — it takes about two minutes; the Parquet load takes seconds.
- Why: a gzip cannot be split — one worker decompresses and parses the whole file alone; 100 Parquet files fan out to 100 workers, and binary columnar means no text parsing.
- Schema angle: Parquet carries real types, so no `--schema` flag — and the whole N/A-in-a-numeric-column failure class disappears.
- Both `bq show` outputs read 60,000,000 rows: more than twice the bytes, roughly 20x faster.

Iowa tie

Your Bronze format policy: Parquet wherever you control the format; for vendor CSVs you do not control, many small files beat one big gzip.

Design patterns

- Splittable, self-describing formats for bulk loads
- Ask vendors for many files over single archives

Anti-patterns

- One giant gzipped CSV as the daily drop
- Passing `--schema` with Parquet — noise that can only cause mismatches

History & what's new

Parquet and ORC support for Cloud Storage data went GA in early 2020 after a 2019 beta, and format investment continues: October 2024 added GA loading and export of Parquet files with GeoParquet metadata.

Flow

Third DIN preview; §26 runs at 13:45 (full card p92). Next: the load as pure SQL.

Links

<https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-parquet>
<https://cloud.google.com/bigquery/docs/batch-loading-data>
<https://docs.cloud.google.com/bigquery/docs/release-notes>

DIN: LOAD DATA — the load job goes SQL

~5–8 minutes · BigQuery query editor — load_data_example.sql

1. Run the load-monday query: LOAD DATA INTO class.usage_sql with a column list pinning the schema, FROM FILES CSV options
2. Run the load-tuesday query: no column list — the table exists; LOAD DATA INTO appends, OVERWRITE would replace
3. Run the verify query: each date holds 1,000,000 meetings and SUM(minutes) works
4. Check Job information on either load: Bytes billed = 0 B — SQL syntax, same free load machinery

Why it matters — the load is now a statement that lives in a script, procedure, or scheduled query — one scheduled LOAD DATA with a date-built URI replaces a nightly extract job.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 91 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

The closing move: everything they did with clicks and CLI flags, as one SQL statement a scheduler can run.

Say · cover

- LOAD DATA INTO with a column list pins the schema inside the statement; the FROM FILES options mirror the bq flags — format, skip_leading_rows, uris.
- The second statement omits the column list because the table now exists; INTO appends, OVERWRITE replaces — the same semantics as the CLI.
- Verify: each date holds 1,000,000 meetings and SUM works; then open Job information — Bytes billed = 0 B. SQL syntax, same free load machinery.
- The strategic beat: this statement lives in source control, a procedure, or a scheduled query — one scheduled LOAD DATA with a date-built URI is the nightly-extract replacement.

Iowa tie

This is the shape that slots into your CI/CD: a reviewed SQL artifact plus a schedule — and the Day-1 closing demo schedules a query in one click.

Design patterns

- Schema pinned in the column list, versioned with the pipeline
- Build URIs from the run date in scheduled queries

Anti-patterns

- Loads that exist only in runbooks humans execute
- Expecting OVERWRITE to be the default — INTO appends

History & what's new

LOAD DATA entered preview in August 2022, went GA for cross-cloud S3/Azure loads that November, and reached full GA for Avro/CSV/JSON/ORC/Parquet in July 2023 — one of BigQuery's younger SQL statements, built for exactly this automation case.

Flow

Final DIN preview; §27 runs at 13:45 (full card p93). Next: the module wrap-up.

Links

<https://cloud.google.com/bigquery/docs/reference/standard-sql/load-statements>

<https://cloud.google.com/bigquery/docs/scheduling-queries>

<https://cloud.google.com/bigquery/docs/loading-data>

M3 — What you now have

- ✓ The EL pattern mapped: console, bq load, and LOAD DATA all submit the same free load job into BigQuery
- ✓ Four loads with your own hands— console & CLI, the autodetect N/A trap, the 60M-row format race, LOAD DATA SQL
- ✓ Formats and pinned schemas are Bronze-layer design decisions, not conveniences
- ✓ Scheduled ingest covered two ways: Data Transfer Service, or a scheduled query running LOAD DATA
- ✓ Next: break, then M4 ELT— transform where the data already lives, and your first look at Dataform

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 92 OF 135 · 13:15 · MODULE 3 — EXTRACT & LOAD

Hook

Close the loop before the hands-on: in 30 minutes they met every way a file becomes a BigQuery table — in the next 45 they do all four themselves.

Say · cover

- Recap the spine: console, bq load and LOAD DATA are one free load job with three front doors; formats and pinned schemas are design decisions, not conveniences.
- Scheduled ingest is covered twice: Data Transfer Service for managed no-code transfers, or a scheduled query running LOAD DATA for SQL-native automation.
- External and Lakehouse tables let data stay put — today's bridge to Day 2's lakehouse, where Bronze-on-buckets gets governance.
- Land the mindset: load first, transform later — Dataform owns Silver-to-Gold, and that is Module 4 after the DIN block and break.

Iowa tie

Every mechanism on this recap maps to a named piece of your migration: Bronze landing, kill-the-extracts scheduling, and the lakehouse end-state.

Design patterns

- One free load job, three doors — pick per situation
- Managed scheduled ingest before self-built schedulers

Anti-patterns

- Building extract windows when loads are free and fast
- Transforming before landing

History & what's new

This module's two ends bookend 16 years of one idea: free batch loads (a 2010-era BigQuery design decision) and the Lakehouse (BigLake, renamed Google Cloud Lakehouse in April 2026) — separate storage from compute, land data cheaply.

Flow

Module done — next: the 13:45 DIN loading block (\$24-27), break at 14:30, then M4 ELT and Dataform.

Links

<https://cloud.google.com/bigquery/docs/loading-data>
<https://cloud.google.com/bigquery/docs/dts-introduction>
<https://cloud.google.com/bigquery/docs/biglake-intro>

Console & CLI load (DIN §24)

~10 minutes — within the 13:45–14:30 loading block · BigQuery console + Cloud Shell — into your class dataset

1. Create table from Cloud Storage on your class dataset:
jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv
2. Format CSV, table usage, check Auto detect → Create; the Details tab shows 1,000,000 rows
3. Append Tuesday from Cloud Shell: bq load --autodetect --source_format=CSV class.usage
gs://.../usage_2026-07-21.csv
4. Refresh Details: 2,000,000 rows — loads append; View all jobs shows two identical Load jobs

Why it matters — the console form and bq are two front ends over one job API — what you click today is what you script in the migration.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 93 OF 135 · 13:45 · DIN LOADING BLOCK — FOUR WAYS TO LAND FILES (45 MIN)

Hook

Everyone here has used the Import and Export wizard or an SSIS flat-file task — this is that job, twice: by mouse and by CLI, same machinery.

Say · cover

- Console: three-dot menu on the class dataset -> Create table -> source Google Cloud Storage. Enter bucket/path with NO gs:// — #1 'file not found' cause.
- Format CSV, table usage, check Auto detect, Create. Details: 1,000,000 rows; autodetect picked INTEGER/DATE — next slide shows why not to trust it.
- Cloud Shell appends Tuesday: bq load --autodetect --source_format=CSV class.usage gs://.../usage_2026-07-21.csv. Loads APPEND; --replace truncates.
- Refresh Details: 2,000,000 rows. View all jobs shows two identical Load jobs — console and bq are two front ends over one job API.
- Batch loads are free (shared slot pool); the bucket may live in another project — read access suffices.

Iowa tie

Your Bronze landing in miniature: vendor file -> bucket -> BQ table — the shape of the extract jobs you're killing; what you click today, CI/CD scripts tomorrow.

Design patterns

- Land files in Cloud Storage, then batch-load — never row-by-row inserts for file drops.
- Prototype in the console, script the identical job with bq.

Anti-patterns

- Pasting gs:// URIs into the console file field.
- Forgetting loads append — re-run and you silently double rows.

History & what's new

Load jobs run in a free shared slot pool, separate from query slots (quotas docs, 2026); the daily load-job quota doubled to 100,000 per project in 2019 (release-notes archive).

Flow

First of four DINs after M3's extract-and-load slides; next: why the Auto detect box you just ticked bites.

Links

<https://docs.cloud.google.com/bigquery/docs/loading-data-cloud-storage-csv>
<https://docs.cloud.google.com/bigquery/docs/reference/bq-cli-reference>
<https://docs.cloud.google.com/bigquery/quotas>

Autodetect pitfall (DIN §25)

~10 minutes — within the 13:45–14:30 loading block · Cloud Shell + BigQuery editor — into your class dataset

1. Paste `autodetect_demo.sh` into Cloud Shell — writes Monday, Tuesday and a pinned-schema JSON
2. Load Monday with autodetect → loud failure: its N/A at row 90,000 sat outside the sample
3. Load Tuesday → silent failure: N/A inside the sample → minutes typed STRING; SUM fails
4. Fix: reload both days with `--schema`, `--skip_leading_rows=1`, `--null_marker='N/A'` (--replace first)
5. Verify: 101,000 rows, exactly 2 NULL minutes, SUM works

Why it matters — same bad value — loud once, silent once; the silent one costs days. Pin schemas in source control; autodetect is a first look, never a pipeline.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 94 OF 135 · 13:45 · DIN LOADING BLOCK — FOUR WAYS TO LAND FILES (45 MIN)

Hook

SSIS's 'Suggest Types' button burned everyone once — autodetect is the same trap, failing twice: loudly once, silently once. The silent one costs days.

Say · cover

- Paste `autodetect_demo.sh` into Cloud Shell: writes Monday (100,000 rows, N/A at row 90,000), Tuesday (1,000, N/A at row 700), and a pinned-schema JSON.
- Autodetect samples up to 500 rows of one file. Monday: N/A outside the sample -> INTEGER chosen -> load FAILS loudly. The failure is the lesson.
- Tuesday: N/A inside the sample -> only STRING fits -> load SUCCEEDS, minutes typed STRING, SUM fails. Same bad value; outcome set by sample position.
- Fix: --replace, then load both days with `--schema`, `--skip_leading_rows=1`, `--null_marker='N/A'`.
- Verify: 101,000 rows, exactly 2 NULL minutes, SUM works. Cheap path: autodetect once on a good file, bq show --schema, edit, pin.

Iowa tie

Your migration ingests vendor files with exactly this dirt; pinned schema JSON plus null-marker becomes a versioned artifact in your SQL Server CI/CD — reviewed like code.

Design patterns

- Pin schemas in source control for anything recurring; autodetect is a first look only.
- Declare vendor sentinels with `--null_marker` instead of hand-cleaning files.

Anti-patterns

- Autodetect inside a scheduled pipeline — the schema drifts with the data.
- Dropping `--skip_leading_rows` once pinned; autodetect was skipping the header.

History & what's new

Autodetect reads up to 500 rows of a random file (schema-detect docs); in March 2022 Google let you pin explicit schemas on external Parquet/ORC/Avro tables, previously always auto-detected (release-notes archive).

Flow

Second DIN — §24's autodetect gamble paid off; next, the 60M-row CSV-vs-Parquet race while this sinks in.

Links

<https://docs.cloud.google.com/bigquery/docs/schema-detect>
<https://docs.cloud.google.com/bigquery/docs/schemas>
<https://docs.cloud.google.com/bigquery/docs/loading-data-cloud-storage-csv>

CSV vs Parquet load race (DIN §26)

~15 minutes — within the 13:45–14:30 loading block · Cloud Shell — into your class dataset

1. 60M rows staged twice: one gzipped CSV (~317 MiB) vs 100 Parquet files (~829 MiB) — place your bet
2. Load the CSV with an explicit schema — about two minutes: gzip can't split, one worker alone
3. Load the Parquet with no schema (the files carry their types) — seconds
4. bq show both tables: 60,000,000 rows each; Parquet got INTEGER/DATE from the files

Why it matters — twice the bytes, ~20x faster — columnar + splittable beats one big gzip. Wherever you control the format, land Parquet.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 95 OF 135 · 13:45 · DIN LOADING BLOCK — FOUR WAYS TO LAND FILES (45 MIN)

Hook

Same 60 million rows staged twice — one 317 MiB gzipped CSV versus 100 Parquet files totaling 829 MiB. Bets: which loads faster?

Say · cover

- Load the CSV with an explicit schema, watch the elapsed counter — about two minutes. Not hung; the wait IS the demo — narrate through it.
- Why: gzip can't be split — one worker decompresses and parses the whole file alone; CSV is text, every byte parsed.
- Load the Parquet with NO --schema — files are self-describing. Seconds: 100 files -> parallel workers, binary columnar, no parsing.
- bq show both: 60,000,000 rows each; Parquet brought real INTEGER/DATE types — §25's 'N/A in a numeric column' failure class can't be represented.
- Twice the bytes, ~20x faster — columnar + splittable beats one big gzip.

Iowa tie

Your Bronze format decision: where you control it, land Parquet; for vendor CSVs, push for many smaller uncompressed files. Same format your Day-2 Iceberg story is built on.

Design patterns

- Parquet (or Avro/ORC) for bulk landing — splittable, columnar, self-typing.
- When CSV is forced on you, negotiate many small uncompressed files.

Anti-patterns

- One monolithic gzipped CSV for bulk loads — a single-worker bottleneck.
- Passing --schema with Parquet — noise that can fight the files' own types.

History & what's new

BigQuery has loaded Parquet since 2018; federated Parquet/ORC querying went beta in October 2019 (Google Cloud blog); by 2025 Google's own Iceberg tables store data as Parquet (Developer forums, 2025).

Flow

Third DIN, the long one (~15 min); §27 closes the block by expressing the whole load as SQL.

Links

<https://docs.cloud.google.com/bigquery/docs/loading-data-cloud-storage-parquet>

<https://docs.cloud.google.com/bigquery/docs/loading-data-cloud-storage-csv>

<https://cloud.google.com/blog/products/data-analytics/keep-parquet-and-orc-from-the-data-graveyard-with-new-bigquery-features>

LOAD DATA SQL (DIN §27)

~10 minutes — within the 13:45–14:30 loading block · BigQuery query editor — into your class dataset

1. Run the Monday statement in `load_data_example.sql` — the column list pins the schema
2. Run the Tuesday statement — `LOAD DATA INTO` appends (`LOAD DATA OVERWRITE` would replace)
3. Run the verify query: 1,000,000 meetings per date — Job information shows Bytes billed: 0 B

Why it matters — a load expressed as SQL lives in source control, procedures and schedules — the nightly-extract replacement in a single statement.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 96 OF 135 · 13:45 · DIN LOADING BLOCK — FOUR WAYS TO LAND FILES (45 MIN)

Hook

Everything so far was clicks and CLI flags — now the same two loads as pure SQL, the form schedulers, procedures, and repos speak.

Say · cover

- Run Monday's statement from `load_data_example.sql`: the column list pins the schema — the SQL version of `--schema`; `FROM FILES` options mirror the bq flags.
- Run Tuesday: no column list now the table exists. `LOAD DATA INTO` appends; `OVERWRITE` replaces — same semantics as bq.
- Run in order, one at a time: Monday creates the table, Tuesday depends on it.
- Verify: 1,000,000 meetings per date; Job information shows Bytes billed: 0 B — SQL driving the same free load machinery. Only the verify query bills.
- Template it: one scheduled `LOAD DATA` with a `FORMAT_DATE`-built URI is a nightly extract in one statement.

Iowa tie

The SQL Agent import-job replacement: a scheduled `LOAD DATA` lands Bronze nightly; pure SQL lives in your CI/CD repo — no CLI dependency for M&O.

Design patterns

- Express recurring loads as SQL in source control plus a scheduler.
- Pin the schema in the column list on the table-creating load.

Anti-patterns

- Running both statements at once or out of order — Tuesday needs Monday's table.
- Assuming SQL syntax means query billing — check Bytes billed first.

History & what's new

`LOAD DATA` hit GoogleSQL as a preview in August 2022; by November 2022 it was GA and extended to Amazon S3 and Azure Blob Storage (release-notes archive).

Flow

Last DIN of the block — break at 14:30, then M4 turns Bronze tables into Silver->Gold with Dataform; the 16:20 close schedules this exact shape.

Links

<https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/load-statements>

<https://docs.cloud.google.com/bigquery/docs/scheduling-queries>

<https://docs.cloud.google.com/bigquery/docs/procedures>

Afternoon Break

15 minutes · 14:30–14:45

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 97 OF 135 · 14:30 · AFTERNOON BREAK (15 MIN)

FACILITATOR NOTE

- 15 minutes, back at 14:45 sharp — the loading block ends Day 1's 'get data in' arc.
- After the break, M4 ELT & Dataform starts the Silver->Gold half of their architecture, with Jeff's dataform-demo walkthrough at 15:10.

Flow: Break card between the 13:45 DIN loading block (§§24–27) and 14:45 Module 4 — ELT & Dataform.

M4 — What this block gets you

- ✓ ELT reframed— land raw in BigQuery, transform with SQL inside the warehouse, not in an SSIS data flow
- ✓ Scripting, UDFs, stored procedures— your T-SQL instincts transfer; watch dialect, not concepts
- ✓ Scheduled queries— the SQL Agent instinct in one click; the closing demo makes it concrete
- ✓ Dataform— your architecture doc's Silver→Gold tool: sqlx models, the ref() graph, assertions, releases
- ✓ Hands-on: watch a real repo end to end, then build your own SQL workflow in the anchor lab

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 98 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

You spent the morning landing data; the next 80 minutes are the block your architecture doc is built around — transforming it.

Say · cover

- ELT reframe: in SSIS the transform was a data flow moving rows through a server; here raw data lands first and SQL transforms it inside BigQuery.
- Scripting, UDFs, stored procedures: T-SQL instincts transfer — DECLARE, IF, WHILE, CALL all exist in GoogleSQL; watch dialect, not concepts.
- Scheduled queries: the SQL Agent instinct in one click; the 16:20 closing demo makes it concrete.
- Dataform: your Silver->Gold tool — sqlx models, the ref() dependency graph, assertions, releases.
- Arc: fast through the procedural slides, slow for Dataform; Jeff walks a real repo at 15:10, then the anchor lab.

Iowa tie

This module is their named Silver->Gold toolchain — Dataform — plus the interim automation (scheduled queries) that bridges until marts exist.

Design patterns

- Land raw first, transform where the data lives
- Pipeline as code in git, not packages on a shared server

Anti-patterns

- Porting SSIS data-flow logic row-by-row into UDFs instead of set-based SQL

History & what's new

Google acquired Dataform in December 2020; it went GA inside BigQuery in May 2023, shipping release and workflow configurations.

Flow

Back from the 14:30 break after the DIN loading block; next slide sequences the afternoon, then the ELT pattern.

Links

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/blog/products/data-analytics/introducing-dataform-in-ga>

<https://cloud.google.com/bigquery/docs/scheduling-queries>

This afternoon, block by block

14:45–15:10	M4 — ELT in BigQuery and the Dataform intro; fast through scripting, UDFs, sprocs	
15:10–15:25	Jeff's dataform-demo repo, live — a real Silver→Gold pipeline in git	DEMO
15:25–16:05	Lab: Create and Execute a SQL Workflow in Dataform — the anchor lab, full slot	LAB
16:05–16:20	M5 + M6 compressed — ETL options and the automation ladder	
16:20–16:30	Close: schedule the derived-table query — one click, your nightly SQL Agent job	DEMO

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 99 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Ninety minutes from now you will have built a Silver->Gold pipeline with your own hands — here is how the afternoon is sequenced to get there.

Say · cover

- 14:45–15:10: ELT and Dataform intro — burn through scripting/UDF/sproc slides; this room reads T-SQL fluently.
- 15:10–15:25: Jeff's dataform-demo repo live — a real raw -> staging -> marts pipeline in git; interruptions welcome.
- 15:25–16:05: the anchor lab, full 40 minutes — protect this slot; it is the one solid lab of the course.
- 16:05–16:20: M5+M6 compressed — ETL escape hatches and the automation ladder; the M&O team leans in here.
- 16:20 close: schedule the derived-table query — one click, their nightly SQL Agent job minus the server.

Iowa tie

Every block maps to their migration: Dataform for Silver->Gold, scheduled queries for interim automation, M6 for the M&O operating model.

Design patterns

- Protect the anchor lab's full slot — cut module slides before lab minutes

Anti-patterns

- Letting UDF/sproc Q&A eat the Dataform and lab time

History & what's new

Dataform went GA in BigQuery in May 2023 — a young service, so console placement and docs still shift; check links before class.

Flow

Block overview just landed; next is the ELT-vs-ETL pattern slide that starts the teaching arc.

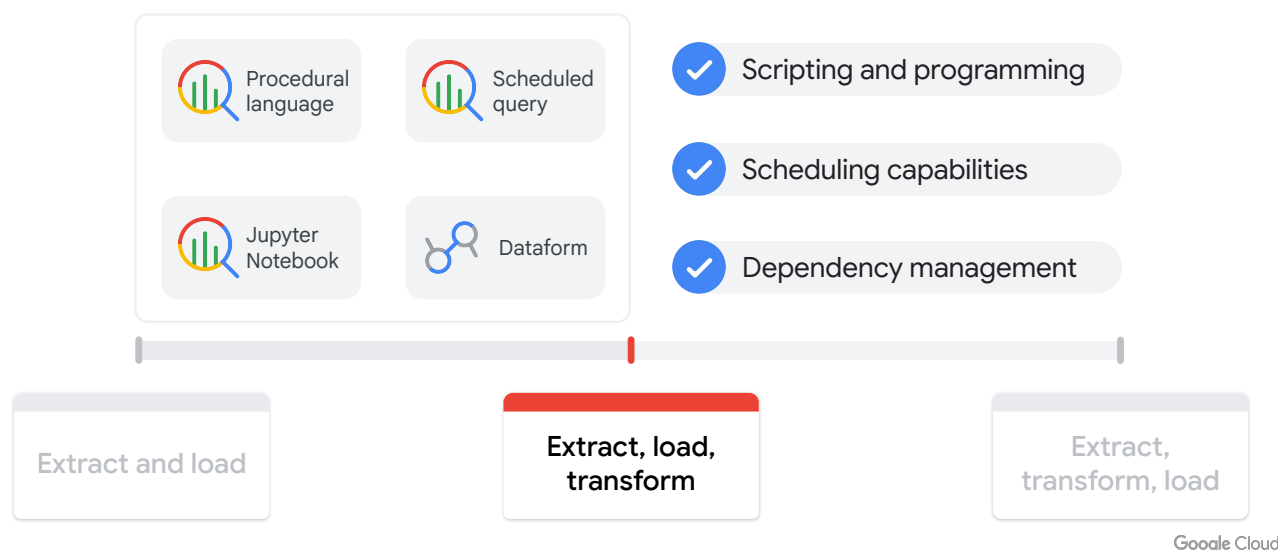
Links

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/dataform/docs/release-configurations>

<https://cloud.google.com/dataform/docs/workspaces>

Once data is loaded into BigQuery, there are multiple ways to transform it



INSTRUCTOR NOTES — PAGE 100 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Same letters, different order: ETL transforms on the way in — SSIS on a server; ELT lands data first and transforms where it lives.

Say · cover

- Once data is loaded, BigQuery itself is the transformation engine — no separate transform tier to provision or patch.
- Four options on this slide: procedural scripting for control flow, scheduled queries for single-statement refreshes, Dataform for managed SQL workflows, notebooks for exploration.
- Dependency management is the differentiator — only Dataform tracks and orders dependencies for you; everything else you sequence by hand.
- Console placement: scripting and scheduled queries live in BigQuery Studio; Dataform has its own section under BigQuery; notebooks under BigQuery Studio too.
- Skip defining ETL/ELT theory — this audience wrote the book; spend the minutes on where each option lives.

Iowa tie

Their flow: Datastream and batch loads land Bronze; Dataform runs Silver->Gold; scheduled queries cover single-table interim refreshes.

Design patterns

- Dataform for multi-table dependency chains; scheduled query for one-statement rebuilds

Anti-patterns

- Hand-sequenced scripts with no dependency tracking or retry semantics
- A notebook as the production pipeline

History & what's new

BigQuery scripting and stored procedures entered beta in 2019; Dataform's 2023 GA completed Google's in-warehouse ELT story.

Flow

Options surveyed; next slide shows the canonical ELT pipeline shape — staging to production.

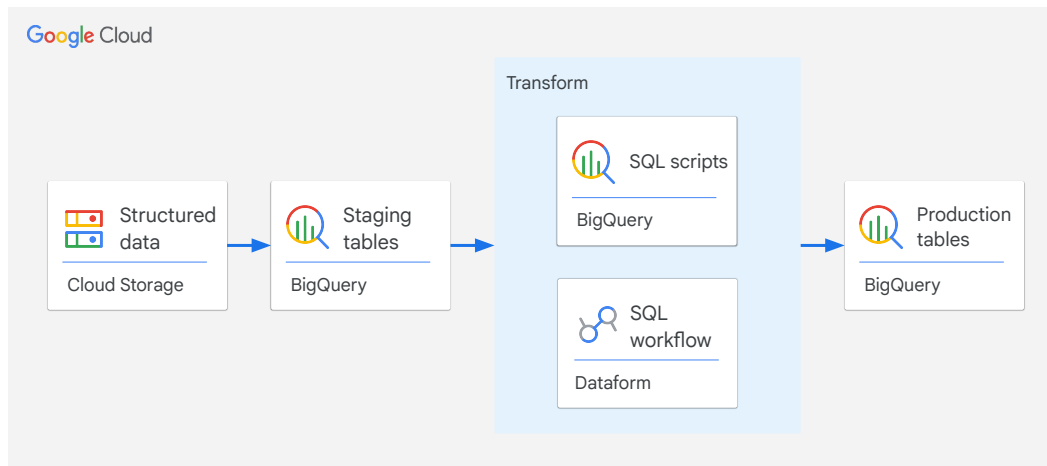
Links

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/scripting>

<https://cloud.google.com/bigquery/docs/scheduling-queries>

Use an ELT pipeline to apply transformations after data is staged in BigQuery



Google Cloud

INSTRUCTOR NOTES — PAGE 101 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Staging tables in, production tables out, SQL doing the work in between — that is your Silver->Gold architecture drawn as one diagram.

Say · cover

- Structured data arrives from Cloud Storage or this morning's Datastream and batch loads into staging tables — your Silver layer.
- The transform is a SQL workflow: plain scripts, or Dataform, reading staged tables and writing production tables — your Gold layer.
- Keep layers in separate datasets — raw, staging, marts — so permissions and lifecycle differ per layer.
- Production tables are what Power BI points at; nobody queries staging as if it were production.
- Rebuilds should be idempotent: CREATE OR REPLACE or overwrite, so any re-run heals the layer.

Iowa tie

The exact shape of their target: Bronze lands via Datastream/loads, Dataform transforms Silver->Gold, Gold marts feed Power BI.

Design patterns

- Dataset-per-layer (raw / staging / marts) with different IAM per layer
- Idempotent rebuilds so re-runs are safe

Anti-patterns

- Transforming in place over the raw layer — you lose replayability
- Analysts querying staging tables as de facto production

History & what's new

ELT rose with cloud MPP warehouses that decoupled storage from compute; Google industrialized the pattern when Dataform went GA in BigQuery in May 2023.

Flow

Pipeline shape established; next three slides are the procedural tools — scripting, UDFs, sprocs — taken fast.

Links

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/dataform/docs/dependencies>

<https://cloud.google.com/bigquery/docs/scheduling-queries>

BigQuery supports procedural language queries to perform multiple statements in a sequence

```
-- create variables
DECLARE book_name STRING DEFAULT 'Ulysses';
DECLARE book_year INT64 DEFAULT 1922;

-- Create a temporary table called Books.
EXECUTE IMMEDIATE
  "CREATE TEMP TABLE Books
    (title STRING, publish_date INT64)";

-- add a row for Ulysses, using the variables
-- declared and the ? placeholder
EXECUTE IMMEDIATE
  "INSERT INTO Books (title, publish_date) VALUES(?, ?)"
  USING book_name, book_year;
```

Procedural language in BigQuery:

- Runs multiple statements in a sequence with shared state.
- Automates management tasks, such as creating or dropping tables.
- Implements complex logic using programming constructs like **IF** and **WHILE**.
- Can declare user-created variables or reference BigQuery's system variables.
- Can use transactions (**COMMIT**, **ROLLBACK**).

Google Cloud

INSTRUCTOR NOTES — PAGE 102 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Your T-SQL control flow survives the move: DECLARE, SET, IF, WHILE, BEGIN...END — same concepts, GoogleSQL spelling.

Say · cover

- Multi-statement queries run in sequence with shared state — one script, variables visible across statements.
- EXECUTE IMMEDIATE is your dynamic SQL; bind values with ? placeholders and USING, never string-concat user input.
- Transactions exist: BEGIN TRANSACTION, COMMIT, ROLLBACK — and BEGIN...EXCEPTION WHEN ERROR with @@error system variables for handling.
- Dialect watch-list for T-SQL vets: CREATE TABLE AS instead of SELECT INTO, different date functions, INT64 not INT.
- Run scripts straight in the BigQuery editor; each runs as a single job with per-statement history.

Iowa tie

Where their extract-sproc control flow lands during migration — but steer Silver->Gold transforms toward Dataform models, not monolithic scripts.

Design patterns

- Scripting for admin and automation tasks (create/drop/swap tables)
- EXECUTE IMMEDIATE with USING parameters — injection-safe dynamic SQL

Anti-patterns

- A 5,000-line script as the pipeline: no dependency graph, no lineage, no retries

History & what's new

Scripting and stored procedures hit beta in November 2019 across all BigQuery regions and went GA in 2020.

Flow

Scripting done; UDFs next, then sprocs — keep these three slides moving.

Links

<https://cloud.google.com/bigquery/docs/reference/standard-sql/scripting>

<https://cloud.google.com/bigquery/docs/procedures>

<https://cloud.google.com/blog/products/data-analytics/command-and-control-now-easier-in-bigquery-with-scripting-and-stored-procedures>

User-defined functions (UDFs) transform data by using SQL or JavaScript

```
# create a permanent user-defined function
CREATE FUNCTION dataset_name.AddFourAndDivide
(x INT64, y INT64) RETURNS FLOAT64 AS (
  (x + 4) / y
);
```

```
# use the function in your SQL script
SELECT
  val, dataset_name.AddFourAndDivide(val, 2)
FROM ...;
```

Usage notes:

- Can be persistent (CREATE FUNCTION) or temporary (CREATE TEMPORARY FUNCTION).
- Use SQL where possible. Use JavaScript if the same cannot be accomplished with SQL.
- JavaScript UDFs can use additional input libraries.
- The `bigquery-utils` project and the `bigquery-public-data.persistent_udfs` public dataset contain community-contributed UDFs.

Google Cloud

INSTRUCTOR NOTES — PAGE 103 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Scalar UDFs, like you know from SQL Server — but persistent, shareable across the org, and able to run JavaScript when SQL runs out.

Say · cover

- Persistent functions live in a dataset (find them under Routines in the console); temporary ones last for the session.
- Write SQL UDFs first; drop to JavaScript only for what SQL cannot express — JS runs in a sandbox, slower per row.
- Before writing one, check the community libraries: the `bigquery-utils` repo and the `bigquery-public-data.persistent_udfs` dataset — URL parsing, stats, geo helpers already exist.
- UDFs compose into views and Dataform models, so one canonical cleansing function serves every mart.
- Cost note: a scalar UDF evaluates per row — fine for cleansing, wrong for per-row business workflows.

Iowa tie

Their shared conversion and cleansing logic (date formats, code mappings) becomes persistent UDFs in a shared dataset, reused by every Silver->Gold model.

Design patterns

- Centralize persistent UDFs in one shared dataset; reference from all marts
- Check `bigquery-utils` before writing your own

Anti-patterns

- Row-by-row UDF logic that should be set-based SQL
- Hiding business rules in JavaScript nobody reviews

History & what's new

Persistent UDFs arrived July 2019; remote functions later extended the idea to call Cloud Run/Cloud Functions from SQL.

Flow

Scripting and UDFs done; stored procedures next — last of the fast trio.

Links

<https://cloud.google.com/bigquery/docs/reference/standard-sql/user-defined-functions>

<https://github.com/GoogleCloudPlatform/bigquery-utils>

<https://cloud.google.com/blog/products/data-analytics/new-persistent-user-defined-functions-increased-concurrency-limits-gis-and-encryption-functions-and-more>

Stored procedures execute a collection of SQL statements

```
# create a stored procedure
CREATE OR REPLACE PROCEDURE
  dataset_name.create_customer()
BEGIN
  DECLARE id STRING;
  SET id = GENERATE_UUID();
  INSERT INTO dataset_name.customers (customer_id)
    VALUES(id);
  SELECT FORMAT("Created customer %s", id);
END
```

```
# call the procedure in your SQL script
CALL dataset_name.create_customer();
```

Usage notes:

- Can be called from queries or other stored procedures.
- Accept input values and return values as output.
- Can access or modify data across multiple datasets.
- Can contain multi-statement queries and support procedural language statements.

Google Cloud

INSTRUCTOR NOTES — PAGE 104 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

CREATE PROCEDURE, CALL, IN/OUT parameters — the T-SQL sproc, minus the server it used to live on.

Say · cover

- Procedures are routines stored in a dataset; CALL them from queries or from other procedures, with IN/OUT/INOUT arguments.
- Bodies are full multi-statement scripts — variables, IF/WHILE, transactions, exception handling all allowed.
- They can read and write across datasets, so one admin sproc can maintain many marts.
- Console: expand a dataset, look under Routines — same place UDFs live.
- A scheduled CALL works for admin chores, but for Silver->Gold transforms prefer Dataform models: dependencies, tests, and logs come free.

Iowa tie

Their run-these-three-sprocs SQL Agent steps translate to either a scheduled CALL or, better, Dataform models with a release schedule.

Design patterns

- Small, single-purpose procedures; orchestrate with Dataform, not sproc-chains

Anti-patterns

- Sproc-call-sproc chains as pseudo-orchestration — no retries, no lineage
- Critical logic living only in sprocs outside version control

History & what's new

Stored procedures beta'd November 2019; in March 2024 Spark stored procedures went GA — Python/Java/Scala on serverless Dataproc, invoked by CALL.

Flow

Fast trio complete; next slide pivots to saving and scheduling queries — the SQL Agent translation.

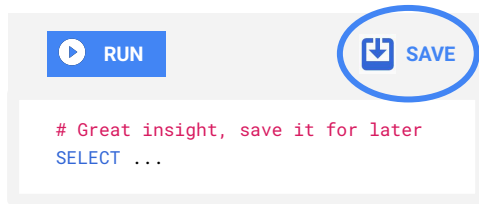
Links

<https://cloud.google.com/bigquery/docs/procedures>

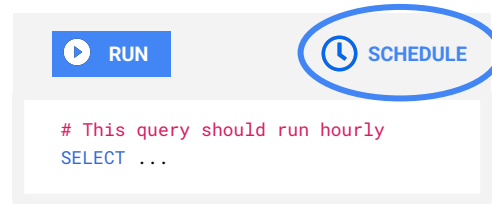
<https://cloud.google.com/blog/products/data-analytics/apache-spark-stored-procedures-in-bigquery-are-ga>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/scripting>

Save your queries and schedule them for repeating use cases



- Is visible in Explorer > Queries.
- Supports version control.
- Share with users or groups.
- Download as `.sql` file.
- Upload other queries from `.sql` files.



- Set repeat frequency.
- Set start and end times.
- Specify destination for query results.
- Can use runtime parameters.
- **Tip:** use Dataform for SQL workflows.

Google Cloud

INSTRUCTOR NOTES — PAGE 105 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Two buttons your SQL Server muscle memory already understands: SAVE is source control for a query; SCHEDULE is SQL Agent in one click.

Say · cover

- Saved queries live in Explorer, support version history, sharing, and `.sql` download/upload — commit-worthy artifacts, not clipboard text.
- Scheduling: set frequency, start/end, destination dataset and table, and write preference; runtime parameters `@run_date` and `@run_time` land in your SQL each run.
- Under the hood a scheduled query is a Data Transfer Service config running batch query jobs — priced exactly like manual queries; M&O monitors it under Scheduled queries with run history and backfill.
- `WRITE_TRUNCATE` for rebuilt snapshots — idempotent, no duplicates; `WRITE_APPEND` for accumulating history.
- Tip on the slide is the punchline: one statement on a clock is fine; multi-step workflows belong in Dataform.

Iowa tie

The kill-the-extracts interim: nightly overwriting queries rebuild derived tables until Dataform-managed Gold marts replace them.

Design patterns

- Overwrite snapshots so re-runs heal; backfill missed days from the console
- Schedule off the top of the hour — Google warns `:00` can double-trigger

Anti-patterns

- Append without dedupe for snapshot tables

History & what's new

Scheduled queries date to 2019 (intervals down to 15 minutes that July); the 2025 GA of continuous queries bookends the spectrum — SQL that never stops.

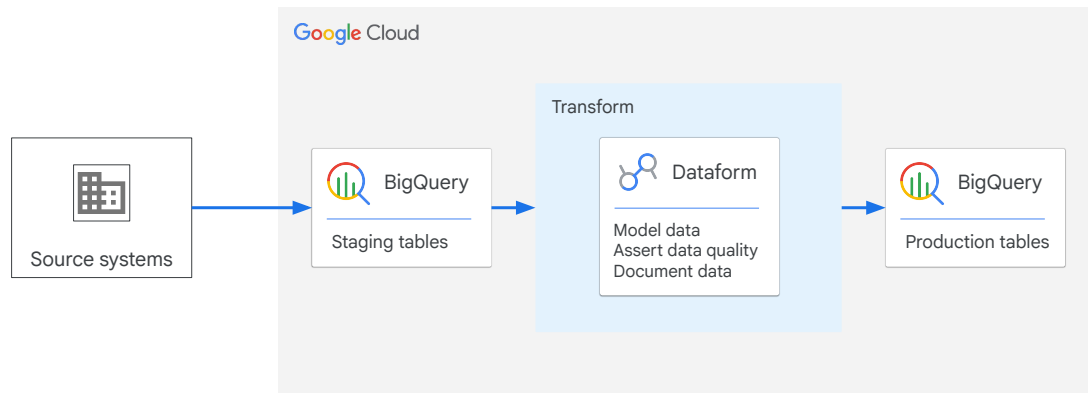
Flow

This sets up the 16:20 closing demo; next, the Dataform intro — the heart of the block.

Links

<https://cloud.google.com/bigquery/docs/scheduling-queries>
<https://cloud.google.com/bigquery/docs/continuous-queries-introduction>
<https://cloud.google.com/dataform/docs/overview>

Dataform is a serverless framework to develop and operationalize ELT pipelines in SQL



Google Cloud

INSTRUCTOR NOTES — PAGE 106 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Here it is — the tool your architecture document names for Silver->Gold. Everything so far was context; this is the centerpiece.

Say · cover

- Dataform is a serverless framework for developing and operationalizing SQL transformation pipelines inside BigQuery — nothing to provision, patch, or size.
- Three verbs on the slide: model data as versioned SQL, assert data quality in-pipeline, document data where it is defined.
- Source systems land in staging; Dataform transforms staging into production — the same picture as the ELT slide, now with a manager.
- Lives in the Google Cloud console under BigQuery -> Dataform: repositories, workspaces, executions.
- Serverless matters to this room: no SSIS server, no agent box, no scheduler VM — the transform tier is just SQL and git.

Iowa tie

Their named Silver->Gold ELT tool; this slide plus Jeff's demo plus the lab is the part they will use at work first.

Design patterns

- Transformations as code in a central git repository — reviewable like any software

Anti-patterns

- GUI-only, ad-hoc transforms nobody can reproduce or review

History & what's new

Google acquired Dataform in December 2020 and made it GA in BigQuery in May 2023, adding release and workflow configurations for scheduled deployments.

Flow

What Dataform is established; next slide shows what it replaces — the DIY framework they would otherwise assemble.

Links

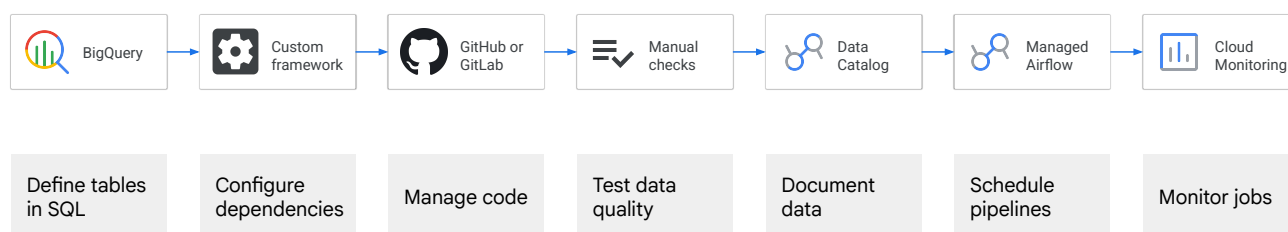
<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/blog/products/data-analytics/introducing-dataform-in-ga>

<https://cloud.google.com/dataform/docs/dataform-core>

Dataform unifies data transformation, assertion, and automation in BigQuery

Without Dataform, this would be a time-consuming and error-prone process



Google Cloud

INSTRUCTOR NOTES — PAGE 107 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Read the bottom row as the stack you would otherwise assemble yourself — and most of you have assembled exactly that around SQL Server.

Say · cover

- Without Dataform: a custom framework for table definitions, GitHub for code, manual checks for quality, a catalog for docs, managed Airflow for scheduling, Cloud Monitoring for jobs.
- With Dataform: define tables in SQL, configure dependencies, manage code, test quality, document, schedule, and monitor — one integrated surface.
- Each piece you self-assemble is a piece someone maintains at 2 AM; the integrated version is what the M&O team will thank you for.
- This is not magic — it is opinionated convention: sqlx files, a dependency graph, execution logs, release configs.
- Ask the room: which of those bottom-row tools do you run today? Their answers are the migration's hidden scope.

Iowa tie

Replaces the DIY orchestration around their sprocs; M&O gets built-in execution logs and schedules instead of Agent history plus custom alerting.

Design patterns

- Adopt the integrated tool before building custom orchestration

Anti-patterns

- Re-creating a Dataform out of scheduled queries, sprocs, and spreadsheets

History & what's new

Since the 2023 GA, Google kept folding the surrounding pieces in — July 2025 brought GA of automatic Dataform metadata cataloging in Dataplex (now Knowledge Catalog).

Flow

Why-it-matters done; next slide explains the mechanics — develop and compile in Dataform, execute in BigQuery.

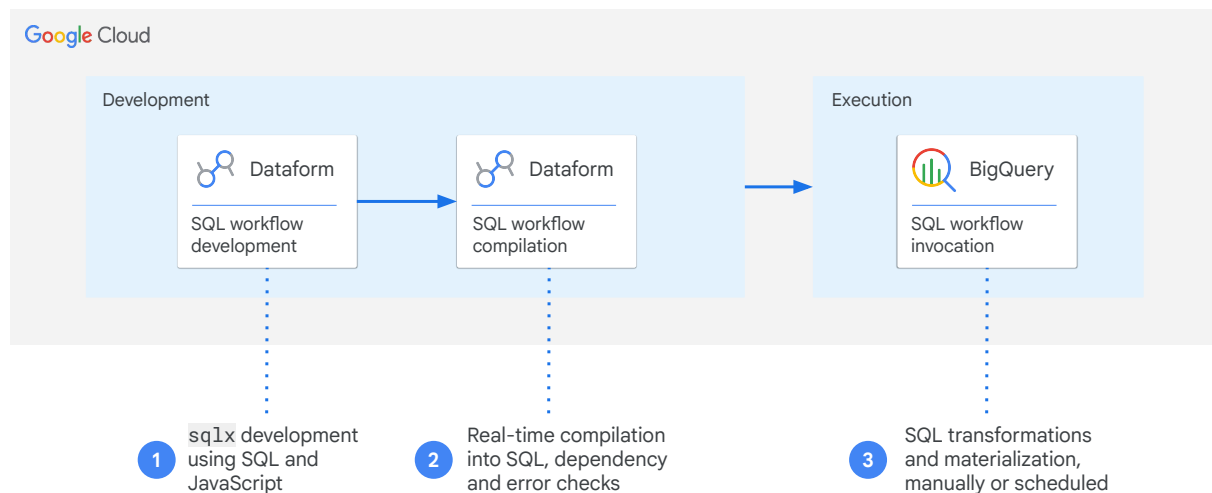
Links

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/dataform/docs/release-notes>

<https://cloud.google.com/dataform/docs/release-configurations>

SQL workflow development and compilation run in Dataform, workflow execution runs in BigQuery



Google Cloud

INSTRUCTOR NOTES — PAGE 108 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

A clean division for your security review: Dataform never touches your data — it writes SQL; BigQuery runs it.

Say · cover

- Develop: sqlx files — SQL plus a config block, with JavaScript for reuse — written in a workspace, saved via git.
- Compile: real-time and hermetic — same code always compiles to the same SQL; Dataform resolves dependencies, catches missing or circular refs, and builds the action graph in a sandbox with no network.
- Execute: BigQuery runs the compiled actions in dependency order, manually or on a schedule; every run logs status, duration, and the exact compiled SQL.
- Map to their CI/CD vocabulary: compilation is the build, a release configuration is the deployable artifact, an invocation is the deploy.
- Inspect compiled SQL during code review — what runs in prod is visible before it runs.

Iowa tie

Executions run as a service account inside their project — fits their M&O and audit model; release configs slot into the deployment pipelines they already run for SQL Server.

Design patterns

- Review the compiled SQL and graph in the PR, not just the sqlx source

Anti-patterns

- Copy-pasting compiled SQL into scheduled queries — you lose the graph and the tests

History & what's new

Hermetic compilation dates to Dataform's origins; Dataform Core 3.0.0 (June 2024) reworked it with `workflow_settings.yaml` and warehouse-agnostic compile output.

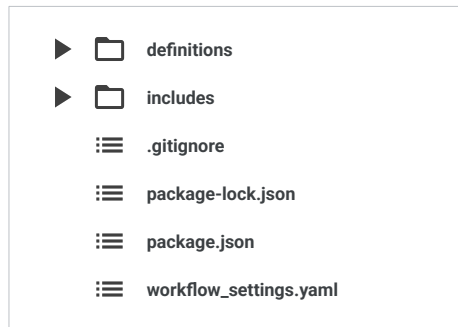
Flow

Mechanics clear; next slide shows where the files live — workspaces and the default project scaffold.

Links

<https://cloud.google.com/dataform/docs/overview>
<https://cloud.google.com/dataform/docs/dataform-core>
<https://cloud.google.com/dataform/docs/release-notes>

Development happens in workspaces with default files and folders



You can add custom files like `README.md` as well.

- **definitions folder:**
contains `sqlx` files.
- **includes folder:**
contains JavaScript files.
- **.gitignore file:**
lists files and directories to ignore during Git commits.
- **package.json file (optional):**
defines JavaScript packages and required versions.
- **package-lock.json file (optional):**
contains installed packages with exact versions.
- **workflow_settings.yaml file:**
contains basic settings required to compile the project.

Google Cloud

INSTRUCTOR NOTES — PAGE 109 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

A development workspace is your feature branch — an isolated, editable copy of the repo that compiles in real time.

Say · cover

- `definitions/` holds your `sqlx` models — tables, views, declarations, assertions; this is where 95% of work happens.
- `includes/` holds JavaScript constants and helpers reused across models — one constants file instead of forty packages burying connection strings.
- `workflow_settings.yaml` is the project settings: GCP project, default dataset, Dataform Core version; `package.json` pins `@dataform/core` and any extra packages.
- A fresh workspace is empty until you click Initialize workspace — Dataform then generates this scaffold, plus a sample `sqlx`.
- Console path: BigQuery Studio -> Dataform -> repository -> workspace; the version-control pane is your commit/push to the repo or a connected GitHub remote.

lowa tie

Direct CI/CD translation: workspace ~ feature branch, commit/push ~ code review, release config ~ their deployment pipeline — the git discipline they already run.

Design patterns

- Keep environment names and dataset IDs in one `includes/constants.js`

Anti-patterns

- Two people editing one shared workspace — separate workspaces, merge via git
- Hardcoding project IDs inside every model file

History & what's new

`workflow_settings.yaml` replaced `dataform.json` with Dataform Core 3.0.0 (June 2024); in April 2026, folders for organizing repositories and code assets went GA.

Flow

Files located; next slide turns them into a picture — the compiled dependency graph.

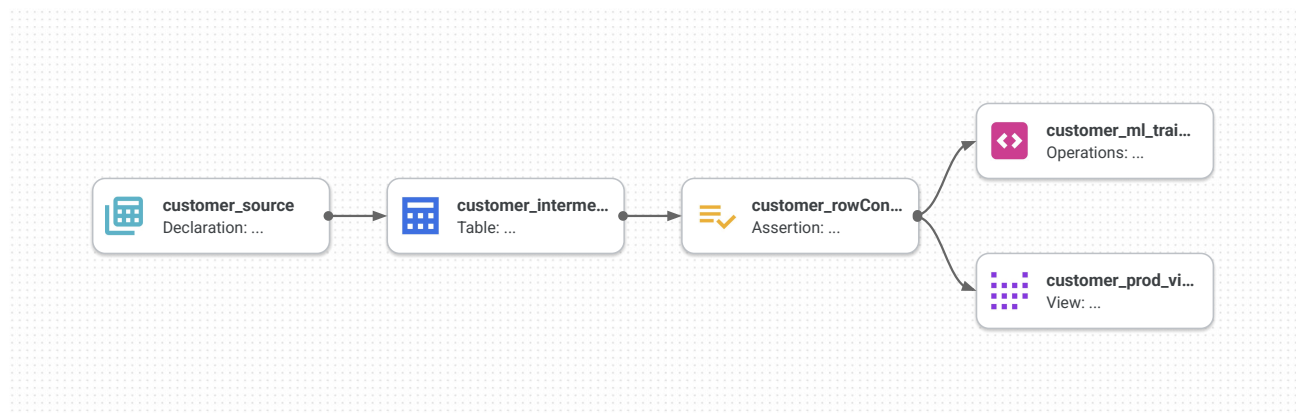
Links

<https://cloud.google.com/dataform/docs/workspaces>

<https://cloud.google.com/dataform/docs/repositories>

<https://cloud.google.com/dataform/docs/release-notes>

The compiled graph visualizes the SQL workflow with its definitions and dependencies



Google Cloud

INSTRUCTOR NOTES — PAGE 110 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

SSIS precedence constraints, finally visible: every ref() call becomes a node and an edge in a live dependency graph.

Say · cover

- Read the node types on this slide: Declaration (Bronze source landed by something else), Table, View, Operations (custom SQL), Assertion (quality test).
- The graph compiles in real time from your workspace; if compilation fails, the error replaces the graph — broken pipelines cannot silently run.
- Click any node for its metadata — full name, dependencies, dependents — and the exact compiled SQL BigQuery will run.
- Filter by type, name, or tag when the graph grows; tags let you run or review one domain at a time.
- This picture is the code-review artifact: your Silver->Gold pipeline as a diff-able, reviewable DAG.

Iowa tie

The assertion node answers their M&O question — how do we know the load is good — inside the pipeline, not in a morning email.

Design patterns

- Naming convention per layer (stg_, mart_) so the graph reads left to right
- Assertions wired into the graph, not bolted on after

Anti-patterns

- One giant model hiding all lineage — split until the graph tells the story

History & what's new

The graph keeps gaining duties: October 2026 added GA of automated metadata enrichment and Knowledge Catalog data-quality scorecards for Dataform workflows.

Flow

Graph concepts done; next Jeff walks a real repo end to end — this slide's theory made flesh.

Links

<https://cloud.google.com/dataform/docs/assertions>

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/dataform/docs/dependencies>

Demo: a real Dataform repo, end to end

15 minutes · github.com/jwdavis/dataform-demo, plus the Google Cloud console

1. Walk the repo tree: dataform.json, one constants file, definitions/ — declarations, staging, marts
2. Read the sqlx: a declaration (Bronze), staging cleanup (Silver), an aggregate mart (Gold)
3. Console: repository, GitHub connection, development workspace — your feature branch
4. Compiled graph: ref() calls become the dependency graph — your Silver→Gold pipeline as code
5. Add an assertion live, then release config + scheduled invocation — your CI/CD, translated

Why it matters — your architecture doc names Dataform for Silver→Gold — here is one running: raw → staging → marts, in git, deployable. The discipline you already run for SQL Server.

Hook

Enough slideware — here is an actual Silver->Gold pipeline in git, the shape your architecture document prescribes, running in the console.

Say · cover

- Walk the repo tree first: project settings, one includes/constants file, definitions/ split into declarations, staging, marts — nine small files.
- Read three sqlx files aloud: a declaration pulling Bronze into the graph, a staging cleanup (Silver), an aggregate mart (Gold) wired with ref().
- In the console show the repository, its GitHub connection, and a development workspace — narrate it as their feature branch.
- Open the compiled graph: ref() calls became the DAG — then add a uniqueKey or rowConditions assertion live and watch a new node appear.
- Finish with release configuration plus scheduled invocation — that is their CI/CD deployment pipeline, translated.
- Students can browse the public repo along with you; invite interruptions — the CI/CD-minded will have the sharpest questions of the day.

Iowa tie

Their architecture doc names Dataform for Silver->Gold; this demo is that sentence made real — deployable, tested, in git.

Design patterns

- Declarations bring externally loaded Bronze tables into the graph as first-class sources

Anti-patterns

- Demoing clicks without narrating the git/release story — the discipline is the point

History & what's new

Dataform GA'd in 2023 and keeps moving — September 2026 brought GA of a remote MCP server so AI agents can author pipelines in workspaces.

Flow

15:10 demo ends; 15:25 they build the same skeleton themselves in the anchor lab.

Links

<https://github.com/jwdavis/dataform-demo>

<https://cloud.google.com/dataform/docs/overview>

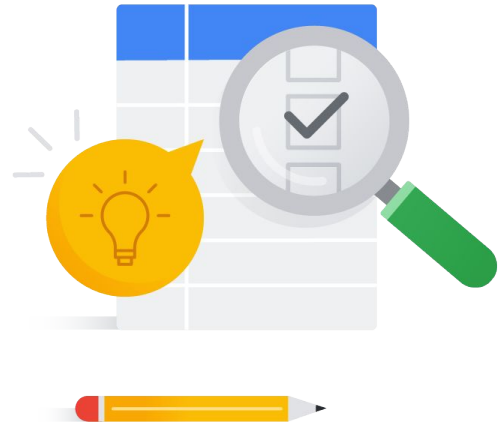
<https://cloud.google.com/dataform/docs/release-configurations>

Lab: Create and Execute a SQL Workflow in Dataform

🕒 30 min

Learning objectives

- Create a Dataform repository.
- Create and initialize a Dataform development workspace.
- Create and execute a SQL workflow.
- View execution logs in Dataform.



Google Cloud

INSTRUCTOR NOTES — PAGE 112 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Four objectives, one arc: repository, workspace, sqlx workflow, execution logs — the same skeleton you just watched, now under your own hands.

Say · cover

- Create a Dataform repository — in the lab project, in the region the lab specifies.
- Create and initialize a development workspace — nothing appears until Initialize.
- Create and execute the SQL workflow: a staging view and a mart table wired by ref().
- View execution logs — actions ran in dependency order, and the exact compiled SQL is there to read.
- Courseware says 30 minutes; we give it the full 40 — the run-sheet and gotchas are on the next slide.

Iowa tie

The migration in miniature: SQL models in git, compiled into a graph, executed in order — what they will do with real DOM data after the course.

Design patterns

- Work only in the lab-provisioned project; verify the project picker first

Anti-patterns

- Stopping at green checkmarks — open the logs and read the compiled SQL

History & what's new

This lab targets Dataform in BigQuery, GA since May 2023; the older standalone legacy Dataform was deprecated in February 2024.

Flow

Objectives set; next slide is our custom run-sheet with timings, gotchas, and the early-finisher stretch.

Links

<https://cloud.google.com/dataform/docs/overview>
<https://cloud.google.com/dataform/docs/create-workspace>
<https://cloud.google.com/dataform/docs/assertions>

Lab: Create and Execute a SQL Workflow in Dataform

40 minutes · Google Skills classroom — lab project, BigQuery Studio → Dataform

1. Create a Dataform repository — watch the region; a mismatch is the classic stall
2. Create and initialize a development workspace — files appear only after Initialize
3. Define a staging view and a mart table in sqlx, wired together with ref()
4. Execute the workflow and read the logs — actions run in dependency order, compiled SQL shown
5. Early finishers: add a uniqueKey or rowConditions assertion and watch it enter the graph

Why it matters — the one solid lab in the course — these 40 minutes are exactly what your migration does at work: SQL models in git, compiled into a graph, executed in order, tested in-pipeline.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 113 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

The one solid lab in the course — give it the full 40 minutes and roam; SQL people fly here, so your job is unblocking.

Say · cover

- Region mismatch at repository creation is the classic stall — repo in one region, datasets in another surfaces later as empty executions.
- Second stall: a workspace that looks empty or broken — they missed Initialize workspace; files appear only after it.
- Third: compiled-graph lag — a saved model takes a few seconds to appear in the graph; wait before assuming it did not save.
- IAM permission errors on execution: runs execute as a service account — flag the instructor rather than burning ten minutes.
- Early finishers: add a uniqueKey or rowConditions assertion, re-run, and watch it enter the graph and the execution logs.
- Debrief line: you just did ELT the way your target architecture will — models in git, compiled graph, ordered execution, tests in-pipeline.

Iowa tie

Exactly their day job after migration; expect the sharpest git/workspace/release questions from the CI/CD-minded — workspace ~ feature branch, release ~ deployment.

Design patterns

- Roam at minutes 10–25; park stalled students with a note instead of stopping the room

Anti-patterns

- Compressing this slot when module slides run long — cut slides, never lab minutes

History & what's new

Assertions matured fast: built-in uniqueKey/nonNull/rowConditions, and unit tests with mocked inputs arrived in Dataform Core 3.0.56 (2025).

Flow

Lab runs 15:25–16:05; next after it: M5+M6 compressed, then the scheduled-query closing demo.

Links

<https://cloud.google.com/dataform/docs/assertions>

<https://cloud.google.com/dataform/docs/create-workspace>

<https://cloud.google.com/dataform/docs/overview>

Close: schedule the derived-table query

5 minutes · BigQuery Studio — Schedule → Create new scheduled query

1. Run the sales-by-zip query once into class.march_zip_sales
2. Schedule → Create new scheduled query: repeat daily at 12:01 AM
3. Destination: your class dataset; write preference Overwrite (WRITE_TRUNCATE)
4. Save, find it under Scheduled queries, trigger one manual run to prove it fires

Why it matters — a SQL statement, on a schedule, overwriting a table — your nightly SQL Agent job minus the server. Day 2 replaces hand-rolled derived tables with Dataform-managed marts.

Hook

End Day 1 with the cheapest automation on Google Cloud: a SQL statement, on a schedule, overwriting a table — your nightly SQL Agent job minus the server.

Say · cover

- Run the sales-by-zip query once into class.march_zip_sales to prove it works before scheduling anything.
- Schedule → Create new scheduled query: daily at 12:01 AM — and note Google's own warning: top-of-the-hour schedules can double-trigger, so :01 is a feature, not a quirk.
- Destination: the class dataset; write preference Overwrite (WRITE_TRUNCATE) — every run rebuilds the table atomically, so re-runs are idempotent.
- Find it under Scheduled queries in the BigQuery console; trigger one manual run to prove it fires — run history lives there for M&O.
- Under the hood it is a Data Transfer Service config running batch jobs, priced like manual queries — no scheduler box to own.

Iowa tie

Their interim automation while extracts die: nightly rebuilt derived tables feeding Power BI, until Day 2's Dataform-managed marts take over.

Design patterns

- Prove the query manually first, then schedule; manual run after saving to validate

Anti-patterns

- Append write preference for a rebuilt snapshot — duplicates accumulate silently

History & what's new

Scheduled queries date to 2019; the opposite extreme — continuous queries, SQL that never stops — went GA in 2025.

Flow

Five-minute closer after M5+M6 compressed; then Day 1 wrap and a preview of Day 2's medallion and governance.

Links

<https://cloud.google.com/bigquery/docs/scheduling-queries>

<https://cloud.google.com/bigquery/docs/continuous-queries-introduction>

<https://cloud.google.com/dataform/docs/release-configurations>

M4 — What you now have

- ✓ ELT on BigQuery— land Bronze, let SQL build Silver and Gold inside the warehouse
- ✓ Dataform, hands-on— repo, workspace, sqlx models, the ref() graph, executed and logged
- ✓ The git discipline you already run— workspace = feature branch, release config = deployment
- ✓ A scheduled query overwriting a derived table nightly— SQL Agent translated, first automation
- ✓ Tomorrow: the medallion and governance— Gold marts feeding the Power BI you already have

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 115 OF 135 · 14:45 · MODULE 4 — ELT & DATAFORM

Hook

Eighty minutes ago Dataform was a name in your architecture doc; now you have run a Silver->Gold pipeline with your own hands.

Say · cover

- ELT on BigQuery: land Bronze, let SQL build Silver and Gold inside the warehouse — no transform server anywhere in the picture.
- Dataform hands-on: repository, workspace, sqlx models, the ref() graph, executed and logged — keep your lab repo as a reference skeleton.
- The git discipline you already run mapped cleanly: workspace = feature branch, release configuration = deployment, scheduled invocation = the SQL Agent schedule.
- A scheduled query overwriting a derived table nightly: your first GCP automation, running as we speak.
- Tomorrow: the medallion architecture and governance — Gold marts feeding the Power BI you already have; the 3-day BigQuery data-warehousing course being scheduled goes deeper still.

Iowa tie

All five bullets are pieces of their target operating model — Silver->Gold transforms, CI/CD discipline, M&O-friendly automation, Power BI last mile.

Design patterns

- Treat the lab repo as the seed skeleton for their first real Silver->Gold domain

Anti-patterns

- Leaving hand-rolled scheduled queries as the final pattern instead of the bridge to Dataform marts

History & what's new

The deployment story keeps maturing: folders for repositories went GA in April 2026, and centralized Dataform deployments entered preview in July 2026.

Flow

M4 block and the teaching core of Day 1 close here; tomorrow opens with the 09:00 recap quiz.

Links

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/dataform/docs/release-configurations>

<https://cloud.google.com/dataform/docs/schedule-runs>

Lab — Create & Execute a SQL Workflow in Dataform

40 minutes — 15:25–16:05 · Google Skills classroom — Course 1, Module 4 lab

1. Launch the lab; work in the lab project, in the region the lab specifies — check the picker
2. Create a Dataform repository (BigQuery Studio → Dataform)
3. Create a development workspace and Initialize it — the #1 stall: no files until you do
4. Define a staging view and a mart table in sqlx, wired with ref() — Silver→Gold in miniature
5. Execute the workflow; read the execution logs — each action's status and compiled SQL
6. Early? Add an assertion (uniqueKey / rowConditions) and watch it join the compiled graph

Why it matters — your architecture names Dataform as the Silver→Gold ELT tool — these 40 minutes are the thing you will actually do at work.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 116 OF 135 · 15:25 · LAB — SQL WORKFLOW IN DATAFORM (40 MIN)

Hook

These 40 minutes are what your architecture doc was written for — Dataform, your named Silver->Gold ELT tool; treat this as day one on the new job.

Say · cover

- Lab project, lab region — resources look 'missing'? It's the project picker.
- BigQuery Studio -> Dataform: repository -> development workspace -> Initialize workspace; empty file tree = skipped that button.
- Staging view + mart table in sqlx, wired with \${ref()} — Dataform compiles ref() into the dependency graph: Silver->Gold in miniature.
- Execute, then read the logs: per-action status and exact compiled SQL — your SQL Agent job-history instinct.
- Early finishers: add an assertion (uniqueKey / rowConditions) and watch it join the graph. Workspace = branch; release config = deployment.

Iowa tie

Dataform is your Silver->Gold ELT tool feeding Power BI marts — git workspaces and release configs map onto your existing CI/CD discipline.

Design patterns

- ref() declares dependencies — graph and run order come free
- Assertions test data where it is made
- One workspace per developer, merged via git

Anti-patterns

- Hardcoding dataset.table instead of ref()
- One shared team workspace
- 'Start execution' is not deployment — prod runs off release configurations

History & what's new

Google acquired Dataform in 2020; re-platformed into Google Cloud (preview 2022, Core 3.0 in 2024). April 2026: strict act-as mode enforced globally — workflows must run under a custom service account; plan IAM with M&O early.

Flow

Follows Jeff's dataform-demo walkthrough; next the compressed M5+M6 block (16:05), then the closing scheduled-query demo.

Links

<https://cloud.google.com/dataform/docs/quickstart-create-workflow>

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/dataform/docs/release-notes>

WATCH FIRST · 3:20 · GOOGLE CLOUD TECH

Moving Cron Jobs to the Cloud

- ✓ Cloud Scheduler = managed cron that calls HTTP, Pub/Sub, or App Engine — the SQL Agent analogue with no scheduler VM to patch.
- ✓ Fifteen minutes from now you'll watch a scheduled BigQuery query fire — this is the general-purpose version of that same idea.



Scan to open on YouTube
youtu.be/7Z1mgOxWTs8

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 117 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

- Play to open the compressed M5+M6 block — the cron framing is instantly familiar to the M&O team.

Say

- “SQL Agent without the agent — and the closing demo schedules a query with zero infrastructure.”

Cue

- Plays in full; pause on the trigger-types screen (~1:40) — that is the SQL Agent mapping.

Link

- <https://www.youtube.com/watch?v=7Z1mgOxWTs8>

M5 + M6 — What this 15-minute block gets you

- ✓ M5 in one line— the escape hatch when SQL is not enough: custom code, streaming, non-BigQuery targets
- ✓ Batch vs streaming as a latency decision— Pub/Sub, Dataflow and templates, named not drilled
- ✓ M6 = your CI/CD analogue— Cloud Scheduler ≈ SQL Agent, Composer ≈ the orchestrator
- ✓ Aimed at the M&O team— who keeps jobs running: scheduling, retries, failure visibility
- ✓ No lab in this block— the Spark lab is optional homework; the close schedules one live query

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 118 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

You already own schedulers and SSIS packages — the next 15 minutes simply maps your SQL Agent and master-package instincts onto Google Cloud.

Say · cover

- M5 answers one question: what you reach for when SQL in BigQuery is not enough — custom code, streaming, non-BigQuery targets.
- Names, not depth: Pub/Sub, Dataflow, templates — recognize them and when to use them, no configuration today.
- M6 is your CI/CD analogue: Cloud Scheduler ~ SQL Agent, Composer ~ the master orchestrator.
- M6 is aimed at the M&O team: retries, failure visibility, who gets paged.
- No lab in this block — the Spark lab is optional homework; we close by scheduling one live query.

Iowa tie

This is the M&O slice of the migration: after Dataform builds Silver->Gold, these services run it on a cadence and surface failures — same discipline as your SQL Server CI/CD, new buttons.

Design patterns

- Default to ELT in BigQuery; open the escape hatch only when SQL genuinely cannot do the job.

Anti-patterns

- Rebuilding every SSIS package in Dataflow — most extract logic dies in this migration.

History & what's new

The streaming ideas trace to Google's 2015 VLDB paper 'The Dataflow Model', now Apache Beam; on the automation side, Cloud Composer 3 — managed Airflow with no visible cluster — went GA in March 2025.

Flow

Just finished the 15:25 Dataform SQL-workflow lab; this compressed double-module runs 16:05–16:20, then the closing scheduled-query demo.

Links

<https://cloud.google.com/dataflow/docs>
<https://cloud.google.com/scheduler/docs>
<https://cloud.google.com/composer/docs>

Last block of Day 1 — no lab, then we close

16:05–16:15 M5 compressed — ETL options: batch vs streaming, Pub/Sub, Dataflow, templates

16:15–16:20 M6 compressed — automation: Scheduler, Composer, Dataform release configs

16:20–16:30 Close: schedule the derived-table query live, Day 1 recap, Day 2 preview

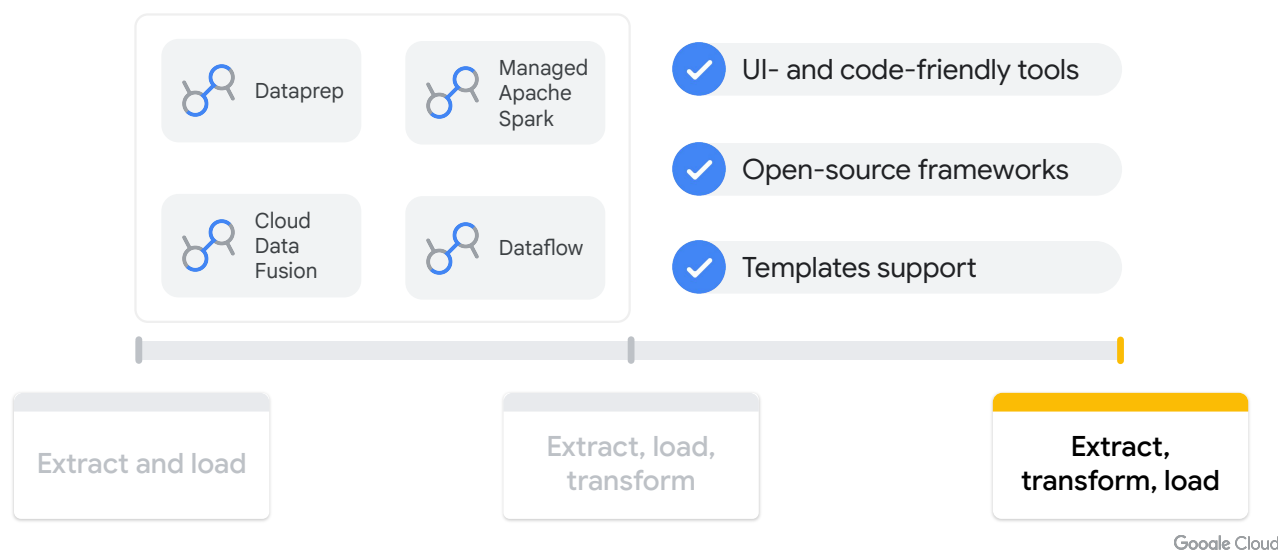
DEMO

FACILITATOR NOTE

- Timing: ten minutes of M5 (ETL options), five of M6 (automation), then the closing demo — no lab, so park deep questions for 16:20 or office hours.
- If discussion runs long, the agenda's overflow valve moves this material to Day 2 at 09:00 before the recap quiz.
- Energy management: this is the last content block of a dense day — keep it brisk and conceptual.

Flow: Follows the p114 framing slide; next is Google's ETL-services landscape slide.

Google Cloud provides multiple services for distributed data processing



INSTRUCTOR NOTES — PAGE 120 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Think of this as the second aisle of the toolbox next to the BigQuery aisle you already know — four code-friendly ways to move data when SQL won't.

Say · cover

- The slide sorts options two ways: UI- and code-friendly tools versus open-source frameworks, across ETL and ELT shapes.
- Dataflow is managed Apache Beam — Google's flagship runner for batch and streaming pipelines.
- Dataproc, now branded Managed Service for Apache Spark, runs existing Spark/Hadoop jobs — lift-and-shift, not new development.
- Cloud Data Fusion is the drag-and-drop pipeline UI, built on open-source CDAP — the closest thing to the SSIS designer.
- Dataprep was the visual data-prep tool; treat it as legacy, not a destination.

Iowa tie

For your SQL Server->BigQuery move none of these is the default — Dataform ELT covers Silver->Gold. These exist for the rare SSIS package whose logic cannot become SQL.

Design patterns

- Match tool to team skill: SQL-first teams stay in Dataform; UI pipelines go to Data Fusion; code pipelines go to Dataflow.

Anti-patterns

- Adopting a new tool per use case — four overlapping services are a menu, not a checklist.

History & what's new

Cloud Dataprep launched at Google Cloud Next 2017 and is deprecated as a standalone product; visual prep survives as Wrangler inside Cloud Data Fusion (renamed in its 2025 release notes).

Flow

First Google slide of the block; the next slide reframes this same landscape as 'escape hatch, not destination.'

Links

<https://cloud.google.com/dataflow/docs>

<https://cloud.google.com/data-fusion/docs/concepts/overview>

<https://cloud.google.com/products/managed-service-for-apache-spark>

M5 — the escape hatch, not the destination

- Default stays ELT in BigQuery— reach beyond SQL only for custom code, streaming, non-BQ targets
- Batch vs streaming— a latency decision, not a religion; nightly covers most warehouse work
- Pub/Sub— the managed messaging backbone; your Service Broker / queue analogue
- Dataflow— managed Apache Beam runner for batch and stream; templates = configure, not code
- Depth lives elsewhere— the Spark lab is optional homework; streaming is Course 4

Conceptual pass only — know the names and when to reach for each.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 121 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Your default tomorrow is still SQL in BigQuery — this module exists so you know where the exits are, not so you take them.

Say · cover

- Escape-hatch triggers: custom code SQL cannot express, true streaming latency, or targets outside BigQuery.
- Batch versus streaming is a latency decision, not a religion — nightly windows cover most warehouse work, including yours.
- Pub/Sub is the managed messaging backbone — your Service Broker / queue analogue.
- Dataflow is the managed Apache Beam runner; templates turn common pipelines into configuration.
- Depth is deliberately deferred: the Spark lab is optional homework and streaming belongs to Course 4.

Iowa tie

Bronze->Silver->Gold via Datastream plus Dataform needs no code pipelines — keep this module as vocabulary for the one legacy job that refuses to become a Dataform model.

Design patterns

- Prove SQL cannot do it before opening the escape hatch; write down the latency requirement that forces streaming.

Anti-patterns

- Streaming because it sounds modern — you pay for it in operational complexity forever.

History & what's new

Dataproc Serverless went GA in 2022 billed as the industry's first autoscaling serverless Spark; in 2025 Google unified it with Dataproc under the Managed Service for Apache Spark brand.

Flow

Instructor framing slide for the M5 concept pass; next comes the batch-versus-streaming distinction itself.

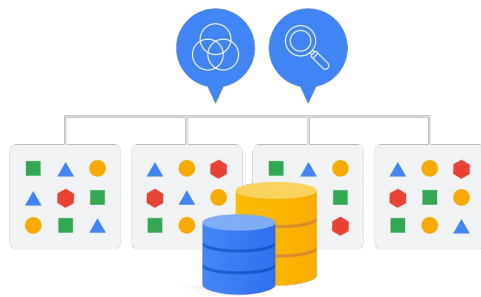
Links

<https://cloud.google.com/products/managed-service-for-apache-spark>

<https://cloud.google.com/pubsub/docs/overview>

<https://cloud.google.com/dataflow/docs/overview>

Batch processing versus streaming data processing

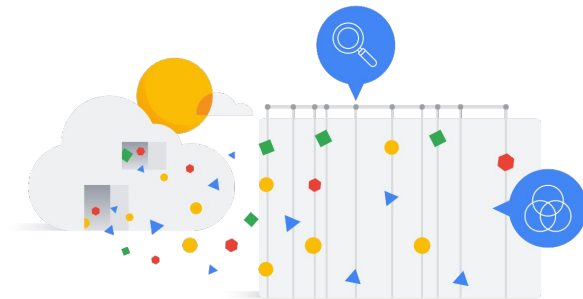


Batch processing

Processing and analysis happens on a set of stored data.

Payroll system

Billing system



Streaming data processing

A flow of data records generated by various data sources.

Fraud detection

Intrusion detection

Google Cloud

INSTRUCTOR NOTES — PAGE 122 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

You have lived both already: the nightly SSIS extract is batch; a Service Broker-triggered process is streaming. The only new question is what latency the business will pay for.

Say · cover

- Batch processes a bounded, stored set of data — payroll and billing are the slide's examples; your nightly warehouse load is another.
- Streaming processes an unbounded flow of records as they arrive — fraud detection and intrusion detection are the canonical cases.
- The trade is latency versus operational simplicity: streaming costs more to build, monitor, and debug.
- Most BI reporting tolerates hours — even Power BI refresh schedules are themselves batch.
- Rule of thumb: choose the slowest acceptable latency; faster is a cost, not a virtue.

Iowa tie

Killing the nightly extract window with Datastream CDC is your one streaming-adjacent move; the Silver->Gold marts feeding Power BI stay happily batch.

Design patterns

- Record each pipeline's latency requirement — hours, minutes, or seconds — before choosing its engine.

Anti-patterns

- Declaring everything 'real-time' — a daily mart does not need sub-second plumbing.

History & what's new

Google's 2015 VLDB paper 'The Dataflow Model' argued batch is just a special case of streaming; that model became Apache Beam, a top-level Apache project since January 2017.

Flow

Second concept slide of M5; next slide assembles the canonical streaming pipeline: Pub/Sub, Dataflow, BigQuery.

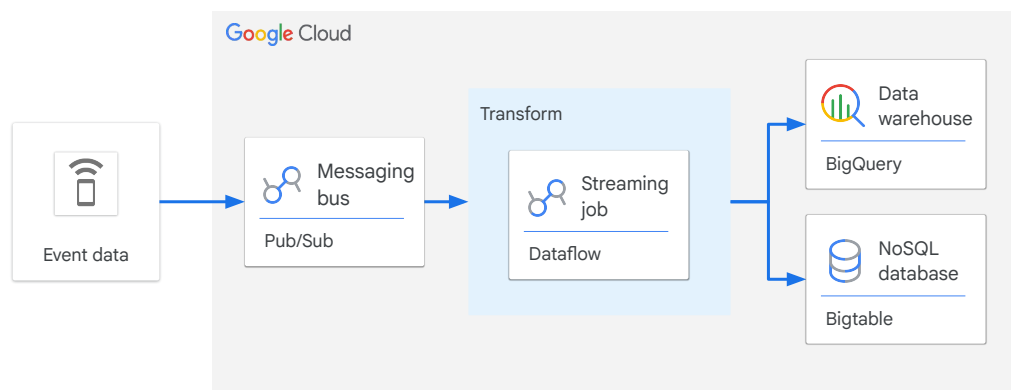
Links

<https://cloud.google.com/dataflow/docs/concepts/streaming-pipelines>

<https://cloud.google.com/pubsub/docs/overview>

<https://cloud.google.com/bigquery/docs/streaming-data-into-bigquery>

Streaming ETL workloads require continuous data ingestion, processing, and near real-time analytics



Google Cloud

INSTRUCTOR NOTES — PAGE 123 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

This is the whiteboard assembly to recognize: events hit a messaging bus, a streaming job transforms them, and results land where they get queried.

Say · cover

- Pub/Sub decouples producers from processors — the bus absorbs bursts so the Dataflow job sees a steady stream.
- Dataflow runs the continuous transform: parse, filter, enrich, window.
- BigQuery is the analytical sink; Bigtable serves low-latency NoSQL lookups — different consumers, same events.
- One event stream can fan out to both sinks simultaneously.
- For this team: the architecture you grow into if a source ever outgrows nightly loads.

Iowa tie

Not part of today's SQL Server migration — but this is the exact pattern if a future Power BI dashboard must reflect events within seconds instead of waiting for refresh.

Design patterns

- Keep the bus dumb and transforms in the pipeline; land raw events alongside curated output so you can replay.

Anti-patterns

- Point-to-point feeds from every producer to every consumer — that coupling is exactly what the bus eliminates.

History & what's new

The bus keeps absorbing features: Pub/Sub Single Message Transforms, announced June 2025, do light format, filter, and masking work in transit — shrinking what Dataflow must handle.

Flow

Architecture assembled; the next two slides drill the two named pieces — Pub/Sub first.

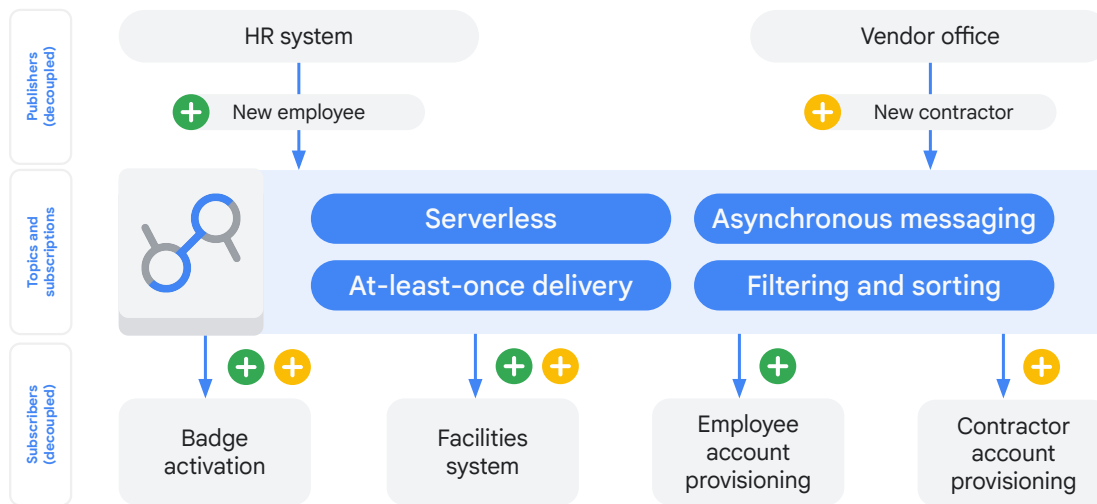
Links

<https://cloud.google.com/pubsub/docs/overview>

<https://cloud.google.com/dataflow/docs/overview>

<https://cloud.google.com/bigquery/docs/streaming-data-into-bigquery>

Use Pub/Sub to ingest high volumes of event data and distribute to different consuming systems



Google Cloud

INSTRUCTOR NOTES — PAGE 124 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Pub/Sub is Service Broker rebuilt as a planet-scale managed service: publishers and subscribers never meet, and Google owns the uptime.

Say · cover

- Publishers send messages to topics; subscribers receive from subscriptions — many-to-many, fully decoupled.
- Serverless and asynchronous: no brokers to patch, and capacity absorbs bursts like the slide's badge-activation example.
- Delivery is at-least-once by default — consumers must tolerate duplicates; exactly-once is an opt-in per subscription.
- Subscription filtering lets each consumer take only the events it cares about.
- Console location: search 'Pub/Sub' -> Topics and Subscriptions tabs; this is where event feeds terminate.

Iowa tie

Your migration uses Datastream's managed CDC instead of hand-rolled events — but Pub/Sub is the glue when M&O wants 'file arrived -> run pipeline' triggers later.

Design patterns

- Design consumers idempotent from day one; at-least-once means duplicates will happen.

Anti-patterns

- Assuming ordering or exactly-once delivery by default — neither is the baseline.

History & what's new

In June 2025 Google announced Pub/Sub Single Message Transforms: validation, format conversion, and PII masking applied per-message inside the bus — no pipeline code required.

Flow

Messaging piece done; next is the processing engine that usually consumes it — Dataflow.

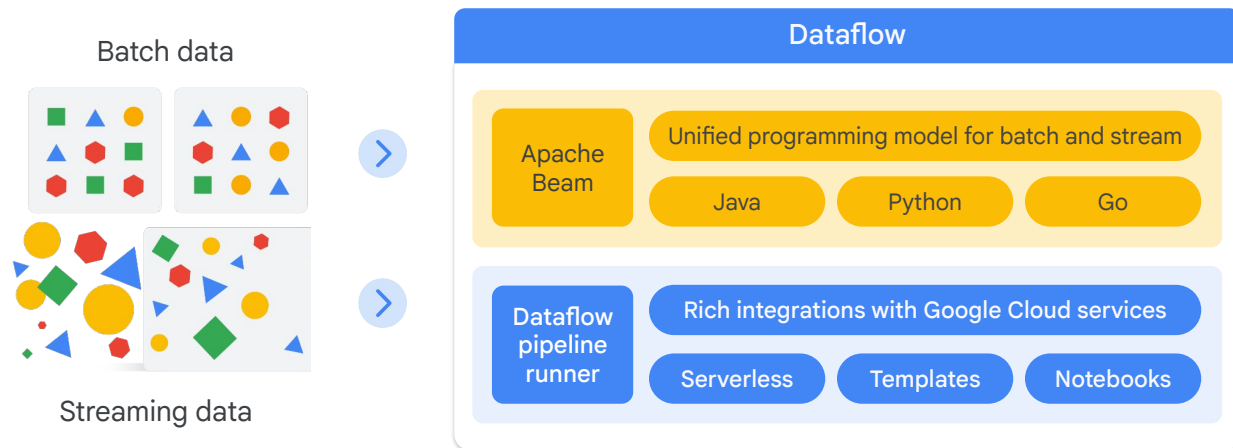
Links

<https://cloud.google.com/pubsub/docs/overview>

<https://cloud.google.com/pubsub/docs/smts/smts-overview>

<https://cloud.google.com/pubsub/docs/exactly-once-delivery>

Dataflow can process both batch and streaming data using the Apache Beam programming model



Google Cloud

INSTRUCTOR NOTES — PAGE 125 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Dataflow is 'SSIS runtime as a service' for code pipelines: you write the transform once in Apache Beam, and Google provisions and autoscales the workers.

Say · cover

- Apache Beam is the unified programming model — the same pipeline code runs batch or streaming.
- Dataflow is Google's managed runner for Beam: serverless, no cluster to size or patch.
- SDKs: Java and Python are first-class; Go is supported — the three languages on the slide.
- Rich integrations: Pub/Sub in, BigQuery and Bigtable out, notebooks for development.
- Templates package a pipeline so non-developers run it with parameters — covered after the code sample.

Iowa tie

Your SQL team will not write Beam — flag it as specialist territory; what matters is Dataflow jobs appear in the same console and billing M&O already watches.

Design patterns

- One Beam codebase serving both batch backfills and streaming increments beats maintaining two engines.

Anti-patterns

- Custom Beam code for work a Google-provided template or plain SQL already covers.

History & what's new

Google donated the Dataflow SDKs to found Apache Beam in early 2016; Beam became a top-level Apache project in January 2017, and Google shipped new Dataflow streaming and ML features in January 2026.

Flow

Engine introduced; the next slide shows actual Beam Python reading Pub/Sub into BigQuery.

Links

<https://cloud.google.com/dataflow/docs/overview>

<https://cloud.google.com/dataflow/docs/concepts/beam-programming-model>

<https://cloud.google.com/blog/products/data-analytics/new-dataflow-features-to-enable-streaming-and-ml-workloads>

Example: Streaming and transforming messages from Pub/Sub to BigQuery

```
with beam.Pipeline(options=pipeline_options) as pipeline:

    # Read messages from Pub/Sub
    messages = pipeline | 'read' >> ReadFromPubSub(topic='<PUBSUB_TOPIC>')

    # Parse messages
    def parse_message(message):
        return json.loads(message.decode('utf-8'))

    parsed_messages = messages | 'parse' >> beam.Map(parse_message)

    # Write parsed messages to BigQuery
    parsed_messages | 'write' >> WriteToBigQuery(
        table='<BIGQUERY_TABLE>',
        schema=table_schema,
        create_disposition=BigQueryDisposition.CREATE_IF_NEEDED,
        write_disposition=BigQueryDisposition.WRITE_APPEND
    )

    # Run the pipeline
    pipeline.run()
```

ReadFromPubSub():

- Retrieves messages from a Pub/Sub topic or subscription.

beam.Map():

- Applies a specified transformation to the message.

WriteToBigQuery():

- Writes the transformed message into a BigQuery table.

Google Cloud

INSTRUCTOR NOTES — PAGE 126 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Do not teach the code — decode its shape for SQL people: three named steps that read, map, and write, exactly like an SSIS data flow's source-transform-destination.

Say · cover

- ReadFromPubSub pulls messages from the topic — the source adapter.
- beam.Map applies a Python function per message — here JSON parsing, your derived-column transform.
- WriteToBigQuery appends rows and can create the table — note the WRITE_APPEND disposition.
- The pipeline is declarative: Beam builds the graph, Dataflow executes it distributed and autoscaled.
- Punchline: Google ships this exact pipeline as a pre-built template — the code is optional.

Iowa tie

If a future feed must land JSON events into a Bronze table continuously, this ten-line shape — or its no-code template — is the whole job; no SSIS package required.

Design patterns

- CREATE_IF_NEEDED plus WRITE_APPEND suits append-only event Bronze tables; keep the schema explicit in code.

Anti-patterns

- Hand-rolling pipelines that duplicate a Google-provided template — configure before you code.

History & what's new

The beam.Map parse step is exactly what Pub/Sub Single Message Transforms (June 2025) can now do inside the bus, before Dataflow ever sees the message; Beam's Python idioms date to the 2016 open-sourcing of the Dataflow SDKs.

Flow

Code demystified; the next slide removes the code entirely with Dataflow templates.

Links

<https://cloud.google.com/dataflow/docs/guides/templates/provided/pubsub-to-bigquery>
<https://cloud.google.com/dataflow/docs/concepts/beam-programming-model>
<https://cloud.google.com/bigquery/docs/streaming-data-into-bigquery>

Use Dataflow templates for recurring pipeline executions with different parameters

+ CREATE JOB FROM TEMPLATE

Dataflow template

Cloud Storage Text to BigQuery (Stream) ▼

Source

gs:// Cloud Storage Input File(s) **BROWSE**

Target

BigQuery output table **BROWSE**

Required Parameters ▼

Benefits of using templates

- Pipeline design is separate from deployment.
- Parameters customize the pipeline and make it reusable with different inputs.
- Templated pipelines can be deployed from the Google Cloud console, command-line interface, or API.
- Use Google-provided pre-built templates for common scenarios.



You can create your own custom templates.

Google Cloud

INSTRUCTOR NOTES — PAGE 127 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Templates are the 'deploy the .dtsx, hand ops a config screen' pattern: a developer builds once, and M&O reruns forever with different parameters.

Say · cover

- Pipeline design is separated from deployment — a developer stages the template once.
- Each run is parameter-driven: input path, output table — the slide's Cloud Storage Text to BigQuery stream example.
- Runs launch from the console, gcloud, or API — no Beam SDK on the operator's machine.
- Google ships pre-built templates for common routes; you can also build custom templates for your own logic.
- Two generations exist: classic templates and newer Flex templates packaged as container images.

Iowa tie

For M&O this is the parameterized SQL Agent job analogue — and the GCS Text->BigQuery template could serve one-off Bronze file loads with zero code.

Design patterns

- Standardize on Google-provided templates for standard routes; reserve custom Flex templates for proprietary logic.

Anti-patterns

- Rebuilding the pipeline per environment — parameterize once and promote the same template through dev/test/prod.

History & what's new

Dataflow templates evolved from classic templates to the newer container-packaged Flex generation; Google's provided-template catalog keeps growing, with streaming and ML pipeline options added in the January 2026 Dataflow update.

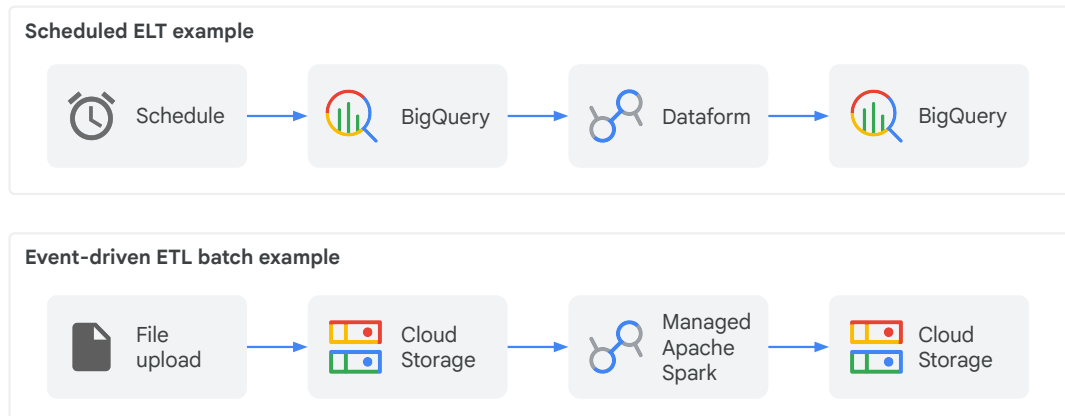
Flow

M5 ends here — names collected. The next slide pivots to M6: making all of this run on a schedule.

Links

<https://cloud.google.com/dataflow/docs/concepts/dataflow-templates>
<https://cloud.google.com/dataflow/docs/guides/templates/provided-templates>
<https://cloud.google.com/dataflow/docs/guides/templates/using-flex-templates>

ELT and ETL workloads can be automated to run on a recurring basis



Google Cloud

INSTRUCTOR NOTES — PAGE 128 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Everything so far ran when you clicked — this slide is about never clicking again: the two automation shapes you will actually deploy.

Say · cover

- Shape one — scheduled ELT: a timer fires and Dataform, or plain scheduled SQL, rebuilds Silver->Gold inside BigQuery.
- Shape two — event-driven ETL: a file landing in Cloud Storage triggers processing — here a managed Spark batch job.
- Timer means 'every night at 2am'; event means 'whenever the county uploads a file.'
- Both shapes exist in your SQL Server world: SQL Agent schedules versus file-watcher triggers.
- Cheapest first rung: a BigQuery scheduled query — exactly what the closing demo does in fifteen minutes.

Iowa tie

Shape one is your Silver->Gold Dataform run on a cadence; shape two is how leftover county file feeds reach Bronze without anyone watching an inbox.

Design patterns

- Prefer event triggers for irregular arrivals and schedules for predictable rebuilds; both must end at a monitored, logged job.

Anti-patterns

- Polling loops — 'check every five minutes whether the file exists' — when an event can fire on the actual upload.

History & what's new

Dataform reached GA in June 2023 as BigQuery's native SQL workflow layer; BigQuery scheduled queries cut their minimum interval from 15 to 5 minutes in March 2024 — near-hourly ELT is now a checkbox, not a pipeline.

Flow

Automation shapes named; the next slide lays out the four services that deliver them.

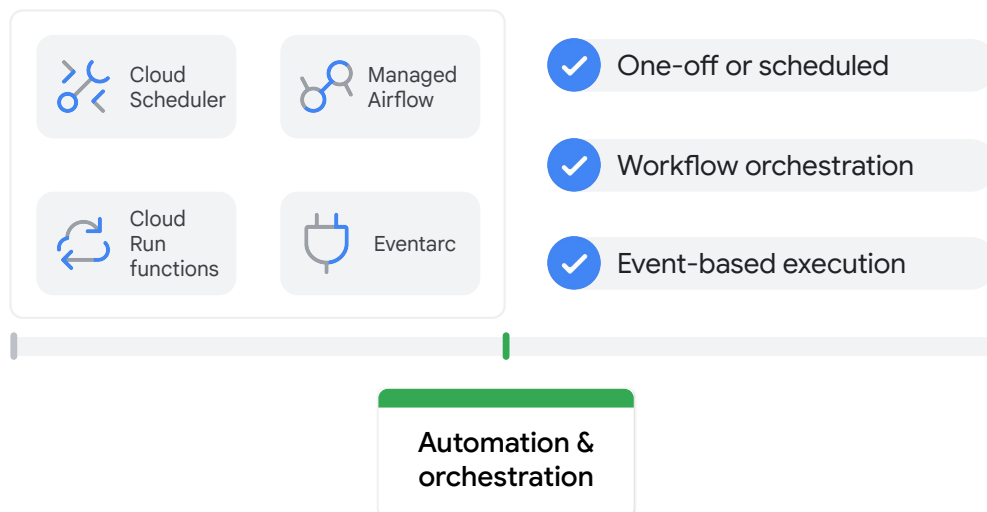
Links

<https://cloud.google.com/bigquery/docs/scheduling-queries>

<https://cloud.google.com/dataform/docs/release-configurations>

<https://cloud.google.com/products/managed-service-for-apache-spark>

Google Cloud provides multiple services for automating and orchestrating your workloads



Google Cloud

INSTRUCTOR NOTES — PAGE 129 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Four names, four jobs: the clock, the tripwire, the conductor, and the glue code — map them to SQL Agent, file watchers, your master package, and script tasks.

Say · cover

- Cloud Scheduler: one-off or recurring cron triggers — the SQL Agent analogue.
- Eventarc: event-based execution — a file uploaded or message published fires a target.
- Cloud Composer: managed Apache Airflow, the workflow orchestrator for pipelines with many dependent steps.
- Cloud Run functions: small glue code reacting to events — your script-task escape valve.
- Console: each lives under its own name; Composer environments take minutes to provision, the rest are instant.

Iowa tie

M&O will meet Scheduler first (nightly Silver->Gold), Eventarc second (file feeds), and Composer only if cross-system dependencies outgrow simple triggers.

Design patterns

- Start with the simplest trigger that meets the need; earn orchestration complexity with actual dependencies.

Anti-patterns

- Deploying Airflow on day one for three independent nightly jobs — that is cron with a control-plane tax.

History & what's new

Eventarc went GA in January 2021; Cloud Composer 3 — fully managed Airflow with no visible cluster — went GA in March 2025, and Google became the first hyperscaler offering Airflow 3 in 2025.

Flow

Landscape named; the next slide is the instructor's M&O framing before per-service detail.

Links

<https://cloud.google.com/scheduler/docs>

<https://cloud.google.com/eventarc/docs/overview>

<https://cloud.google.com/composer/docs>

M6 — for the M&O team: who keeps the jobs running

- Your instincts translate— SQL Agent schedules and master packages, one for one
- Cloud Scheduler— managed cron triggering HTTP or Pub/Sub; no scheduler VM to patch
- Scheduled queries— one SQL statement on a timer; the closing demo puts one live
- Dataform release configs— your Silver→Gold runs on a cadence, compiled from a git commit
- Composer (managed Airflow)— the orchestrator when many pipelines depend on each other
- Eventarc + Cloud Run functions— event triggers and glue code, your file-watcher analogue

Your discipline is unchanged — source control, review, staged deployment, monitoring. Only the buttons move.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 130 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

This slide is written for the people who get paged: every instinct from SQL Agent schedules and master-package operations survives — only the buttons move.

Say · cover

- Cloud Scheduler is managed cron hitting HTTP or Pub/Sub targets — no scheduler VM to patch.
- Scheduled queries: one SQL statement on a timer — the closing demo puts one live.
- Dataform release configurations run Silver->Gold on a cadence, compiled from a pinned git commit.
- Composer, managed Airflow, earns its keep when many pipelines depend on each other.
- Eventarc plus Cloud Run functions are your file-watcher and glue-code analogues.

Iowa tie

This is the migration's operating model: Dataform releases replace SSIS build deployments, Scheduler replaces SQL Agent, and your CI/CD discipline — source control, review, staged rollout — carries over unchanged.

Design patterns

- Compile releases from git commits, never from a developer's workspace — your existing CI/CD rule restated.

Anti-patterns

- Manual console clicks as the production trigger — if it is not in source control, it does not run.

History & what's new

Composer 3 went GA in March 2025 with no visible GKE cluster; BigQuery's console gained a 'Manage Airflow DAGs' preview backed by Composer 3 in October 2024.

Flow

M&O framing done; the next three slides detail Scheduler, the Workflows->Dataform trigger pattern, and the compare table.

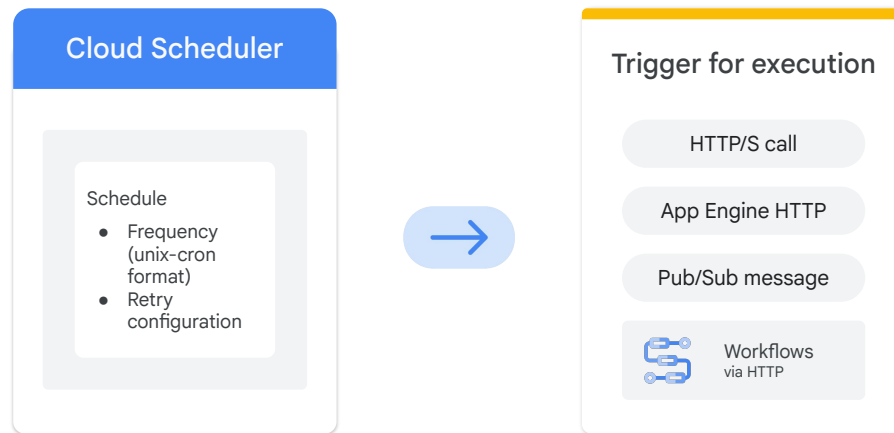
Links

<https://cloud.google.com/scheduler/docs>

<https://cloud.google.com/dataform/docs/release-configurations>

<https://cloud.google.com/blog/products/data-analytics/cloud-composer-3-for-apache-airflow>

Cloud Scheduler invokes your workloads at recurring intervals



Google Cloud

INSTRUCTOR NOTES — PAGE 131 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

If SQL Agent is your reference point, Cloud Scheduler is the same idea minus the server: cron syntax, retry policy, and Google runs the clock.

Say · cover

- A job is a unix-cron schedule plus a target — nothing else to operate.
- Targets: an HTTP/S endpoint, a Pub/Sub message, or App Engine HTTP — Workflows is reachable over HTTP.
- Retry configuration is built in — attempts and backoff are job properties, not wrapper scripts.
- Console: search 'Cloud Scheduler' -> Create job; schedule, timezone, and target type sit on one screen.
- Typical first job: fire a Workflows execution or publish 'run the nightly build' to Pub/Sub.

Iowa tie

This replaces the SQL Agent schedules firing today's extract jobs — the M&O runbook becomes: jobs defined once, retried declaratively, visible in one console list.

Design patterns

- Keep the schedule dumb — cron fires a target; dependency logic lives in Workflows or Composer, not in cron contortions.

Anti-patterns

- Encoding dependencies as staggered cron times ('job B runs 30 minutes after job A') — that is what orchestrators exist for.

History & what's new

Cloud Scheduler entered beta in November 2018 and went GA in 2019; the modern pairing is Scheduler firing Workflows (GA January 2021) for multi-step API sequences like the next slide shows.

Flow

Scheduler detailed; the next slide shows the canonical thing it triggers — a Dataform run via Workflows.

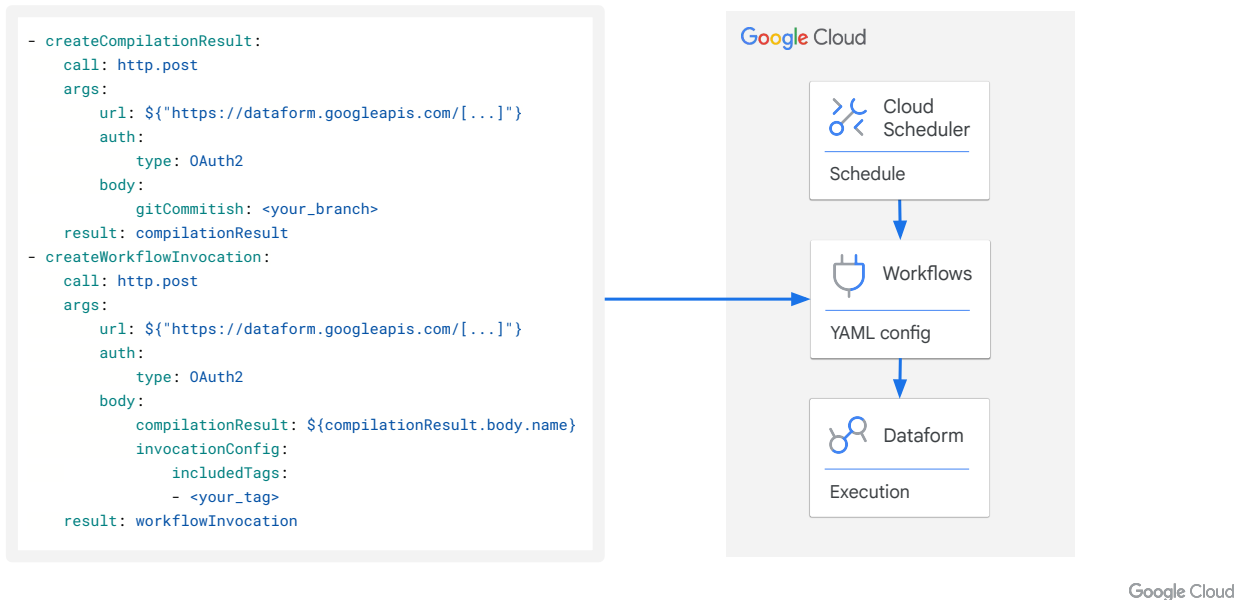
Links

<https://cloud.google.com/scheduler/docs/overview>

<https://cloud.google.com/scheduler/docs/configuring/cron-job-schedules>

<https://cloud.google.com/workflows/docs/overview>

Example: Trigger a Dataform SQL workflow



INSTRUCTOR NOTES — PAGE 132 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

This YAML is the documented production pattern for 'run my Dataform project nightly' — Scheduler fires, Workflows makes two API calls, Dataform compiles and runs.

Say · cover

- Step one calls the Dataform API to create a compilation result from your git branch — that is the build.
- Step two creates a workflow invocation from that compilation, filtered by tags — the deploy-and-run.
- OAuth2 auth is declarative: Workflows holds the service-account identity, not your script.
- Cloud Scheduler triggers this workflow on cron — the same Scheduler from the previous slide.
- Tag filtering means 'run only the daily marts' — your Silver->Gold subset, not the whole project.

Iowa tie

This is exactly how your Dataform Silver->Gold workflow runs unattended: compiled from a pinned commit so CI/CD stays intact, tag-scoped, with invocation history for M&O.

Design patterns

- Two API calls in Workflows beats scripting gcloud from a cron box — retries and auth are built in.

Anti-patterns

- Scheduling runs from a development workspace checkout — production must compile from the repository, not a laptop.

History & what's new

Workflows went GA in January 2021, just after Google's December 2020 Dataform acquisition; Dataform itself GA'd in June 2023 and gained Knowledge Catalog data-quality scorecards for workflows in October 2026.

Flow

Trigger chain complete; the next slide compresses all four automation options into one compare table.

Links

<https://cloud.google.com/dataform/docs/schedule-runs>
<https://cloud.google.com/workflows/docs/overview>
<https://cloud.google.com/scheduler/docs>

Compare automation options

	Cloud Scheduler	Managed Airflow	Cloud Run functions	Eventarc
Trigger type	schedule, manual	schedule, manual	event	event
Serverless	yes	no	yes	yes
Coding effort	low	medium	high	high
Programming languages	YAML (with Workflows)	Python	Python, Java, Go, Node.js, Ruby, PHP, .NET core	any

Google Cloud

INSTRUCTOR NOTES — PAGE 133 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

Screenshot this table for the runbook: it answers 'which service do I reach for' — and the rows tell one consistent story.

Say · cover

- Trigger type is the decider: schedule-driven (Scheduler, Composer) versus event-driven (Cloud Run functions, Eventarc).
- Serverless row: everything except Composer — it is the only option with standing infrastructure and cost.
- Coding effort rises left to right: Scheduler is YAML config; Composer means authoring Python DAGs.
- Language row matters to this team: Composer is Python-only; functions offer Python, Java, Go, Node.js and more.
- Rule of thumb: Scheduler until dependencies appear, Composer when the DAG gets real, Eventarc/functions for events.

Iowa tie

Your near-term reality is the left column — Scheduler plus Dataform releases; Composer enters only when Power BI marts depend on many upstream systems in strict sequence.

Design patterns

- Choose by trigger type first, then by dependency complexity — the table's first two rows answer most cases.

Anti-patterns

- Choosing Composer for the resume — the 'serverless: no' row means paying for an idle environment every night.

History & what's new

The Composer column keeps improving: Composer 3 (GA March 2025) hides the GKE cluster entirely, and Airflow 3 reached Composer customers in 2025 — Google was the first hyperscaler to offer it.

Flow

The compare table closes M6; the next custom slide recaps all of Day 1 before the closing demo.

Links

<https://cloud.google.com/composer/docs>

<https://cloud.google.com/eventarc/docs/overview>

<https://cloud.google.com/functions/docs/concepts/overview>

Day 1 done — what you now have

- ✓ Your data lands in BigQuery— Datastream CDC killed the nightly extract window before lunch
- ✓ The loading toolbox— console, CLI, autodetect pitfalls, Parquet vs CSV, LOAD DATA in SQL
- ✓ The Silver→Gold transform— a working Dataform SQL workflow from this afternoon's lab
- ✓ The automation map— Scheduler ≈ SQL Agent, Composer ≈ orchestrator, releases compiled from git
- ✓ Tomorrow— lakehouse and the medallion architecture: Bronze/Silver/Gold made formal, then Power BI

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 134 OF 135 · 16:05 · MODULES 5+6 — ETL & AUTOMATION (COMPRESSED)

Hook

One-sentence recap: your data lands in BigQuery by CDC, your transforms are versioned SQL, and in ten minutes one of them will run itself on a schedule.

Say · cover

- Datastream CDC replaced the nightly extract window this morning — Bronze fills itself.
- The loading toolbox: console, CLI, autodetect pitfalls, Parquet versus CSV, LOAD DATA in SQL.
- This afternoon's Dataform lab is the Silver->Gold engine — SQL workflows living in git.
- The automation map: Scheduler ~ SQL Agent, Composer ~ orchestrator, releases compiled from git.
- Tomorrow formalizes Bronze/Silver/Gold as the medallion lakehouse, ending at Power BI.

Iowa tie

Every Day-1 block maps to your migration: kill-the-extracts (Datastream), Bronze loading, Silver->Gold (Dataform), M&O automation — Day 2 adds governance and the Power BI last mile.

Design patterns

- Recap discipline: each block today produced one working artifact — a loaded table, a stream, a workflow, a schedule.

Anti-patterns

- Letting Day 1 stay slideware — the optional Spark homework and tomorrow's quiz are what make it stick.

History & what's new

The closing demo's scheduled queries became meaningfully more useful in March 2024, when their minimum interval dropped from 15 to 5 minutes (GA) — near-hourly marts without any pipeline.

Flow

Recap done — next is the 16:20 closing demo: schedule the derived-table query live, then the Day 2 preview.

Links

<https://cloud.google.com/bigquery/docs/scheduling-queries>
<https://cloud.google.com/datastream/docs/overview>
<https://cloud.google.com/dataform/docs/overview>

Closing demo — schedule the derived-table query

10 minutes — 16:20–16:30 · BigQuery Studio — instructor-led; replicate it after class

1. Run the sales-by-zip query once on the roi-bq-demos nested table — note bytes processed
2. Above the editor: Schedule → Create new scheduled query
3. Name it; repeat daily at 12:01 AM; destination class.march_zip_sales
4. Write preference: Overwrite table (WRITE_TRUNCATE) — rebuilt nightly, idempotent re-runs
5. Save, open Scheduled queries, and use the manual-run option once to prove it fires

Why it matters — your nightly SQL Agent job, translated: SQL on a schedule overwriting a table — no scheduler box, no agent service, no SSIS package.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 135 OF 135 · 16:20 · DEMO — SCHEDULED QUERY, THE CLOSING DEMO (10 MIN)

Hook

Your nightly SQL Agent job in one click — SQL on a schedule overwriting a table: no scheduler box, no agent service, no SSIS package.

Say · cover

- Run the sales-by-zip query once; note bytes processed — the cost case for materializing what Power BI hammers.
- Schedule -> Create new scheduled query; it rides on the BigQuery Data Transfer Service — Enable if prompted.
- Daily 12:01 AM -> class.march_zip_sales; the destination dataset must already exist.
- Overwrite table = WRITE_TRUNCATE: rebuilt nightly, idempotent re-runs; Append accumulates history.
- Manual-run once to prove it fires. It runs as you — in production, a service account: your SQL Agent proxy instinct.
- Bridge: hand-rolled derived tables go stale and multiply — tomorrow Dataform governs the same idea.

Iowa tie

The Gold-mart cost story in miniature, and the M&O team's first serverless scheduled job — the floor of their automation ladder.

Design patterns

- Materialize hot aggregations once; stop re-scanning base tables
- WRITE_TRUNCATE for rebuilt snapshots — idempotent by construction
- Production schedules run as a service account

Anti-patterns

- Accidental Append doubling the table nightly
- Power BI hammering base tables
- Ungoverned hand-rolled derived tables multiplying

History & what's new

Scheduled queries entered beta in October 2018. March 2025: the unified Scheduling page went GA, managing scheduled queries alongside notebooks, pipelines, and data preparations.

Flow

Closes Day 1 after the M5+M6 block; tomorrow opens with the recap quiz, then medallion architecture formalizes Bronze/Silver/Gold.

Links

<https://cloud.google.com/bigquery/docs/scheduling-queries>

<https://roitraining.github.io/gcp-demos/bigquery.html>

<https://docs.cloud.google.com/bigquery/docs/release-notes>